Reasoning is compute, search, and verification: the inference-time triangle in frontier LLMs, 2022–2026

Theory KB — Paper 3 of 5

2026-05-04 (draft)

Abstract

Reasoning gains 2022–2026 are a triangle of (a) inference-time compute scaling (chain-of-thought, self-consistency, best-of-N), (b) verifier signals at inference (process reward models, MCTS, tree search), and (c) RL on verifiable rewards using those signals (RLVR, GRPO, DAPO). Strong reasoning models co-design all three. Isolating any one understates the gains; pretending reasoning is one mechanism produces the wrong predictions about scaling.

Contents

1 Introduction: the reasoning triangle **3**
- 1.1 What this paper does, section by section . . . 5
- 1.2 The open question this paper forwards . . . 6

2 The CoT phenomenon and its scale-emergence **7**
- 2.1 The decoding factorisation . . . 7
- 2.2 Few-shot CoT (Wei et al. 2022) . . . 8
- 2.3 Zero-shot CoT (Kojima et al. 2022) . . . 8
- 2.4 Emergence with scale . . . 9
- 2.5 The post-2024 reframing: prompting trick vs. compute axis . . . 9
- 2.6 Forward link to faithfulness . . . 10
- 2.7 What this section sets up . . . 10

3 Self-consistency: sampling and majority vote **11**
- 3.1 The mechanism . . . 11
- 3.2 Why it works . . . 12
- 3.3 Headline results . . . 12
- 3.4 The baseline interpretation . . . 13
- 3.5 When self-consistency fails . . . 14
- 3.6 Forward link . . . 14

4 Inference-time compute as a scaling axis **15**
- 4.1 The compute-optimal test-time strategy . . . 15
- 4.2 The proposer/verifier decomposition . . . 16
- 4.3 Question difficulty as a sufficient statistic . . . 17
- 4.4 The 4× compute-optimal vs. best-of-N gain . . . 18
- 4.5 The FLOPs-matched comparison: 14× pretraining substitution . . . 18

4.6 Tensor-shape and operational notes . . . 19
4.7 The functional form of $L(\theta, N)$ is open . . . 20
4.8 Cross-paper and forward links . . . 21

5 Process reward models: the verifier leg **21**
5.1 What a process reward model commits to . . . 21
5.2 PRM800K: the canonical process-supervision experiment . . . 22
5.3 Math-Shepherd: removing the human-annotation bottleneck . . . 23
5.4 ThinkPRM: process reward models that think . . . 24
5.5 R-PRM and EDU-PRM: the 2025 PRM lineage . . . 24
5.6 The 2025 PRM survey landscape . . . 25
5.7 PRM utility post-R1: the open contradiction . . . 25
5.8 Forward link . . . 26

6 Tree-search families: BFS, DFS, MCTS **26**
6.1 The search tree as a datastructure over partial traces . . . 27
6.2 Beam search and BFS-V over reasoning steps . . . 27
6.3 The MCTS recurrence (rStar-Math) . . . 28
6.4 Tensor-shape and operational notes . . . 30
6.5 ReST-MCTS*: AlphaZero-shaped self-training . . . 30
6.6 MCTS-RAG: tree search over retrieval and reasoning . . . 31
6.7 Search-vs-trained-CoT: the live contradiction . . . 31
6.8 Forward link . . . 32

7 The RLVR paradigm: GRPO and DAPO **33**
7.1 The verifiable reward . . . 33
7.2 The GRPO objective . . . 34
7.3 DAPO: four targeted refinements . . . 35
7.4 Empirical headline: R1-Zero and DAPO . . . 37
7.5 Open contradiction: VAPO — are value models really useless? . . . 38
7.6 Forward link . . . 38

8 DeepSeek-R1 and the reasoning-trained model family **39**
8.1 The four-stage R1 protocol . . . 39
8.2 R1-Zero versus R1: what does cold-start add? . . . 41
8.3 The long-CoT compounding curve . . . 41
8.4 The distillation track and the open-weight ecosystem . . . 43
8.5 The AIME-2024 contamination contradiction . . . 43
8.6 Forward link . . . 44

9 Test-time scaling beyond CoT: think-tokens, budget forcing, reasoning architectures **45**
9.1 Thinking-token budget L . . . 45
9.2 The s1 budget-forcing rule . . . 46
9.3 Logarithmic-ish returns to L . . . 46
9.4 Inference-time discipline vs. training-time discipline . . . 47
9.5 Reasoning-architecture as a third axis . . . 48
9.6 Forward link . . . 49

10 Search-vs-RL under fixed compute **49**
10.1 The three-channel allocation . . . 49
10.2 Snell's pretraining-vs-inference slice . . . 50
10.3 The RL-vs-inference slice . . . 50
10.4 The identity that ties the channels . . . 51
10.5 Where the curves cross is benchmark-specific . . . 52
10.6 Operating-point heuristics from the anchor anecdotes . . . 53
10.7 Cross-paper and forward links . . . 54

11 CoT faithfulness: does the trace cause the answer? **54**
11.1 Operational definition: from conditional to causal . . . 55
11.2 Arcuschin et al.: CoT in the wild is not always faithful . . . 56
11.3 Shen et al. FaithCoT-Bench: instance-level measurement . . . 56
11.4 Mehta et al.: does inference scaling improve faithfulness? . . . 57
11.5 The contradiction with reasoning-training framings . . . 57
11.6 Cross-paper consequences: probe surface and monitor channel . . . 58
11.7 Forward link: does the verifier signal still matter? . . . 59

12 Process supervision after R1: do PRMs still help? **59**
12.1 R1 is the data point . . . 60
12.2 Inference-time PRMs on reasoning-trained policies . . . 60
12.3 Training-time PRMs after R1: where the contradiction lives . . . 61
12.4 Process-vs-outcome RL: when does the per-step signal pay? . . . 62
12.5 What the post-R1 evidence actually says . . . 63
12.6 Forward link . . . 64

13 Frontier-2026 snapshot: reasoning protocols **64**
13.1 The 2-page comparison table . . . 64
13.2 R1-style: long-CoT + RLVR + verifier-only . . . 65
13.3 o-series: RL-trained thinking with proprietary mix . . . 66
13.4 Hybrid think / non-think with productionised budget knobs . . . 66
13.5 Where the lineup agrees . . . 67
13.6 Where the lineup diverges . . . 68
13.7 Cross-paper hook and forward link . . . 69

14 Contradictions live and open frontiers **69**
14.1 The six contradictions . . . 70
14.2 What closure would require . . . 72
14.3 What is settled . . . 73
14.4 Closing thesis . . . 73

Glossary **74**

1 Introduction: the reasoning triangle

Between the late-2022 release of et al. (Google Brain) and the early-2025 release of DeepSeek-AI (2025a), the publicly reported pass@1 of an open-weight model on AIME 2024 — a contest of fifteen graduate-style mathematics problems — moved from indistinguishable-from-noise to 77.9%, and

to 86.7% under majority vote over 64 samples (DeepSeek-AI, 2025a, §2.3).[1] The same checkpoint, DeepSeek-R1-Zero, started its reinforcement-learning phase from a 15.6% baseline on the same benchmark (DeepSeek-AI, 2025a, §2.3).[2] The size of the jump — and the comparable jumps reported by OpenAI's o1, Google's Gemini 2.5-thinking, Anthropic's Claude 3.7 extended-thinking mode, Alibaba's QwQ, and the Qwen3-thinking series in the months following — is the clearest empirical event in language-model research between 2022 and 2026. It is also the event around which the literature is most prone to one-leg explanations: "the model thinks in chains"; "a verifier ranks its samples"; "a reinforcement-learning loop trains it." Each one-leg story is partially right and structurally incomplete.

The thesis of this paper is that the 2022–2026 reasoning gains are best understood as a *triangle of co-designed levers*. The three legs:

1. **Inference-time compute** — chain-of-thought (et al. , Google Brain; Kojima et al., 2022), sampling-based aggregation (Wang et al., 2022), and the elevation of test-time compute from prompting trick to scaling axis by Snell et al. (2024).[3]

2. **Verifier signals** — process reward models that score partial reasoning at step granularity (Lightman et al., 2023) and the tree-search families (beam, BFS, MCTS) that turn those scores into search trajectories (et al. , Microsoft).

3. **Reinforcement learning on verifiable rewards (RLVR)** — group-relative policy optimisation over programmatically-checkable outcomes (Shao et al., 2024), with DAPO's four targeted modifications (Seed, 2025) as the current open-recipe state of the art.

Frontier reasoning models in 2026 ship all three. They differ in mixture, in budget, in disclosure, and in which leg is treated as the primary product surface (R1 ships the long-CoT trace; o1 ships a hidden reasoning channel and an exposed effort knob; Gemini 2.5 exposes a literal `thinkingBudget`). They do not, as of 2026-Q2, differ in shipping fewer than three legs. Treating any one leg in isolation reproduces an incomplete reading of the headline gains, of which mechanism the gains generalise from, and of which open questions remain.

Intuition

A useful one-line picture is that the three legs are dual to each other across the train/inference boundary. RLVR *shapes* a policy whose long-CoT output is worth amplifying; inference-time compute *amplifies* that policy by spending more sampling, search, or trace-length budget; verifier signals *rerank* the amplified trajectories. Pull any one leg and the other two lose a lot of their explanatory power: long-CoT without an RL phase that taught the model to use it is short-CoT in a costume; a strong verifier without a proposer that produces diverse-and-mostly-right candidates does not have anything to verify; an RL loop without a verifiable reward collapses back to RLHF and its 2023-vintage pathologies. Returning to canonical math: the GRPO objective transcribed in section 7.2 is structurally a re-weighted maximum-likelihood update over verifier-passed trajectories sampled from the current policy, with KL anchoring to a reference. All three legs appear in that single objective.

The triangle framing also locates this paper relative to two of its companions. Paper 2 establishes a six-stage post-training pipeline in which RLHF is succeeded by RL-on-verifiable-rewards as a

[1] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime`.
[2] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime`.
[3] KB excerpt: `kb/excerpts/snell2024.md#sec-1-flops-matched`.

distinct stage, with different load-bearing variables (programmatic verifier instead of learned reward, group baseline instead of value network). That cross-reference is the inheritance route: section 7 deepens Paper 2's RLVR treatment in the reasoning-specific direction (long traces, terminal verifiers, R1-style protocols). Paper 1 is the architectural precondition: the 10K-token reasoning traces that section 8 reports as the operational signature of an RL-trained reasoning model are tractable only because Paper 1's long-context attention (sliding-window, FlashAttention-2/3, KV-compression) makes their KV cache survivable.[4] Without those two predecessors the triangle's three legs are a vocabulary; with them, they are a co-designed system.

1.1 What this paper does, section by section

The body of the paper walks the triangle from each leg's historical entry point through its anchor mechanisms to the live contradictions the literature has not yet closed.

Section 2 starts with the phenomenon: et al. (Google Brain) and Kojima et al. (2022) on chain-of-thought as a few-shot and zero-shot prompting recipe, the decoding factorisation $z \sim p_\theta(\cdot \mid x), y \sim p_\theta(\cdot \mid x, z)$ that every later test-time-compute strategy in the paper inherits, and the emergence-with-scale finding that distinguished CoT from compute-monotonic prompting tricks. Section 3 continues with the cheapest extension: Wang et al. (2022)'s sample-N-and-majority-vote, no verifier needed — still a competitive baseline well into the R1 era, where it lifts R1-Zero's AIME 2024 from 77.9% to 86.7% (DeepSeek-AI, 2025a, §2.3).

Section 4 is the first anchor section: the Snell et al. (2024) reframing of test-time compute as a scaling axis with its own functional form, comparable to Chinchilla on the training side. The compute-optimal definition (Eq. 1 of Snell et al. (2024)), the proposer/verifier decomposition that unifies every other section of the paper, and the FLOPs-matched headline that test-time compute substitutes for $\sim$14$\times$ pretraining FLOPs on easy and intermediate prompts but not on the hardest tail, all live there.

Section 5 formalises the verifier leg. Lightman et al. (2023)'s process reward model is the canonical artefact; the four trace-aggregation rules (min, product, last-step, weighted-mean) and the headline that min-aggregation is strongest are the load-bearing mechanics. The post-2023 PRM lineage — Math-Shepherd on automated step labelling, ThinkPRM and R-PRM on generative verifiers that themselves reason — closes the section. Section 6 is the second anchor section: the search-tree datastructure over partial reasoning prefixes, the canonical reasoning-MCTS recurrence transcribed from et al. (Microsoft), the four MCTS phases (selection, expansion, simulation, backup), and ReST-MCTS* as the AlphaZero-style outer loop in which MCTS rollouts harvest correct trajectories that re-train the proposer and the verifier.

Section 7 is the third anchor section and the third triangle leg. The verifier reward $R_{\text{verifier}}(x, y) = \mathbb{1}[\,\text{verify}(x, y) = \mathsf{true}\,]$, the GRPO objective (Shao et al., 2024, §4.1) with group-relative advantage replacing a value network, and DAPO's four targeted modifications (Seed, 2025, §2) are the section's mechanics. The empirical proof point that section 8 unfolds in detail — DeepSeek-R1-Zero's continuous GRPO climb — is the cleanest single number in the literature for the proposition that the three legs compose under one training loop.

Section 9 treats test-time scaling that does not lean on parallel sampling or search: et al. (Stanford+)'s minimum recipe of 1,000 long-CoT exemplars plus a budget-forcing rule that appends `Wait` to extend thinking, isolating the *reasoning-architecture* axis (the `<think>...</think>` phase as a structurally distinct decode-time region) from the base-architecture axis. Section 10 confronts

[4] The KV-cache cost of a 10^4-token trace at full $\mathsf{B} \times \mathsf{H} \times \mathsf{S} \times d_h$ shape is non-trivial. The architectural choices of Paper 1 are what determine whether a frontier reasoning model is feasible as a deployment artefact, not only as a research curiosity.

the operational tradeoff the triangle implies: at fixed total compute, when does the marginal FLOP go to inference search, to RLVR training, or to pretraining? The honest answer in 2026 is benchmark-specific and base-model-specific; that section transcribes what is settled and demarcates what is anecdotal.

Section 11 addresses the most consequential live open question: does the trace z causally drive the answer y, or is it post-hoc rationalisation that happens to be human-readable? The 2025–2026 wave (Arcuschin et al. on naturalistic prompts, Shen et al. on FaithCoT-Bench, Mehta et al. on whether faithfulness improves with scale) finds substantial unfaithfulness on naturalistic distributions; whether it improves with scale is contested. Section 12 returns to the verifier leg with a sharper question: post-R1, do PRMs still pay for themselves? R1's headline result was achieved with terminal verifier rewards only, and the 2025–2026 PRM literature is split between diminishing-returns findings and continued-gain findings.

Section 8 sits between section 7 and section 9 as the empirical proof point: the four-stage R1 protocol (cold-start SFT → GRPO with verifiable rewards → rejection-sampling SFT on harvested traces → general-domain RL), the R1-Zero ablation that strips the cold-start to isolate what the SFT initialisation contributes, and the open-weight distillation track that produced the post-R1 reasoning ecosystem (R1-Distill-Qwen-7B/14B/32B, the Tülu 3 lineage, OpenR1, QwQ, Qwen3-thinking).

The final two sections close the loop. Section 13 catalogues which 2026-frontier models ship which combination of legs at what budget; the table is intended as the most-screenshotted figure of the paper. Section 14 concentrates the [CONTRADICTION] markers from section 2–section 12 into one place and forecasts which of them are likely to close in the next model generation.

1.2 The open question this paper forwards

The thesis commits the paper to one position — the legs are co-designed in the strong sense, not three independent gains that happen to ship together — and one open question that section 14 returns to:

Is the triangle a real co-design, or three independent gains we happen to ship together?

The strong-co-design claim is that the legs interlock causally: RLVR without long-CoT sampling cannot find the signal that lengthens traces, long-CoT without RLVR is mechanistically possible but empirically weaker, and verifier signals are what convert sampling diversity into a reranked answer that propagates training reward. The independent-gains claim is that the three legs each work in isolation and the field has co-shipped them only for engineering reasons. The 2025 various (2025) result — RLVR sharpens pass@1 within base-model competency but does not expand pass@K — sits exactly on this fault line: under the strong-co-design reading, RLVR is doing the right thing (it shapes a policy whose sampling and verification are then meaningful); under the independent-gains reading, RLVR is doing less than the headline numbers suggest because the underlying capability already exists in the base model and only the trace-length needed to express it is being elicited. Section 14 catalogues five further contradictions on the same axis (PRM utility post-R1, CoT faithfulness scaling, search-vs-RL tradeoff geometry, AIME contamination, MCTS-vs-beam at matched budget). The paper's position is that none of them are closed as of 2026-Q2; the engineering question of how to ship a frontier reasoning model is, by contrast, well past the point where any single leg is optional.

The remainder of the paper makes that position defensible at mechanism, equation, and empirical-benchmark granularity. We begin in section 2 with the historical entry point that every later leg of the triangle inherits: the chain-of-thought decoding pattern.

2 The CoT phenomenon and its scale-emergence

The historical entry point of every leg of the reasoning triangle is a single decoding pattern: instead of mapping a problem x directly to an answer y, the model first produces an intermediate *reasoning trace* z and only then commits to y. et al. (Google Brain) introduced this pattern as a few-shot prompting recipe under the label **chain-of-thought (CoT)** prompting, with the headline finding that an 8-exemplar CoT prompt raises a 540B-parameter language model on the GSM8K math word-problem benchmark to state-of-the-art, surpassing a fine-tuned GPT-3 with a verifier (et al. , Google Brain, abstract).[5] The same pattern was shown a few months later to elicit competitive reasoning under *zero* hand-written exemplars, by appending the literal trigger phrase *"Let's think step by step"* before answer extraction (Kojima et al., 2022, abstract).[6] The 2022 community read both results as prompting tricks. The 2024–2026 reframing — anchored in Snell et al. (2024) and the inference-time-compute scaling literature that section 4 formalises — reads them as the first empirical instance of *trading inference compute for parameters*. This section walks the original mechanism, transcribes the decoding factorisation that every later test-time-compute strategy in the paper will build on, presents the emergence-with-scale finding that distinguished CoT from compute-monotonic prompting, and closes by deferring CoT's faithfulness question to section 11.

2.1 The decoding factorisation

Let x denote the problem and y the desired final answer. Direct prompting decodes $y \sim p_\theta(\cdot \mid x)$ in a single step. CoT prompting introduces an intermediate variable z — the reasoning trace — and decodes a two-stage factorisation (et al. , Google Brain, §3.1):[7]

$$z \sim p_\theta(\cdot \mid x), \qquad y \sim p_\theta(\cdot \mid x, z). \tag{1}$$

In practice the trace and answer are emitted jointly by autoregressive decoding of a single concatenated sequence $z \cdot y$, where z is a natural-language explanation terminating in a stable answer-extraction template (e.g. *"The answer is* $\langle\cdot\rangle$*"*) and y is recovered post-hoc by a regex on that template (et al. , Google Brain, §3.1). The KB note for this paper records the factorisation as Eq. 1 of `chain-of-thought.md`.[8]

Tensor shapes and the trace-length budget. The trace z in eq. (1) is a token sequence whose length empirically spans two orders of magnitude across protocols and benchmarks: the original GSM8K few-shot CoT exemplars are on the order of 10^2 tokens per trace (et al. , Google Brain, §3.1), while the long-CoT reasoning models of section 8 routinely produce 10^3–10^4 tokens of z per problem (DeepSeek-AI, 2025a, §2.3). The answer y remains the same shape it was under direct prompting (a few tokens). The implication for accounting is structural: under eq. (1) the inference-time compute and KV-cache budget are dominated by z, not by y. Decoding a single CoT response of shape $\mathsf{S} = |z| + |y| \approx |z|$ at hidden dim D over L layers carries a per-layer KV-cache cost $O(\mathsf{S} \cdot \mathsf{D})$ that is the binding constraint long before the model's parameter count is. The architectural prerequisites for tractable long traces — long-context attention, KV compression, flash attention — are the subject of Paper 1 and re-enter this paper in section 8 as the precondition for 10^4-token z.

[5] KB excerpt: `kb/excerpts/wei2022-cot.md#abstract`.
[6] KB excerpt: `kb/excerpts/kojima2022.md#abstract`.
[7] KB excerpt: `kb/excerpts/wei2022-cot.md#sec-3-1`.
[8] KB note: `kb/notes/reasoning/chain-of-thought.md#1`.

Few-shot exemplar count K. The CoT prompt in et al. (Google Brain) is a sequence of K in-context demonstrations $\{(x_i, z_i, y_i)\}_{i=1}^{K}$ followed by the query x_*, with each z_i a hand-written reasoning trace. The arithmetic-reasoning benchmarks in et al. (Google Brain) use $K = 8$ (et al. , Google Brain, §3.1).[9] The exemplars do double duty: they (i) shift the model's output distribution toward sequences in which z precedes y — the format-conditioning role — and (ii) supply task-specific reasoning style. Kojima et al.'s zero-shot result below shows the format-conditioning role does not in fact require $K > 0$, suggesting the exemplars' marginal contribution at sufficient scale is style, not format.

2.2 Few-shot CoT (Wei et al. 2022)

The original recipe is the simplest version of eq. (1): construct a prompt of $K = 8$ exemplars with hand-written reasoning traces, decode greedily (or at low temperature) from $p_\theta(\cdot \mid \text{prompt}, x_*)$ until the stop pattern, and parse the trailing *"The answer is ⟨·⟩"* suffix to recover y_* (et al. , Google Brain, §3.1). et al. (Google Brain) apply this protocol to three model families (LaMDA, GPT-3, PaLM) at sizes spanning roughly 0.4B to 540B parameters, evaluated on three task families: arithmetic word problems (GSM8K, SVAMP, ASDiv, AQuA, MAWPS), commonsense (CSQA, StrategyQA, Date, Sports), and symbolic reasoning (Last Letter, Coin Flip) (et al. , Google Brain, §3.2).[10] The headline *single-number* result is GSM8K at the 540B scale: an 8-exemplar CoT prompt achieves state-of-the-art accuracy, surpassing a fine-tuned GPT-3 supplemented with a verifier (et al. , Google Brain, abstract).[11]

2.3 Zero-shot CoT (Kojima et al. 2022)

Kojima et al. (2022) report that the entire CoT-eliciting effect can be recovered with $K = 0$ — no hand-written exemplars at all — by appending the literal string *"Let's think step by step"* to the prompt. The protocol is two-stage (Kojima et al., 2022, §3):[12]

1. **Reasoning extraction.** Prompt the model with x_* + *"Let's think step by step"* and decode the trace z_*.
2. **Answer extraction.** Prompt the model with x_* + z_* + *"Therefore, the answer is"* and decode y_*.

The two-stage form is the canonical protocol; the trigger phrase is *"Let's think step by step"*, identified empirically among candidate variants by Kojima et al. (2022) as the strongest cue. The headline numbers (Kojima et al., 2022, headline)[13] are striking for a purely prompting-side change: with InstructGPT (`text-davinci-002`), MultiArith rises from 17.7% to 78.7% and GSM8K from 10.4% to 40.7%, with comparable gains on PaLM-540B.

The structural consequence: at sufficient scale the format-conditioning role of Wei's $K = 8$ exemplars is already supplied by pretraining and instruction-tuning, and a five-word trigger reaches into that latent capability without a single demonstration (Kojima et al., 2022, §5).[14] The reasoning content is not absent in the smaller-prompt setting; it is just not elicited.

[9] KB excerpt: `kb/excerpts/wei2022-cot.md#sec-3-1`.
[10] KB excerpt: `kb/excerpts/wei2022-cot.md#sec-3-2`.
[11] KB excerpt: `kb/excerpts/wei2022-cot.md#abstract`.
[12] KB excerpt: `kb/excerpts/kojima2022.md#sec-3`.
[13] KB excerpt: `kb/excerpts/kojima2022.md#headline`.
[14] KB excerpt: `kb/excerpts/kojima2022.md#sec-5`.

Intuition

"Let's think step by step" is a soft retrieval cue, not a reasoning module. Pretraining text — math tutorials, instructional material, worked examples — contains many instances of that exact phrase preceding step-by-step explanations, so appending it shifts the conditional distribution $p_\theta(z, y \mid x_* + \text{cue})$ toward sequences in which a trace precedes the answer. Returning to math: the cue does not change the family eq. (1); it only nudges $p_\theta(\cdot \mid x)$ to put mass on z-then-y continuations rather than on direct-answer continuations. The mechanism is a learned co-occurrence (Kojima et al., 2022, §5), not a discovered reasoning faculty.

2.4 Emergence with scale

The single most consequential empirical claim of et al. (Google Brain) for the rest of this paper is that the gain from CoT prompting over direct prompting is itself an *emergent* property of model scale. The headline figure (et al. , Google Brain, §3.3, fig. 4)[15] shows GSM8K accuracy plotted against model size for three families (LaMDA, GPT-3, PaLM), each with and without the 8-exemplar CoT prompt. Two qualitative features survive every replication:

- **No gain — sometimes a loss — below ∼100B parameters.** Below the inflection, CoT prompting often *underperforms* direct prompting on GSM8K. The model emits a trace whose internal arithmetic is brittle, and the trace's errors propagate into the answer.
- **Substantial gain at PaLM-540B.** Above the inflection, CoT prompting opens a sharp gap over direct prompting — the gap that produces the abstract's headline of state-of-the-art GSM8K (et al. , Google Brain, abstract).

et al. (Google Brain, §6) explicitly frame this in their limitations discussion: the technique's benefit is not a uniform multiplicative factor over model scale but a regime-dependent effect that activates only past a problem-dependent capacity threshold.[16] The same paper hedges that "chain-of-thought" need not reflect any cognitive sense of "reasoning" — a hedge that the 2025 faithfulness literature (section 11) makes load-bearing.

2.5 The post-2024 reframing: prompting trick vs. compute axis

Reading the 2022–2023 reception of CoT in retrospect, the field largely treated eq. (1) as a prompting innovation: a clever demonstration of in-context learning, evaluated relative to zero-shot or few-shot baselines, on benchmarks chosen to expose *prompting* differences. The compute axis was implicit but unaccounted-for. Wei's 8-exemplar CoT prompt at 540B emits roughly 10^2 tokens of z per problem; the direct-prompting baseline emits roughly 10^0. The protocols differ by two orders of magnitude in inference compute per problem, and the GSM8K gap is the joint product of *(i)* the model's latent capacity to use that compute and *(ii)* the prompt that elicited the trace.

Snell et al. (2024) promote this reading explicitly: test-time compute becomes a *scaling-law-shaped* axis with its own functional form, on which CoT prompting was the first attested operating point. The emergence-with-scale finding of et al. (Google Brain, §3.3) is then re-readable as the joint shape of the inference-compute / parameter-count surface — sub-100B models under eq. (1) cannot convert the extra inference compute into accuracy because their per-token reasoning step is

[15] KB excerpt: `kb/excerpts/wei2022-cot.md#sec-3-3`.
[16] KB excerpt: `kb/excerpts/wei2022-cot.md#sec-6`.

too noisy; PaLM-540B can. The 2025–2026 reasoning-trained models of section 8 push the operating point further along the same axis: z grows from 10^2 to 10^4 tokens, the model is RL-trained to use that budget, and the gain over a 10^0-token direct answer becomes the dominant accuracy lever on hard math benchmarks. CoT was not magic prompting; it was the first published instance of substituting inference compute for parameters along an axis the field would later formalise.

Analogy

CoT is to direct prompting what stochastic gradient descent with K inner-loop steps is to a single gradient step: the same model now spends additional compute per query before committing to an output, and the marginal accuracy returned by that compute is the load-bearing quantity. Returning to math: the analogy collapses to eq. (1) — direct prompting collapses z to a length-zero trace ($p_\theta(y \mid x)$ alone), CoT prompting decodes a non-trivial $|z| > 0$, and section 4 treats $|z|$ together with the sample count N of section 3 as orthogonal compute knobs whose joint optimisation is the inference-side counterpart of training-side scaling laws (Snell et al., 2024, §1).

2.6 Forward link to faithfulness

A question eq. (1) raises but does not settle: *does z in fact cause y?* The factorisation is well-defined as a generative model — the trace is sampled, the answer is sampled conditional on the trace — but the conditional dependence $y \sim p_\theta(\cdot \mid x, z)$ is not the same as a causal claim that perturbations to z propagate to y. The 2025–2026 faithfulness literature (Arcuschin et al., 2025; et al., 2025b, 2026) measures this directly and finds the answer is regime-dependent.

Contradiction

Reasoning vs. rationalisation. eq. (1) treats z as a sampled latent that conditions y, but the operational question is whether perturbations to z that should change y in fact do. Production non-thinking models exhibit substantial post-hoc rationalisation rates (on the order of 7–13% on et al. (2026)'s measurement); thinking models exhibit rates two orders of magnitude smaller (et al., 2026, §4).[a] Whether the gap is caused by long-CoT SFT, by RLVR post-training, or by general scale is not separated by the published 2026 ablations. The hedge that et al. (Google Brain, §6) themselves placed on the cognitive interpretation of CoT — "chain-of-thought does not necessarily reflect model reasoning in any cognitive sense" — has aged into the load-bearing question of section 11, which catalogues the 2025–2026 evidence and the open ablations needed to settle it.

[a] KB note: `kb/notes/reasoning/chain-of-thought.md#4.1`.

2.7 What this section sets up

The decoding factorisation eq. (1) is the substrate every remaining section of this paper builds on. Section 3 takes the cheapest leg of the inference-time compute triangle by sampling eq. (1) N times at $T > 0$ and majority-voting the extracted answers, with no verifier or training change; the gain is the simplest demonstration that the per-sample noise in z is recoverable when the answer space is low-cardinality. Section 4 promotes both the trace length $|z|$ and the sample count N from prompting choices to a joint compute axis, transcribes Snell et al. (2024)'s compute-optimal selection problem, and presents the FLOPs-matched headline that on easy and intermediate prompts

test-time compute substitutes for ∼14× more pretraining FLOPs. Section 5 adds the verifier leg by scoring per-step prefixes of z, and sections 7 and 8 add the RL leg by training p_θ so that the trace distribution itself concentrates on correct answers. The CoT phenomenon, in et al. (Google Brain)'s 2022 form, was the first leg of this triangle to appear in the literature; the rest of the paper traces what happens when the field stops reading it as a prompting trick and starts treating it as a compute axis with its own scaling law.

3 Self-consistency: sampling and majority vote

Wang et al. (2022) replace the greedy decoder of section 2 with the cheapest non-trivial inference-time compute strategy in the reasoning literature: sample N chain-of-thought traces independently at temperature $T > 0$, extract the final answer from each, and return the one that appears most often. No verifier is trained, no search tree is expanded, no model parameter is changed. Despite this minimality, self-consistency lifts greedy CoT by **+17.9%** on GSM8K, **+11.0%** on SVAMP, and **+12.2%** on AQuA (Wang et al., 2022, abstract).[17] The result reframes the basic CoT decoder as one point on a sampling-budget axis, and supplies the baseline against which every later test-time compute strategy in this paper — best-of-N with verifier (section 5), tree-search (section 6), RLVR-trained long traces (sections 7 and 8) — must show its incremental value.

3.1 The mechanism

Let x be a problem, z a reasoning trace, and y the final answer extracted from z by a regex on the trailing *"The answer is ⟨·⟩"* pattern (et al. , Google Brain, §3.1). The CoT decoder of eq. (1) samples a single trace, $z \sim p_\theta(\cdot \mid x)$, and answers $y \sim p_\theta(\cdot \mid x, z)$. Self-consistency replaces this single draw with N independent draws and aggregates by plurality (Wang et al., 2022, §3):[18]

$$\hat{y} \;=\; \arg\max_{y} \sum_{n=1}^{N} \mathbb{1}[y^{(n)} = y], \qquad z^{(n)} \sim p_\theta(\cdot \mid x), \;\; y^{(n)} \sim p_\theta(\cdot \mid x, z^{(n)}). \tag{2}$$

Sampling is at temperature $T > 0$ (top-k or nucleus optional); the value $T = 0.7$ used by Wang et al. (2022, §4.1) on GSM8K is a recurring default in the downstream literature.

Listing 1: wang2022-self-consistency. Inputs: problem x, model p_θ, sample count N, temperature $T > 0$. Output: aggregated answer $\hat{y}$.]Self-consistency (Wang et al., 2022, §3). Inputs: problem x, model p_θ, sample count N, temperature $T > 0$. Output: aggregated answer $\hat{y}$.

```
function self_consistency(x, p_θ, N, T):
    𝒴 ← ∅                              # collected answers
    for n = 1 to N:
        z^(n) ∼ p_θ(· | x) at temperature T # sample trace
        y^(n) ← extract_answer(z^(n))    # regex / parser
        𝒴 ← 𝒴 ∪ {y^(n)}
    return arg max_y Σ_{n=1}^{N} 𝟙[y^(n) = y] # majority vote
```

The compute cost is $C \propto N$ and the N samples are embarrassingly parallel, sharing only the prompt-prefix KV cache (Snell et al., 2024, §2.1). Tensor shapes: each sample is a sequence of shape $\mathsf{S}_n \le L_{\max}$ tokens; the answer space the model votes over is finite and (for the targeted benchmarks)

[17] KB excerpt: `kb/excerpts/wang2022-self-consistency.md#abstract`.
[18] KB excerpt: `kb/excerpts/wang2022-self-consistency.md#sec-3`.

low-cardinality — integer answers on GSM8K and SVAMP, multiple-choice letters on AQuA (Wang et al., 2022, §4).

Wang et al. (2022, §3) present the same mechanism in a Bayesian framing: $\hat{y}$ is the mode of the marginal $p_\theta(y \mid x) = \sum_z p_\theta(y \mid x, z)\, p_\theta(z \mid x)$ estimated by Monte Carlo over the N traces. Marginalising out z is the conceptual contribution; the unweighted count is the simplest estimator of the marginal mode. Later work adds importance weights from verifiers (section 5); the weight $w_n = 1$ here recovers Wang et al.

3.2 Why it works

The empirical fact that majority-over-traces beats any single trace holds whenever two conditions are met. First, the model places higher probability mass on *correct* reasoning paths than on any single *incorrect* one — even if no individual sample is more likely than 0.5, the correct answer can be the modal answer across samples when the model spreads error mass across many distinct wrong trajectories. Second, the answer space is small enough that the correct-and-correlated samples accumulate on a single bin while errors diffuse (Wang et al., 2022, §3). Both conditions are empirically satisfied on math word problems and constrained-choice benchmarks; both fail on open-ended generation (section 3.5).

The connection to ensembling theory is direct. Treat each draw as a noisy classifier $\hat{y}^{(n)}$ with $\Pr[\hat{y}^{(n)} = y^*] = p$ for some $p > 1/|\mathcal{Y}|$; the N draws are exchangeable and weakly dependent through the shared prompt. The probability that the plurality of N samples equals y^* approaches 1 as $N \to \infty$ by the law of large numbers, with rate determined by the gap $p - \max_{y \neq y^*} \Pr[\hat{y}^{(n)} = y]$. The rate is *multiplicative* in this gap, which is why a small per-sample edge produces a large vote-margin advantage at moderate N. This is the standard variance-reduction argument used to justify ensemble voting in classical ML, transposed to the language-model decoder (Wang et al., 2022, §3).

Why $T > 0$ is required: greedy decoding ($T = 0$) returns the same trace every time, so $N > 1$ samples carry no additional information. Wang et al. (2022, §4.3) ablate temperature and find gains stable across $T \in [0.4, 0.7]$, with degradation at very high T (the per-sample correctness probability p drops below the threshold where majority recovers it) and at $T \to 0$ (samples collapse to the greedy trajectory).

Intuition

Self-consistency is "ask the model the same question N times and take the answer it gives most often." The folk gloss is exactly right; the only subtlety is that the question is asked at $T > 0$ so the N answers are not literally identical. Returning to math: $\hat{y}$ in eq. (2) is the mode of the empirical distribution $\hat{p}_N(y \mid x) = \frac{1}{N} \sum_n \mathbb{1}[y^{(n)} = y]$, which is a Monte Carlo estimator of the answer marginal $p_\theta(y \mid x) = \mathbb{E}_{z \sim p_\theta(\cdot \mid x)}\big[p_\theta(y \mid x, z)\big]$ filtered through the deterministic extraction map $z \mapsto y$. The estimator is unbiased and consistent in N; the rate of convergence is governed by the variance of the indicator, which shrinks fastest when the answer marginal is peaked on the correct y^*.

3.3 Headline results

Wang et al. (2022) report gains on five benchmarks at the PaLM-540B / GPT-3-175B scale where greedy CoT is itself non-trivial (Wang et al., 2022, abstract):

[19] KB excerpt: `kb/excerpts/wang2022-self-consistency.md#headline`.

Benchmark	Self-consistency gain over greedy CoT
GSM8K (arithmetic word problems)	+17.9%
SVAMP (arithmetic word problems)	+11.0%
AQuA (algebra, multiple choice)	+12.2%
StrategyQA (commonsense)	+6.4%
ARC-challenge (science MC)	+3.9%

Table 1: Self-consistency over greedy CoT, headline gains (Wang et al., 2022, abstract).[19] The arithmetic benchmarks (GSM8K, SVAMP, AQuA) show the largest gains; the commonsense and science MC benchmarks where the answer space is constrained but the model's per-sample edge is smaller show correspondingly smaller — but consistently positive — improvements.

The gains are produced by a single change to the decoder: from greedy to N-sample-plus-majority-vote at $T = 0.7$. No new model is trained, no exemplars are added to the CoT prompt, no answer verifier is introduced. Wang et al. (2022, §4) additionally report a saturation curve in N: gains are stable across $N \in [10, 40]$ with diminishing returns past $N \approx 40$.[20] The practical sweet spot in the downstream literature is $N = 20$, which captures most of the gain at half the compute of $N = 40$.

The result has not aged out. DeepSeek-AI (2025a, §2.3) report that DeepSeek-R1-Zero — an RL-trained reasoning model that already lifts AIME 2024 pass1 from 15.6% to 77.9% — gains a further 8.8% to 86.7% when the same self-consistency rule is applied at inference time over 64 samples.[21] The mechanism that worked on PaLM-540B in 2022 still adds nearly nine points of pass1 to the strongest open-weight reasoning model in 2025.

3.4 The baseline interpretation

Self-consistency is the floor of the test-time compute axis (section 4). Every other strategy in this paper adds either a verifier signal that re-weights the vote, a search structure that reuses partial-prefix compute, or a training-time intervention that changes the per-sample distribution itself:

- **Best-of-N with outcome verifier** (ORM) (Lightman et al., 2023, §2): replaces the unweighted majority of eq. (2) with $\hat{y} = y^{(n^*)}$ where $n^* = \arg\max_n R_\phi(x, z^{(n)}, y^{(n)})$. Adds a learned signal at training cost C_ϕ and runs only at fixed prompt overhead per sample; the vote becomes weighted by verifier confidence.
- **Best-of-N with process reward model** (PRM, section 5): scores each *step* of the trace and aggregates per-trace by min, product, or last-step. Lightman et al. (2023, §4) report PRM beats ORM at matched N on the MATH benchmark; the gap is largest at small N, where a single bad-step penalty is fatal under min-aggregation.
- **Tree search with PRM** (section 6): replaces flat sampling with MCTS or step-level beam search guided by a step verifier (et al. , Microsoft, §3). Reuses partial-prefix KV-cache across siblings; cost scales as $B \times L$ rather than $N \times L$.

[20] KB excerpt: `kb/excerpts/wang2022-self-consistency.md#sec-4`.
[21] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime`.

- **RL on verifiable rewards** (sections 7 and 8): trains the model so the per-sample distribution itself concentrates on correct answers. Self-consistency over an RLVR-trained model is then the same algorithm operating on a sharper underlying distribution (DeepSeek-AI, 2025a, §2.3).

The pattern is consistent: self-consistency is what every test-time-compute paper compares against because it is what users get "for free" from any decoder once the model emits CoT traces. Outside this paper's reasoning context, the same role of "cheapest baseline" makes self-consistency the standard calibration point for verifier- and search-based methods at section 5 and section 6, and an inference-time control even on top of the strongest RLVR-trained models in section 8.

3.5 When self-consistency fails

Three failure modes are catalogued in the literature; each stresses a different precondition of eq. (2).

Brittle answer extraction. The vote in eq. (2) is over the *extracted* answer $y^{(n)}$, not the full trace $z^{(n)}$. If the regex fails — the model omits the "The answer is X" template, or formats numbers inconsistently (*"42"* vs. *"forty-two"* vs. *"42.00"*) — different traces arriving at the same underlying answer can land in different vote bins, splitting the correct cluster and causing a wrong answer to win by plurality. This is the dominant production failure mode of CoT pipelines (Wang et al., 2022, §2.4); mitigations are robust parsers, structured-output decoders (Paper 1's inference-adjuncts section on structured-output / FSM-constrained decoding), and trained answer-format consistency.

Temperature too high or too low. Wang et al. (2022, §4.3) ablate T and find an inverted-U shape: at $T \to 0$ all N samples collapse to the greedy trace and the vote is degenerate; at large T (e.g., $T \geq 1.2$) the per-sample correctness rate drops below the threshold where majority recovers it. The gain band is roughly $T \in [0.4, 0.9]$ on the reported benchmarks.

Many degenerate answer paths. The mechanism assumes the correct answer is the *modal* extracted answer. This fails when many traces converge on the same plausible-but-wrong answer — for instance, on a word problem with a tempting incorrect arithmetic shortcut, the model may produce that shortcut consistently across samples and majority-vote into a wrong answer with high confidence. Wang et al. (2022, §5) acknowledge that self-consistency cannot recover from systematic error. Open-ended generation (creative writing, summarisation, code with non-canonical correct outputs) breaks the precondition that the answer space has finite support; the analogue there is best-of-N under a learned verifier (section 5), not majority vote.

3.6 Forward link

Self-consistency moves the inference decoder from a single greedy draw to N independent samples plus a deterministic vote. The N axis is the simplest form of inference-time compute; the next section (section 4) generalises to the full taxonomy of parallel-sampling, tree-search, and sequential-thinking compute budgets, formalises the compute-optimal selection problem of Snell et al. (2024), and presents the FLOPs-matched headline that on medium-difficulty problems a 14× smaller model with test-time-compute beats the larger model at matched inference FLOPs. Self-consistency is then re-derivable as the unweighted-majority special case of that taxonomy at fixed sample count and verifier weight $w_n = 1$.

4 Inference-time compute as a scaling axis

Sections 2 and 3 treat chain-of-thought and self-consistency as decoder-side tactics: the model is fixed, the prompt is fixed, and a knob (sample T, sample count N) is turned. They are correctly described as "prompting tricks" in the 2022 literature. Snell et al. (2024) reframe the entire family. Test-time compute, in their formulation, is not a bag of tricks but a *scaling axis* — a quantitative dimension along which loss, accuracy, and FLOP cost trade off in measurable, allocateable ways, structurally analogous to the training-side compute axis of Kaplan and et al. (OpenAI) and Hoffmann and et al. (DeepMind). The headline empirical claim — a smaller base model plus compute-optimal test-time spend matches a 14× larger base model at FLOPs-matched cost on easy and intermediate MATH problems (Snell et al., 2024, §1, §6)[22] — promotes inference-time compute from a deployment optimisation to a research-grade scaling-law variable.

This section transcribes Snell's compute-optimal definition (the only load-bearing equation), walks the proposer/verifier decomposition that unifies every other section of this paper, formalises the question-difficulty operationalisation that makes the prompt-conditional argmax tractable, and presents the FLOPs-matched substitution result that makes the structural Chinchilla parallel (Paper 2 §scaling-laws) explicit. It closes on the open functional form $L(\theta, N)$ — power law, log, or something else — which remains unsettled as of 2026.

4.1 The compute-optimal test-time strategy

Fix a base LLM and a prompt q. Let $y^*(q)$ denote the ground-truth correct response to q, and let $\text{Target}(\theta, N, q)$ denote the distribution over output tokens induced by running the LLM on q under test-time hyperparameters θ with a compute budget of N inference FLOPs (or, equivalently, generated tokens). Snell et al. (2024, §3.1, Eq. 1) define the **test-time compute-optimal scaling strategy**[23] as

$$\theta^*_{q,\,y^*(q)}(N) \;=\; \arg\max_{\theta}\; \mathbb{E}_{y\sim\text{Target}(\theta,\,N,\,q)}\Big[\mathbb{1}_{y\,=\,y^*(q)}\Big]\,. \tag{3}$$

The argmax is over a strategy space θ that ranges over every test-time hyperparameter: sampling temperature, sample count N, beam width M, MCTS rollout count N_{rollout}, revision-chain depth, verifier weighting scheme. The objective is the expected indicator of correctness under that strategy at the fixed compute budget N.

Symbol glossary. For eq. (3):

Symbol	Meaning
q	the prompt / question
$y^*(q)$	ground-truth correct response
N	test-time compute budget (FLOPs or token count)
θ	test-time hyperparameters of a chosen strategy
$\text{Target}(\theta, N, q)$	output distribution induced by spending N FLOPs with strategy θ
$\mathbb{1}_{y=y^*(q)}$	indicator: 1 if y matches ground-truth, else 0

[22] KB excerpt: `kb/excerpts/snell2024.md#sec-1-flops-matched`.

[23] KB excerpt: `kb/excerpts/snell2024.md#sec-3-1`. Notation note: Snell et al. (2024) write the subscript with mixed labels a^* in the index and y^* in the indicator; we unify to $y^*(q)$ throughout for readability without altering the equation's content.

What the equation commits to. Three commitments are non-trivial. First, θ^* is indexed by the prompt q, not by the model. Snell et al. (2024, §3.2) are explicit: the optimal test-time strategy is *prompt-conditional*, and treating it as a fixed model-level hyperparameter — "always use best-of-32" or "always use a beam of width 4" — leaves a measurable amount of performance on the table. Second, the budget N is a hard constraint: the argmax in eq. (3) is implicitly subject to the FLOP cost of θ at budget N matching the budget exactly, which couples the strategy choice to the cost model of the underlying operations (a wider beam costs more per step but explores more; a deeper revision chain costs proportionally to depth; a larger MCTS budget grows polynomially in branch-times-depth). Third, the ground-truth $y^*(q)$ appears explicitly in the objective. This is the move that distinguishes a scaling *law* from a scaling *heuristic*: the optimum is defined against verifiable correctness, not against a proxy score. Operationalising the argmax without access to $y^*(q)$ at deployment is the central practical move of section 4.3.

Intuition

The folk gloss of eq. (3) is "given a hard problem and a fixed compute budget, what's the smartest way to spend it inside the model's inference loop?" The folk gloss is exactly right; the only subtlety is that the answer depends on *which* problem. Returning to math: the indicator under the expectation in eq. (3) is the per-prompt success probability of strategy θ, and the argmax is the strategy that maximises that probability at compute N. The dependence on q enters because the inner expectation is over $\text{Target}(\theta, N, q)$, which is itself a function of the prompt — different prompts induce different output distributions, so different strategies bend those distributions toward $y^*(q)$ most efficiently.

4.2 The proposer/verifier decomposition

Across the otherwise-disparate test-time-compute literature — chain-of-thought (section 2), self-consistency (section 3), best-of-N with verifier (section 5), beam search and MCTS (section 6), revision-finetuned models, RLVR-trained long-CoT — Snell et al. (2024, §2)[24] identify two and only two structural levers for shaping $\text{Target}(\theta, N, q)$:

1. **Modify the proposal distribution.** Augment the prompt with additional tokens the LLM conditions on to obtain a different output distribution. Concretely: prepend a chain-of-thought instruction (et al. , Google Brain; Kojima et al., 2022), finetune the model to iteratively revise its own answer (Snell et al., 2024, §2), or RL-train the model so its base distribution itself emits long thinking traces (sections 7 and 8). Each of these reshapes $\text{Target}(\theta, N, q)$ from the inside.

2. **Apply a verifier on outputs.** Sample K candidates from the (unchanged) base distribution and select among them with a learned scorer. Concretely: best-of-N with an outcome reward model, best-of-N with a process reward model (section 5), beam search or MCTS guided by a PRM (Snell et al., 2024, §5.2).[25] This reshapes $\text{Target}(\theta, N, q)$ from the outside, by post-hoc reweighting.

The two levers compose. Self-consistency is the proposer-only special case at unit verifier weight (section 3); best-of-N with ORM is the verifier-only special case on a fixed base distribution; RLVR-trained long-CoT plus PRM-weighted self-consistency (sections 5.2 and 8) deploys both levers

[24] KB excerpt: `kb/excerpts/snell2024.md#sec-2-proposer-verifier`.
[25] KB excerpt: `kb/excerpts/snell2024.md#sec-5-2-search`.

simultaneously. Snell et al. (2024, §2) argue that every published test-time-compute method to date is one of {proposer-only, verifier-only, both}, and the taxonomic claim has not been counter-exampled in the 2024-2026 literature.

Analogy

The proposer/verifier decomposition is structurally Markov-chain Monte Carlo. In an MCMC sampler targeting a hard distribution π, the two design choices are the *proposal kernel* (which candidate next state to sample) and the *accept/reject rule* (which candidate moves to keep). Modifying the proposal kernel and tuning the accept/reject rule are independent levers that compose multiplicatively: a better kernel reduces the rejection rate; a sharper accept rule filters more aggressively at fixed kernel quality. Snell et al. (2024, §2) make this connection explicit. Returning to math: eq. (3) maximises $\mathbb{E}_{y\sim\text{Target}(\theta,N,q)}\left[\mathbb{1}_{y=y^*(q)}\right]$, and the proposer/verifier decomposition factors θ into a proposal-shaping component (the kernel-analog — prompt augmentation, revision, RL-training of the base distribution) and a verifier-applying component (the accept-rule-analog — ORM, PRM, search-against-PRM). Both components shape the same $\text{Target}(\theta, N, q)$; optimising over both jointly is the content of eq. (3).

4.3 Question difficulty as a sufficient statistic

The argmax in eq. (3) is unevaluable as written: it requires $y^*(q)$ at deployment. Snell et al. (2024, §3.2)[26] introduce a tractable approximation by replacing the per-prompt argmax with a **difficulty-conditional** argmax over a discrete bucket of prompts.

The operationalisation: take the base LLM, sample 2048 completions on each test-set prompt q, and compute the empirical pass@1

$$\widehat{\text{pass@1}}(q) \;=\; \frac{1}{2048}\sum_{n=1}^{2048}\mathbb{1}\left[y^{(n)} = y^*(q)\right], \qquad y^{(n)} \sim p_\theta(\cdot \mid q). \tag{4}$$

Bin the $\widehat{\text{pass@1}}(q)$ values into five quantiles (easy → very hard) over the test set, yielding five *difficulty bins*. The compute-optimal strategy for an unseen prompt q is then approximated by the strategy that maximises validation accuracy on the difficulty bin q falls into:

$$\theta^*(q, N) \;\approx\; \arg\max_\theta \frac{1}{|\mathcal{D}_{\text{bin}(q)}|}\sum_{q'\in\mathcal{D}_{\text{bin}(q)}}\mathbb{E}_{y\sim\text{Target}(\theta,N,q')}\left[\mathbb{1}_{y=y^*(q')}\right], \tag{5}$$

where $\mathcal{D}_{\text{bin}(q)}$ is the validation subset that shares q's difficulty bin. Snell et al. (2024, §3.2) define this as the **oracle difficulty** setting, since $\mathbb{1}_{y=y^*}$ on the validation fold requires ground-truth.

The deployable variant replaces the indicator with a verifier score: sample 2048 completions, score each with a learned outcome verifier, average the score, and bin on the averaged score instead. This is Snell et al. (2024, §3.2)'s **model-predicted difficulty**, and it costs the same compute as one strategy run while delivering most of the oracle difficulty's gain.

Empirical content of the binning. The five difficulty bins turn out to be more than a discretisation convenience. Snell et al. (2024, §3.2) report that the strategy that maximises accuracy is *qualitatively different* across bins:

[26] KB excerpt: `kb/excerpts/snell2024.md#sec-3-2-difficulty`.

- **Easy prompts** benefit most from *revision* (modify the proposal). The model already nearly solves the problem; the right move is to refine the existing draft.
- **Intermediate prompts** benefit most from *search* (best-of-N-weighted, beam-search, or MCTS against a PRM (Snell et al., 2024, §5.2)). The model occasionally solves the problem; the right move is to sample many and let the verifier select.
- **The hardest prompts** fail to benefit from current test-time-compute strategies and require *more pretraining* (section 4.5).

Intuition

Different problems want different test-time strategies because they sit at different points on the model's competence curve. A problem the model already nearly solves wants *refinement* — iterate on the existing draft. A problem the model occasionally solves wants *sampling* — draw many independent attempts and let a verifier pick. A problem the model essentially never solves wants *a better model* — no amount of inference compute will rescue it. Returning to math: the per-bin argmax in eq. (5) selects the strategy that maximises the bin-averaged success indicator, and the empirical content of Snell et al. (2024, §3.2, Fig. 9) is that the argmax shifts from revision-heavy to search-heavy to "no inference strategy works" as the bin moves from easy to hard.

4.4 The 4× compute-optimal vs. best-of-N gain

Snell et al. (2024, abstract)[27] report that picking the right strategy per difficulty bin — the operationalised eq. (5) — delivers $\sim 4\times$ **efficiency** over a fixed best-of-N baseline at matched accuracy. Equivalently, holding inference FLOPs constant, the bin-conditional strategy reaches accuracy levels that fixed best-of-N needs four times the budget to match. The single largest source of the 4× is the easy-prompt-uses-revision / hard-prompt-uses-search split: applying revision uniformly across the test set leaves search-favoured bins underserved, and applying search uniformly leaves revision-favoured bins overspending on parallel sampling when an iterative refine would suffice.

Forum signal

The 4× figure is sometimes informally cited as "Snell's 4×." The number is specific to the MATH benchmark, the PaLM 2-S base model, and the strategy menu of best-of-N-weighted, beam search, and revision under the test-time-compute setup of Snell et al. (2024, §5–6). It is not a universal exchange rate. Cited or repeated outside that benchmark and model class, the 4× figure should be treated as a discovery signal that compute-optimal allocation matters, not as a numerical constant transferable to other domains. Snell et al. (2024, §6) themselves note the result depends on "the specific conditions on the pretraining and inference workload."

4.5 The FLOPs-matched comparison: 14× pretraining substitution

The structural Chinchilla parallel becomes explicit in Snell et al. (2024, §6)'s FLOPs-matched experiment, where total end-to-end FLOPs (training plus inference) are held fixed and the allocation

[27] KB excerpt: `kb/excerpts/snell2024.md#abstract`.

between training and inference is varied. The two compared configurations:

$$C^{\text{train}}_{\text{small}} + C^{\text{test}} \;\approx\; C^{\text{train}}_{\text{large}} + C^{\text{1-pass}}_{\text{large}}. \tag{6}$$

The left side is a smaller base model plus compute-optimal test-time spend (the operationalised eq. (5)); the right side is a 14× larger base model running a single-pass inference at matched total FLOPs. Snell et al. (2024, §1)[28] report:

> On easy and intermediate questions, and even hard questions (depending on the specific conditions on the pretraining and inference workload), additional test-time compute is often preferable to scaling pretraining. … *[I]n some settings it is be more effective to pretrain smaller models with less compute, and then apply test-time compute to improve model outputs.* That said, with the most challenging questions, we observe very little benefits from scaling up test-time compute. Instead, we find that on these questions, it is more effective to make progress by applying additional pretraining compute, demonstrating that current approaches to scaling test-time compute may not be 1-to-1 exchangeable with scaling pretraining.

The 14× substitution ratio is therefore *conditional on the difficulty regime*. For easy and intermediate prompts (which are most prompts in deployment), test-time compute substitutes one-for-one with *at least* 14× more pretraining FLOPs at matched accuracy. For the hardest tail (the very hard bin, the bin where the base model's $\widehat{\text{pass@1}}(q)$ is near zero), the substitution breaks down and pretraining FLOPs cannot be replaced by inference FLOPs.

The structural Chinchilla parallel. Paper 2's scaling-laws section transcribes Hoffmann and et al. (DeepMind, §3, Eq. 2)'s parametric loss $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ and derives the compute-optimal allocation between parameters N and tokens D at fixed training compute $C_{\text{train}} = 6ND$. Snell et al. (2024)'s Eq. 1 (transcribed as eq. (3) above) is the inference-side analog: an argmax over inference-strategy hyperparameters θ at fixed inference compute $C_{\text{test}} = N$. The two argmaxes target different objects — training loss vs. per-prompt success indicator — and live on different sides of the training/inference boundary, but they share the optimisation form: a constrained argmax over an allocation parameter at a fixed compute budget. The FLOPs-matched experiment of eq. (6) closes the loop by relating the two budgets: at fixed total FLOPs $C_{\text{train}} + C_{\text{test}}$, how much of each should be spent? The empirical answer of Snell et al. (2024, §6) is that the two budgets substitute on easy and intermediate prompts and decouple on the hardest tail. A *joint* closed-form $L(N, D, \theta, C_{\text{test}})$ scaling law that subsumes Chinchilla and Snell remains an open question (section 4.7, section 10).

4.6 Tensor-shape and operational notes

The strategy parameter θ in eq. (3) is not a tensor in the usual sense — it is a vector of inference-time hyperparameters whose shape and content depend on the strategy class. For reference:

Strategy class	Components of θ	Cost in N
Best-of-N with ORM	(K, T)	$K \cdot L_{\text{ans}}$
Best-of-N-weighted with PRM	(K, T, agg)	$K \cdot L_{\text{ans}} \cdot (1 + c_{\text{verify}})$
Beam search against PRM	(K, M, T)	$K \cdot D \cdot c_{\text{step}}$
MCTS against PRM	$(N_{\text{rollout}}, c_{\text{puct}}, D)$	$N_{\text{rollout}} \cdot D \cdot L_{\text{step}}$
Revision chain	$(K_{\text{revisions}}, T)$	$K_{\text{revisions}} \cdot L_{\text{ans}}$
Self-consistency	(N, T)	$N \cdot L_{\text{ans}}$

[28] KB excerpt: `kb/excerpts/snell2024.md#sec-1-flops-matched`.

where L_{ans} is mean answer-trace length, L_{step} is mean per-step token count, D is search depth, K is sample count, M is beam width, T is sampling temperature, c_{verify} is per-verifier-call cost in equivalent generation tokens, and c_{puct} is the MCTS exploration constant. The budget N in eq. (3) is the sum of the right-hand cost column. The strategy menu Snell et al. (2024, §5–6) explore is the {best-of-N-weighted, beam search, revision} subset; later sections of this paper (sections 6 and 7) extend the menu to MCTS and to RL-trained long-CoT proposers.

The pass@1 difficulty estimator of eq. (4) is, in operational terms, a |test set| × 2048 shape of indicators, summed along the second axis to yield a scalar per prompt. The 2048-sample budget is itself non-trivial inference compute — a preprocessing pass that Snell et al. (2024, §3.2) amortise across the entire downstream strategy-selection pipeline.

4.7 The functional form of $L(\theta, N)$ is open

Snell et al. (2024) produce empirical curves — accuracy as a function of test-time FLOP budget, broken down by strategy and difficulty bin (Snell et al., 2024, §5–6) — but they do not commit to a closed-form $L(\theta, N)$ in the spirit of Chinchilla's $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ (Hoffmann and et al. , DeepMind, §3, Eq. 2). The shape of the empirical curves is qualitatively concave in $\log N$ on easy and intermediate prompts and roughly flat on the hardest bin, but no parametric fit with the structural status of Chinchilla's law has been attested for inference-time compute. The 2025 budget-forcing experiment of et al. (Stanford+) measures accuracy as a function of explicit thinking-token budget on a small reasoning-trained model and reports *logarithmic* returns to thinking-token count on its evaluation suite (s1's Fig. 1, qualitatively monotone in $\log L$ where L is the thinking-budget length). This is suggestive but does not establish a functional form: it is a single-paper, single-model report on a single benchmark family.

Contradiction

The functional form $L(\theta, N)$ is unsettled. Snell et al. (2024) report empirical accuracy-vs-FLOPs curves but stop short of fitting a parametric scaling law. et al. (Stanford+) report roughly logarithmic returns to thinking-token budget on a small reasoning-trained model. DeepSeek-AI (2025a) report monotone increase of AIME 2024 pass@1 with response-length ramp from ~ 200 to $\sim 10\,000$ tokens during GRPO training (section 8), but do not parameterise the inference-time accuracy-vs-budget curve at fixed training. The candidate functional forms are: power law in N (Kaplan-style), logarithmic in N (s1-suggestive), bounded sigmoid saturating at the base-model competence ceiling, or a difficulty-bin mixture of all three. As of 2026-Q2 no closed-form $L(\theta, N)$ analogous to Chinchilla's $L(N, D)$ is attested in the literature, and the joint $(C_{\text{train}}, C_{\text{test}})$ frontier — the natural generalisation of eq. (6) into a parametric form — is an open research direction (sections 10 and 14).

A second open frontier: Snell et al. (2024, §4–6) run their entire analysis on the MATH benchmark using PaLM 2-S (Codey) as the base model. The 14× substitution ratio, the 4× compute-optimal-vs-fixed-best-of-N gain, and the difficulty-bin strategy split are all measured on this single benchmark/model combination. The 2025-2026 reasoning-trained-model literature (sections 8 and 9) extends the empirical support for Snell's framing to AIME and competition-grade math under DeepSeek-R1 and s1, but a Snell-style FLOPs-matched comparison has not been redone on a frontier-2026 reasoning-trained base model. The substitution-ratio number is therefore best read as a discovery-grade result — it establishes that the substitution exists and is large — rather than as a transferable constant.

4.8 Cross-paper and forward links

The Snell framing is the load-bearing scaffolding for the rest of this paper:

- Section 5 deepens the verifier leg of section 4.2: Lightman et al. (2023)'s PRM loss, the three aggregation rules, and the post-2023 PRM lineage. The verifier-only special case of eq. (3) is what section 5 formalises.
- Section 6 deepens the search side: beam search, BFS, and MCTS are the three strategies that operationalise Snell et al. (2024, §5.2)'s "search against a PRM" under the $\theta = (K, M, c_{\text{puct}}, D)$ rows of section 4.6.
- Sections 7 and 8 treat the proposer leg's training-time form: RL-trained long-CoT models (DeepSeek-R1, o1, the distilled-Qwen variants) ship a base distribution that is itself shaped to emit long thinking traces, so $\text{Target}(\theta, N, q)$ becomes a different object before any test-time strategy is applied.
- Section 10 returns to the FLOPs-matched experiment of eq. (6) and asks the deeper question: at fixed total compute, when do the marginal FLOPs go to inference search, RLVR training, or pretraining? Snell's curves give one slice (inference vs. pretraining); the RLVR literature gives partial answers for inference vs. RL.

The structural picture: Snell's Eq. 1 (transcribed as eq. (3)) is the inference-time analog of the Chinchilla compute-optimal allocation in Paper 2 §scaling-laws. Both are constrained-optimisation problems with empirically-fitted exchange rates between dimensions; both produce headline substitution-ratio numbers (Chinchilla: parameters-vs-tokens at fixed C_{train}; Snell: strategy-choice at fixed C_{test}, and 14× inference-vs-pretraining at fixed $C_{\text{train}} + C_{\text{test}}$); and both leave open how to compose the two argmaxes into a single joint frontier. The verifier leg picks up in section 5; the third leg of the triangle — RL on verifiable rewards — picks up in section 7.

5 Process reward models: the verifier leg

The third corner of the reasoning triangle is the *verifier*: a learned signal that scores a candidate solution at granularity finer than the final answer, and uses that score to re-rank, re-weight, or guide further computation. Its canonical artefact is the **process reward model (PRM)** — a function that scores each step of a chain-of-thought trace (Lightman et al., 2023, §2.2).[29] PRMs were introduced by Lightman et al. (2023) as the contrast to outcome reward models (ORMs), which score only the final answer; the headline finding of that paper — process supervision substantially outperforms outcome supervision on the MATH benchmark — supplies the empirical motivation for the entire 2024–2026 PRM lineage. This section formalises the per-step reward, walks the four canonical aggregation rules, transcribes the PRM800K experiment, traces the lineage through Math-Shepherd's automated labelling and ThinkPRM's generative-verifier shift, and closes on the open question of whether external PRMs still add value once the policy itself has been RL-trained for verifiable reasoning.

5.1 What a process reward model commits to

Let x denote a problem and $z = (s_1, s_2, \ldots, s_T)$ a reasoning trace decomposed into T steps, with each s_t a contiguous span of trace tokens (a sentence, an equation line, a paragraph). A **process**

[29] KB excerpt: `kb/excerpts/lightman2023-prm800k.md#sec-2-2`.

reward model is a function[30]

$$R_\phi^P : (x,\ s_1, \ldots, s_t) \longmapsto r_t \in [0,1], \tag{7}$$

trained to predict the per-step correctness label $c_t \in \{0,1\}$. The training objective is binary cross-entropy on the prefix (Lightman et al., 2023, §2.2):

$$\mathcal{L}_{\text{PRM}} = -\,\mathbb{E}_{(x,\, s_{1:t},\, c_t)\sim\mathcal{D}}\Big[\, c_t \log R_\phi^P(x, s_{1:t}) + (1-c_t)\log\!\big(1 - R_\phi^P(x, s_{1:t})\big)\Big]. \tag{8}$$

The conditioning is the only structural change from the ORM loss (Lightman et al., 2023, §2.1):

$$\mathcal{L}_{\text{ORM}} = -\,\mathbb{E}_{(x,\, z,\, y,\, c)\sim\mathcal{D}}\Big[\, c \log R_\phi^O(x, z, y) + (1-c)\log\!\big(1 - R_\phi^O(x, z, y)\big)\Big], \tag{9}$$

where $c \in \{0,1\}$ is the outcome correctness label on the final answer y. A PRM emits one score per prefix; an ORM emits one score per trace.

To compare a PRM and an ORM at fixed N in best-of-N reranking, the T-vector $(r_1, \ldots, r_T)$ must be aggregated to a scalar. Three rules dominate the literature (Lightman et al., 2023, §3.1):

$$R^P_{\min}(z) = \min_{1\le t\le T} R_\phi^P(x, s_{1:t}), \tag{10}$$

$$R^P_{\prod}(z) = \prod_{t=1}^{T} R_\phi^P(x, s_{1:t}), \tag{11}$$

$$R^P_{\text{last}}(z) = R_\phi^P(x, s_{1:T}). \tag{12}$$

The min reading treats one bad step as fatal; the product reading treats steps as conditionally-independent multiplicands; the last-step reading scores only the final state. Lightman et al. (2023, §4.2) ablate all three on PRM800K-grade verifiers and find **min-aggregation** the strongest;[31] we return to this finding in section 5.2.

Intuition

A PRM is a per-step graded checkmark; an ORM is one final thumbs-up-or-down at the end of the trace. Returning to math: the supervisory signal in eq. (8) is concentrated at T distinct prefixes $s_{1:t}$ for $t = 1, \ldots, T$, while eq. (9) concentrates it at a single (z, y) pair. With dense per-step labels the credit-assignment problem is solved locally — the gradient identifies which step's residual was mis-scored, rather than back-propagating one outcome label across an entire 10^3–10^4-token trace.

5.2 PRM800K: the canonical process-supervision experiment

Lightman et al. (2023) construct **PRM800K**, a dataset of ≈ 800,000 human step-correctness labels on GPT-4-generated solutions to MATH problems (Lightman et al., 2023, §3).[32] The construction protocol: a generator (a GPT-4 variant) emits several candidate solutions per problem; human annotators label each step as positive, negative, or neutral; the dataset is released openly. The resulting verifier is the foundational reference PRM against which all subsequent open-source efforts (Math-Shepherd, ThinkPRM, R-PRM, EDU-PRM) benchmark.

[30] KB note: `kb/notes/reasoning/process-supervision.md#1.2`.
[31] KB excerpt: `kb/excerpts/lightman2023-prm800k.md#sec-4-2`.
[32] KB excerpt: `kb/excerpts/lightman2023-prm800k.md#sec-3`.

The headline experiment fixes N across two reranking strategies and measures best-of-N accuracy on MATH test (Lightman et al., 2023, §4):[33]

$$\hat{y}_{\text{ORM}} = y^{(n^\star)}, \quad n^\star \in \arg\max_n R^O_\phi(x, z^{(n)}, y^{(n)}), \tag{13}$$

$$\hat{y}_{\text{PRM}} = y^{(n^\star)}, \quad n^\star \in \arg\max_n R^P_{\min}(z^{(n)}). \tag{14}$$

Across the full sweep $N \in [1, 1024]$ the PRM dominates the ORM at matched N, with the gap widest at moderate N (Lightman et al., 2023, Fig. 4). Both reranking strategies in turn dominate plain self-consistency (eq. (2)) at the same sample budget. The process-supervised verifier solves **78%** of problems on a representative MATH test subset (Lightman et al., 2023, abstract).[34]

The min-aggregation finding deserves emphasis. Among eqs. (10) to (12), only the min explicitly penalises a single low-scored step regardless of context. The product formulation, despite being the obvious likelihood-style aggregator under a step-independence assumption, allows many high-scored steps to mask one low-scored step in the geometric mean. The last-step rule discards intermediate signal entirely. Empirically (Lightman et al., 2023, §4.2) the min wins by a margin large enough that it is the default aggregator in every downstream PRM paper. Lightman et al. (2023, §4.3) additionally show that PRM-weighted self-consistency — majority vote with each ballot weighted by the trace's min-PRM score — improves over plain majority vote, recovering some of the PRM gain at lower verifier-cost overhead.

Intuition

Why min wins: if a single algebraic slip turns "$3x = 12$" into "$x = 5$", every subsequent step inherits the error. The trace's truth-value is bounded above by its weakest step. Mean and product let many strong steps mask one critical error; the min surfaces it. The empirical advantage of eq. (10) is consistent with this picture, and the structural form of the gain is a weakest-link-of-the-chain inequality: $R^P_{\min}(z) \le R^P_\phi(x, s_{1:t})$ for every t, with equality at the worst step.

5.3 Math-Shepherd: removing the human-annotation bottleneck

The PRM800K protocol is human-expensive — $\approx$ 800K step-labels at expert-level math annotation cost. Wang et al. (2024) remove the bottleneck by generating per-step labels automatically from *Monte Carlo rollout completion rate*.

deepen-cite: wang2024-mathshepherd §3 (excerpt skeleton-only as of 2026-05; PDF acquisition pending)

The automatic-labelling rule, drawn from the process-supervision KB note's lineage table[35] and the surveyed framing in et al. (2025d), is: for each prefix $s_{1:t}$ of a candidate trace, sample K continuation rollouts $z^{(k)} \sim p_\theta(\cdot \mid x, s_{1:t})$ to completion, extract the answer $y^{(k)}$ from each, and define

$$\tilde{c}_t = \frac{1}{K} \sum_{k=1}^{K} \mathbb{1}[y^{(k)} = y^\star(x)], \tag{15}$$

i.e., the empirical *completion-success rate* from prefix $s_{1:t}$. The label is then thresholded (or used directly as a soft label) and substituted for the human c_t in eq. (8). The rationale is the credit-assignment heuristic: prefixes that lead to correct answers in expectation are good prefixes; prefixes

[33] KB excerpt: `kb/excerpts/lightman2023-prm800k.md#sec-4`.
[34] KB excerpt: `kb/excerpts/lightman2023-prm800k.md#headline`.
[35] KB note: `kb/notes/reasoning/process-supervision.md#3.1`.

that do not are bad. Outcome supervision on the rollouts implicitly produces process supervision on the prefixes.

The implication is structural: any reasoning policy plus a verifiable outcome checker (an arithmetic verifier, a Python interpreter, a regex on the boxed final answer) can manufacture its own PRM training data. The human-annotation cost collapses from $O(\text{PRM800K})$ to the compute cost of $K \times T$ rollout extensions per training example. Math-Shepherd-style automatic labelling is the substrate that subsequent generative-PRM efforts (section 5.4) and reasoning-trained PRMs (section 5.5) build on.

Analogy

Math-Shepherd is the PRM equivalent of Q-learning's bootstrapped value estimate: the value of a state is approximated by the empirical return-from-state under the current policy, rather than by human labelling of the state. Returning to math: eq. (15) is a Monte Carlo estimator of the true success probability $\Pr_{z\sim p_\theta(\cdot|x,s_{1:t})}[y^\star(x)]$, and the PRM trained on eq. (8) with $\tilde{c}_t$ in place of c_t is a learned approximator to that conditional success probability. The bias-variance characteristics are the standard ones of K-sample Monte Carlo.

5.4 ThinkPRM: process reward models that think

The 2025 standout shift: instead of a classifier head that emits one scalar per prefix, the verifier is itself a small reasoning LLM that reads the prefix, emits its own short verification chain-of-thought, and outputs a verdict token (et al., 2025a, §3).[36] Formally, ThinkPRM replaces the discriminative R^P_ϕ of eq. (7) with a generative verifier

$$R^{P,\text{gen}}_\phi(x, s_{1:t}) \;=\; p_\phi\big(\langle\text{verdict}\rangle = \textsf{correct} \mid x,\; s_{1:t},\; v\big), \quad v \sim p_\phi\big(\cdot \mid x, s_{1:t}\big), \tag{16}$$

where v is the verifier's own reasoning trace over the candidate prefix.

The headline result: a long-CoT verifier fine-tuned on **1%** of PRM800K (roughly 8K step-labels rather than 800K) plus self-generated synthetic verification traces beats discriminative verifiers trained on the full PRM800K (et al., 2025a, abstract).[37] On out-of-domain evaluation, ThinkPRM exceeds the full-PRM800K discriminative baseline by **8%** on a GPQA-Diamond subset and **4.5%** on LiveCodeBench, and beats LLM-as-a-Judge by **7.2%** on ProcessBench at matched verification-token budget (et al., 2025a, abstract).[38] The data-efficiency ratio is two orders of magnitude on labels, traded for verification compute.

The cost asymmetry is real: best-of-N with a discriminative PRM costs $O(N)$ classifier passes; with a generative PRM, $O(N \times L_{\text{verify}})$ tokens, where L_{verify} is the verification trace length. The asymmetry is the same one that distinguishes a chain-of-thought policy from a direct-answer policy (section 2), applied now to the verifier role: spend inference compute inside the verifier to extract more signal per labelled example. The mechanism: a verifier's CoT can apply algebraic-verification, type-checking, and dimensional-analysis checks that a feed-forward classifier cannot internalise from labelled examples alone (et al., 2025a, §5).[39]

5.5 R-PRM and EDU-PRM: the 2025 PRM lineage

The 2025 PRM literature splits along two axes orthogonal to the discriminative-vs-generative split:

[36] KB excerpt: `kb/excerpts/thinkprm2025.md#sec-3`.
[37] KB excerpt: `kb/excerpts/thinkprm2025.md#abstract`.
[38] KB excerpt: `kb/excerpts/thinkprm2025.md#headline`.
[39] KB excerpt: `kb/excerpts/thinkprm2025.md#sec-5`.

Reasoning-aware PRMs. She et al. (2025) train a PRM that emits a short reasoning step before scoring; the architecture is discriminative (a verdict head) but the input now includes a brief verifier-reasoning prefix. This generalises better across distribution shifts than vanilla discriminative PRMs.

deepen-cite: she2025-r-prm §3 (excerpt skeleton-only as of 2026-05)

The contrast with eq. (16) is one of degree: ThinkPRM's verifier emits a full long-CoT trace and a verdict token; R-PRM emits a short reasoning preamble and a discriminative score. The two converge as the preamble length grows.

Entropy-driven step boundaries. et al. (Huawei) challenge the common assumption that steps are fixed by the policy's linebreaks or paragraph markers. EDU-PRM defines step boundaries by a local entropy spike in the policy's per-token distribution: a step ends where the model is most uncertain about the next token, on the intuition that high-entropy positions correlate with reasoning branch-points. Trained on **1.5%** of the data of Math-Shepherd, EDU-PRM matches or beats Math-Shepherd on ProcessBench.

deepen-cite: cao2025-edu-prm §3 (excerpt skeleton-only as of 2026-05)

The framing recasts step labelling from a syntactic problem (where do paragraphs end?) to an information-theoretic one (where does the model branch?).

The two strands jointly suggest that the 2023 PRM800K formulation — human-labelled, syntactically-bounded, classifier-head — was a specific, easily-relaxable instantiation of a more general per-step verification programme. Each relaxation (Math-Shepherd, ThinkPRM, R-PRM, EDU-PRM) trades human labels or syntactic priors for some combination of automated rollouts, verifier compute, and information-theoretic boundaries.

5.6 The 2025 PRM survey landscape

et al. (2025d) catalogue the 2023–2025 PRM landscape along the discriminative-vs-generative axis and the test-time-vs-training-time axis.

deepen-cite: zheng2025-prm-survey overview (excerpt skeleton-only as of 2026-05)

The survey's synthesised finding (drawn from the KB note's treatment[40]) runs in two parts. First, generative PRMs win on competition-math benchmarks where each step has a verifiable certificate (algebraic verification, equation manipulation, type-checking) the verifier's CoT can apply; discriminative PRMs remain competitive on broader domains where step-level checks are not formal. Second, and more consequentially, the survey reports that most PRMs improve test-time-compute performance *until the base model becomes a strong reasoning model itself* — at which point the marginal gain from an external PRM at test time decreases. This is the opening of the contradiction discussed in section 5.7.

5.7 PRM utility post-R1: the open contradiction

DeepSeek-AI (2025a) achieved the headline AIME 2024 jump from 15.6% to 77.9% pass1 with *terminal verifier rewards only* — no PRM in the RL loop (DeepSeek-AI, 2025a, §2.2).[41] The R1 reward is rule-based: an answer-match check on math problems and a compiler / unit-test check on code, returning a single binary outcome reward per trajectory. PRMs do not appear in the headline RL loop of the strongest open-weight reasoning model.

[40] KB note: `kb/notes/reasoning/process-supervision.md#3.2`.
[41] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-2-reward`.

The implication, surfaced jointly by et al. (2025a) and et al. (2025d): as the policy itself internalises verification through long-CoT RL, the marginal value of an external PRM at test time shrinks. The 2026 picture is mixed; the contradiction deserves explicit treatment.

Contradiction

PRM utility post-R1 — diminishing or persistent? The PRM800K-era headline "PRM > ORM at matched N" was established on a non-reasoning-trained generator (a GPT-4 variant) (Lightman et al., 2023, §4). As the policy class moves to RLVR-trained reasoning models, the picture shifts. Some 2026 measurements show PRM-best-of-N over R1-distill outputs still delivers gains; others find diminishing returns as the policy's own internal verification approaches the PRM's signal (et al., 2025a, §5). The 2025 PRM survey (et al., 2025d) reports that most PRMs improve test-time-compute performance until the base model becomes a strong reasoning model itself. Whether external PRMs survive the reasoning-trained-model era as a training signal — as opposed to inference-time rerankers — is unresolved as of 2026-Q2; section 12 revisits the question with the post-R1 evidence.

A second, related tension: *process hacking*. PRM-shaped RLVR (Lightman et al., 2023, §2.4)[42] replaces or augments the binary outcome reward of RLVR (section 7) with a per-step PRM score. The middle-ground appeal is theoretical: dense signal beats sparse signal for credit assignment. The empirical risk: a misaligned PRM rewards steps that *look* correct without producing the right answer, the step-level analogue of outcome-hacking under a learned reward. Whether PRM-shaped GRPO actually outperforms verifier-only GRPO at matched compute is benchmark-specific and unsettled in 2026.

5.8 Forward link

The PRM scores per-step value $r_t \in [0, 1]$ on a partial prefix $s_{1:t}$. With this signal in hand, the inference decoder can do more than rerank — it can *steer expansion* during generation. Section 6 introduces the tree-search families (best-first, beam, MCTS) that use a PRM as the value function $V_\phi(s_{1:t}) = R^P_\phi(x, s_{1:t})$ in the canonical AlphaZero-style recurrence, formalises the rStar-Math UCT rule, and presents the cost analysis of $O(N_{\text{rollout}} \cdot D \cdot L_{\text{step}})$ per simulation. The aggregation rules of eqs. (10) to (12) carry over: tree-search nodes are scored by $R^P_{\min}$ over their root-to-node prefix, so the weakest-link inequality of section 5.2 continues to shape which branches survive expansion. The PRM is then re-encountered in section 12 as the centerpiece of the post-R1 survival question — whether external verifiers still earn their training and inference cost once the policy itself reasons.

6 Tree-search families: BFS, DFS, MCTS

Section 4 factored test-time compute into a proposer leg and a verifier leg; section 5 built out the verifier leg as a per-step scoring function R^P_ϕ over partial prefixes $s_{1:t}$. The third decomposition — after sampling (section 3) and sequential revision (section 4.6) — is *tree search*: explicitly construct and explore a tree of partial reasoning prefixes, using the PRM as a value function to direct expansion. The taxonomy of the 2023–2025 literature, surveyed in et al. (2025c),

deepen-cite: wei2025-tree-search-survey §2–3 (taxonomy enumeration; excerpt file pending)

catalogues three families that bracket the design space: beam over reasoning steps, BFS or level-order expansion, and Monte-Carlo Tree Search. This section formalises the search-tree datastructure,

[42] KB note: `kb/notes/reasoning/process-supervision.md#4.2`.

transcribes the canonical reasoning-MCTS recurrence from et al. (Microsoft),[43] walks the four phases of an MCTS iteration, situates ReST-MCTS* (Zhang et al., 2024) as the AlphaZero-shaped reasoning loop, notes MCTS-RAG (Hu et al., 2025) as the retrieval-extended cousin, and closes on the live contradiction between MCTS-headline framings and Snell et al. (2024, §5–6)'s evidence that BFS-with-PRM is competitive at fixed inference budget on math.

6.1 The search tree as a datastructure over partial traces

Fix a problem x and let $z_{1:t} = (s_1, \ldots, s_t)$ denote a partial reasoning trace decomposed into t steps, with each step s_i a contiguous span of trace tokens (a sentence, an equation line, a bullet)[44]. The **search tree** has nodes indexed by partial traces $z_{1:t}$ and edges labelled by the next step s_{t+1}, with the root at $z_{1:0} = \varnothing$. Three signals attach to the tree:

- the *policy prior* $\pi_\theta(s_{t+1} \mid x, z_{1:t})$, which assigns mass to candidate next steps under the underlying LLM;
- the *value function* $V_\phi(x, z_{1:t}) \to \mathbb{R}$, which scores partial states. The canonical instantiation is $V_\phi = R^P_\phi$, the per-step PRM of eq. (7);
- the *terminal verifier* $\mathcal{V}(x, y) \to \{0, 1\}$, which scores complete answers. For math and code this is the same rule-based check that defines the RLVR reward of eq. (20).

The three search families differ entirely in how they combine π_θ and V_ϕ to direct expansion; the datastructure is common.

Symbol glossary. For the rest of this section:

Symbol	Meaning
B	beam width / branching factor at each expansion
D	maximum search depth (max number of steps)
L_{step}	mean per-step token count
L_{ans}	mean full-answer trace length, $L_{\text{ans}} \le D \cdot L_{\text{step}}$
N_{rollout}	number of MCTS simulation iterations
N_s	MCTS visit count at node s
$Q(s)$	MCTS running mean rollout outcome through s
c_{puct}	UCT exploration constant
V_ϕ	learned value function on partial prefixes (typically a PRM)
$\mathcal{V}$	terminal verifier (rule-based for math/code)

6.2 Beam search and BFS-V over reasoning steps

The simplest search family generalises decoder beam search from token-level to step-level granularity. Maintain a frontier of B partial traces; at each depth, expand each by sampling k next-step continuations from π_θ; rescore the resulting $B \cdot k$ partials with V_ϕ; retain the top B by value. Iterating to depth D produces B candidate full traces; the highest-V_ϕ or highest-$\mathcal{V}$ trace is returned.

The cost is

$$C_{\text{beam}} \;=\; O(B \cdot k \cdot D \cdot L_{\text{step}}) \quad \text{tokens.} \tag{17}$$

[43] KB excerpt: `kb/excerpts/rstar-math2025.md#sec-3-mcts`.
[44] KB note: `kb/notes/reasoning/inference-time-search.md#1-formal-definition`.

Snell et al. (2024, §5.2) formalise the variant they call **BFS-V**: a level-order beam with N initial samples and beam width M. The procedure (Snell's quoted text): "sample N initial predictions for the first step in the solution; score the generated steps according to the PRM's predicted step-wise reward-to-go estimate; filter for only the top $\frac{N}{M}$ highest scoring steps; now from each candidate, sample M proposals from the next step, resulting in a total of $N/M \times M$ candidate prefixes again. Then repeat steps 2–4" (Snell et al., 2024, §5.2).[45] BFS-V differs from classical beam only in the per-step scoring object: the PRM's reward-to-go on $s_{1:t}$ replaces the language-model log-likelihood, and the level-order structure means every depth-t node is fully evaluated before any depth-$(t+1)$ node is expanded.

Why beam-class methods are competitive. Two structural properties favour beam-shaped search on reasoning workloads. First, the cost in eq. (17) is linear in D and in $B \cdot k$, both directly tunable knobs that compose with the difficulty-bin allocation of section 4.3: spend wider beams on harder prompts, narrower on easier ones. Second, the per-step PRM signal is *exactly* what beam needs — a scalar score on a partial prefix — and section 5.2's min-aggregation result ($R^P_{\min}(z) = \min_t R^P_\phi(x, s_{1:t})$) carries over node by node: a beam node's score is the worst PRM score along its root-to-node path, and the weakest-link inequality of section 5.2 prunes weak branches early. The DFS and tree-of-thoughts variants

deepen-cite: yao2023-tree-of-thoughts §3 (canonical citation pending)

add backtracking on a per-node "promising / not promising" heuristic, but the underlying datastructure is the same; they trade the level-order regularity of BFS for depth-first traversal under a node-evaluation gate.

6.3 The MCTS recurrence (rStar-Math)

Monte-Carlo Tree Search adds a fourth ingredient that beam lacks: *rollout-derived value estimates* are accumulated at each node and used to bias future expansion via an exploration–exploitation rule. et al. (Microsoft) take MCTS as the central inference-time mechanism — "a math policy SLM performs test-time search guided by an SLM-based process reward model" (et al. , Microsoft, abstract)[46] — and report 7B policies reaching 90.0% on MATH and 53.3% on AIME 2024 after four rounds of self-evolution (et al. , Microsoft, headline numbers).[47]

The PUCT selection rule. At each iteration, descend the tree from the root by repeatedly choosing the child s that maximises[48]

$$U(s) \;=\; Q(s) \;+\; c_{\text{puct}}\, \pi_\theta(s \,|\, \text{parent}(s))\, \frac{\sqrt{N_{\text{parent}}}}{1 + N_s}. \tag{18}$$

The first term $Q(s)$ is the running mean of rollout outcomes through s (the exploitation term: nodes that have produced correct rollouts are preferred). The second term is the PUCT exploration bonus (et al. , Microsoft)

deepen-cite: rstar-math2025 §3 (exact constants pending PDF acquisition)

: the policy prior $\pi_\theta(s \mid \text{parent})$ shapes which children are *a priori* preferred, the $\sqrt{N_{\text{parent}}}/(1+N_s)$ ratio inflates as the parent has been visited many times relative to s, and c_{puct} dials the overall

[45] KB excerpt: `kb/excerpts/snell2024.md#sec-5-2-search`.
[46] KB excerpt: `kb/excerpts/rstar-math2025.md#abstract`.
[47] KB excerpt: `kb/excerpts/rstar-math2025.md#headline`.
[48] KB note: `kb/notes/reasoning/inference-time-search.md#2-2-mcts-over-reasoning-steps-rstar-math`.

exploration aggressiveness. The constant c_{puct} is the only free hyperparameter of the rule itself; rStar-Math sets it empirically.

The four phases of an MCTS iteration.

1. **Selection.** From the root, repeatedly apply eq. (18) to descend until a leaf ℓ is reached. "Leaf" here means a node that has not yet been expanded under MCTS bookkeeping — not necessarily a terminal in the trace sense.

2. **Expansion.** Sample k candidate next-steps from $\pi_\theta(\cdot \mid x, z_\ell)$ and add them as children of ℓ, each with $Q = 0$ and $N = 0$.

3. **Simulation.** From one of the freshly-expanded children c, roll out a complete trace under π_θ to a terminal state; score the terminal answer with $\mathcal{V}$, and optionally weight intermediate states by $V_\phi = R^P_\phi$ to obtain a real-valued return $G_c \in [0, 1]$.

4. **Backup.** Propagate G_c up the path from c to root, incrementing $N_s \mathrel{+}= 1$ and updating $Q(s) \leftarrow Q(s) + (G_c - Q(s))/N_s$ at each visited node.

After N_{rollout} iterations, the answer is selected as either the most-visited path from root, the rollout that achieved highest return, or a PRM-weighted vote across rollouts; rStar-Math uses the last

deepen-cite: rstar-math2025 §3.x (answer-selection rule pending PDF)

.

Cost analysis. Each MCTS iteration costs one rollout (a full trace under π_θ) plus $O(D)$ tree-traversal operations plus k expansion samples. The rollout dominates, so total cost is

$$C_{\text{MCTS}} \;=\; O(N_{\text{rollout}} \cdot D \cdot L_{\text{step}}) \quad \text{tokens}, \tag{19}$$

matching the bound assumed by section 4.6's strategy table. Comparing eq. (17) and eq. (19): at matched depth D and per-step length L_{step}, N_{rollout} plays the role of $B \cdot k$. The two algorithms differ in *how* the budget is spent: beam fans out uniformly at each depth, while MCTS allocates iterations adaptively to nodes whose Q-values suggest the path is promising. On a benign loss landscape (good policy, calibrated PRM), the adaptive allocation buys MCTS a constant-factor edge; on rough landscapes (under-trained PRM, narrow correct path), the same adaptivity can collapse onto a wrong branch.

Intuition

MCTS is a *compute-allocator* that lets early rollout outcomes re-shape where later rollouts go. Beam search, by contrast, allocates the same width to every depth regardless of how the frontier evolves. On problems where one branch has a sharp success advantage that manifests early in rollouts, MCTS amplifies the signal — $Q(s)$ rises, N_s rises, the UCT bonus on alternate children shrinks, and N_{rollout} increasingly concentrates on the winning subtree. On problems where rollout outcomes are noisy or where the success signal is buried deep, MCTS spends iterations re-validating already-committed paths instead of broadening, and beam can match or beat it. Returning to math: eq. (18)'s exploitation term $Q(s)$ is a Monte-Carlo estimate of the expected return from s, and the ratio $\sqrt{N_{\text{parent}}}/(1 + N_s)$ is the standard PUCT exploration bonus from AlphaGo / AlphaZero, here applied in the text-step action space rather than the board-position action space.

6.4 Tensor-shape and operational notes

The MCTS state is a node s in the trace tree, indexed by its root-to-node prefix $z_{1:t}$. The PRM is the value function on the prefix: $V_\phi(s) = R^P_\phi(x, s_{1:t}) \in [0, 1]$, with the aggregation rules of eqs. (10) to (12) carrying over node by node. Per-iteration cost decomposes as

Operation	Cost
Selection (root $\to$ leaf ℓ)	$O(D)$ tree-traversal ops, no LLM calls
Expansion (k candidate children)	$k \cdot L_{\text{step}}$ tokens
Simulation (full rollout)	$D \cdot L_{\text{step}}$ tokens, plus terminal $\mathcal{V}$
Backup (root $\leftarrow$ leaf)	$O(D)$ tree-traversal ops

The *rollout* dominates at typical $k \in [4, 16]$ and $D \in [10, 50]$, which is why eq. (19) drops the expansion and traversal terms. Operationally, the PRM call cost adds a constant per simulation step — c_{verify} in section 4.6's table — but is normally amortised by batched verifier inference across multiple rollouts in flight.

6.5 ReST-MCTS*: AlphaZero-shaped self-training

Zhang et al. (2024) elevate MCTS from an inference-time mechanism to a *training-data generator*.

deepen-cite: zhang2024-rest-mcts §3 (excerpt skeleton-only as of 2026-05; PDF acquisition pending)

The protocol, drawn from the inference-time-search KB note's treatment[49]:

1. Run MCTS with the current policy $\pi_\theta^{(r)}$ and PRM $V_\phi^{(r)}$ to solve a batch of training problems.
2. Collect MCTS-discovered correct trajectories (those whose terminal $\mathcal{V}(x, y) = 1$).
3. SFT the policy on these harvested trajectories: $\pi_\theta^{(r+1)} \leftarrow \text{SFT}\big(\pi_\theta^{(r)}; \{(x, z, y) : \mathcal{V}(x, y) = 1\}\big)$.
4. Update the PRM $V_\phi^{(r+1)}$ using the new policy's trajectory distribution and the trajectory-correctness labels.
5. Repeat from step 1.

The structural identity with AlphaZero

deepen-cite: silver2017-alphazero §algorithm (canonical citation pending; KB note: `kb/notes/reasoning/inference-time-search.md#7-intuitions-and-analogies`)

is exact:

- self-play games $\leftrightarrow$ MCTS rollouts on math problems;
- board positions $\leftrightarrow$ partial reasoning prefixes $z_{1:t}$;
- value head $\leftrightarrow$ PRM V_ϕ;
- policy network $\leftrightarrow$ next-step policy π_θ;
- self-play SFT $\leftrightarrow$ SFT on MCTS-discovered correct trajectories.

[49] KB note: `kb/notes/reasoning/inference-time-search.md#2-3-rest-mcts`.

rStar-Math (et al. , Microsoft) runs a structurally similar loop across four self-evolution rounds, with one important variation: the PRM is replaced by a *process preference model* (PPM) trained from preferences over MCTS-discovered step-pairs rather than from absolute step-level scores (et al. , Microsoft, §3).[50] The choice trades absolute calibration for a more robust pairwise signal that does not require pinning the verifier's score scale; the structural role inside the MCTS loop is unchanged.

Analogy

ReST-MCTS* is AlphaZero *with text steps*. Selection, expansion, simulation, and backup are the same four phases. The action space at each tree node is the LLM's next-step distribution $\pi_\theta(s_{t+1} \mid x, z_{1:t})$ rather than the board's legal-move distribution. The value head is the PRM $R^P_\phi(x, s_{1:t})$ rather than the position-evaluation network. The self-play loop is replaced by a generate-MCTS-trajectories-then-SFT-on-correct-ones loop. Returning to math: eq. (18)'s PUCT formula is the same equation that drives AlphaZero's selection, with the policy prior $\pi_\theta(s \mid \text{parent})$ playing the role of the move-prior network's output and $Q(s)$ playing the role of the running-mean win rate. The analogy is load-bearing because it explains why the iterative SFT-on-search-discovered-correct loop converges: each round's MCTS uses a stronger policy and a stronger value head than the last, and the harvested trajectories are strictly better-than-policy on average, supplying a positive-gradient SFT signal that ratchets the policy upward without ever introducing a separate value-network bootstrap.

6.6 MCTS-RAG: tree search over retrieval and reasoning

Hu et al. (2025) extend the MCTS state space to include *retrieval* actions alongside reasoning steps.

deepen-cite: hu2025-mcts-rag §3 (excerpt skeleton-only as of 2026-05; PDF acquisition pending)

Inside an MCTS iteration, the expansion step samples either a next reasoning step from $\pi_\theta(\cdot \mid x, z_{1:t})$ or a retrieval call into a document index, with the choice itself an action under a hybrid policy. The value function V_ϕ is generalised to score states that mix reasoning and retrieved evidence; the terminal verifier $\mathcal{V}$ remains domain-specific (a fact-match check on a knowledge-grounded QA benchmark, an arithmetic check on a calculation-grounded one).

The structural advance: tree search lets the model decide *when* to retrieve and *what* to retrieve as part of the reasoning trajectory, rather than fixing retrieval as a one-shot preprocessing step. The cost analysis of eq. (19) generalises: the per-step token cost L_{step} is now a mixture over reasoning-step length and retrieval-call length (typically shorter, but with an additional index-lookup latency that the cost expression does not capture). MCTS-RAG closes the gap between two otherwise-disjoint inference-time-compute literatures — search-based TTC for reasoning and RAG for knowledge access — under one tree-search framework.

6.7 Search-vs-trained-CoT: the live contradiction

The 2024 hypothesis articulated by the early reasoning-MCTS literature was that explicit search at inference time was *necessary* for strong reasoning performance: a small policy plus heavy MCTS was expected to dominate a large policy without search. The 2025–2026 evidence is more nuanced. et al. (Microsoft, §4) demonstrate the small-policy-with-heavy-MCTS extreme: a 7B policy plus four rounds of MCTS self-evolution reaches 90.0% on MATH and 53.3% on AIME 2024, matching or

[50] KB excerpt: `kb/excerpts/rstar-math2025.md#sec-3`.

exceeding o1-preview at much smaller policy scale.[51] DeepSeek-AI (2025a) demonstrate the opposite extreme: a large policy with RLVR-trained long CoT and *no* explicit inference-time search reaches 77.9% pass@1 on AIME 2024 and 86.7% with self-consistency, on the strength of trace-length growth from ∼200 to ∼10 000 tokens during training (section 8). The two extremes occupy roughly the same accuracy region at very different policy scales and inference-time-compute profiles.

The clean reading[52] is that explicit search is most valuable where the policy is weak (small model, narrow domain, hard tail). At strong policy plus standard distributions, internalised search via long CoT is competitive at matched compute. This matches the difficulty-bin operationalisation of section 4.3: search wins on the hard bin, sampling/refinement wins on the easy bin, and both collapse to the same answer on prompts the policy nearly solves.

Contradiction

BFS-with-PRM vs. MCTS at fixed inference budget. Snell et al. (2024, §5–6) report BFS-V (the level-order beam-with-PRM of section 6.2) competitive with or outright better than alternative search strategies at matched compute on the MATH benchmark with PaLM 2-S as the base model (Snell et al., 2024, §5.2).[a] The implication is that the extra MCTS machinery (UCT bookkeeping, rollout-derived $Q(s)$ updates, PUCT exploration constant c_{puct}) does not necessarily pay for itself at fixed test-time FLOPs. et al. (Microsoft)'s headline framing instead foregrounds MCTS as the central mechanism by which 7B SLMs match o1-preview, and the AlphaZero structural parallel of section 6.5 runs entirely through the MCTS recurrence of eq. (18). Reconciling the two: Snell's evaluation uses a non-reasoning-trained base model and a single round of search; rStar- Math's MCTS is co-trained with the policy and PRM across four rounds of self-evolution (et al. , Microsoft, abstract), so the "MCTS" that wins is a different object from the "MCTS" that ties. Whether the MCTS-vs-BFS gap reverses on reasoning-trained base models, on harder benchmarks, or under multi-round co-evolution is not closed in 2026; the et al. (2025c) taxonomy catalogues the disagreement without resolving it.

deepen-cite: wei2025-tree-search-survey §5 (resolution discussion; excerpt file pending)

[a] KB excerpt: `kb/excerpts/snell2024.md#sec-5-2-search`.

Section 10 returns to the search-vs-RL face of the same tradeoff.

6.8 Forward link

Tree search is the verifier-leg's most expressive realisation. Its cost-per-trajectory dominates self-consistency (section 3) and best-of-N (section 5.2), and its operational complexity dominates beam search; the question is whether it earns its budget. The answer, across section 6.3–section 6.7, depends on the policy's strength, the value function's calibration, and whether the search is co-evolved with the policy (rStar-Math, ReST-MCTS*) or run once over a fixed checkpoint (Snell-style evaluation).

Section 7 introduces the third leg of the reasoning triangle — RL on verifiable rewards — which is the *training-time* answer to the question this section treated at inference: how to spend compute to amplify reasoning. The eq. (19) cost expression and the eq. (18) PUCT recurrence both reappear there in a different role: ReST-MCTS*'s search-then-SFT loop and RLVR's group-of-rollouts both *generate* training trajectories from a current policy, with the verifier $\mathcal{V}$ deciding which trajectories

[51] KB excerpt: `kb/excerpts/rstar-math2025.md#headline`.

[52] KB note: `kb/notes/reasoning/inference-time-search.md#5-the-search-vs-trained-cot-debate`.

survive. Section 10 then puts curves on the tradeoff, asking at fixed total compute when marginal FLOPs are best spent on more inference search (section 6.3), more RLVR training (section 7), or more pretraining (section 4.5). The MCTS-vs-BFS contradiction of section 6.7 and the absent closed-form $L(\theta, N)$ flagged in section 4.7 are two faces of the same open frontier: the search-leg's quantitative contribution to inference-time-compute returns is precisely what remains unresolved in 2026.

7 The RLVR paradigm: GRPO and DAPO

The third leg of the reasoning triangle of section 1 is *reinforcement learning on verifiable rewards* (RLVR). The previous two legs — inference-time compute (section 4) and verifier signals at inference (section 5, section 6) — both spend test-time FLOPs to amplify a fixed policy. RLVR is the training-time leg that *produces* the policy whose test-time compute is worth amplifying. Two coupled substitutions define the paradigm: a programmatic verifier replaces a learned reward model, and a group-mean baseline replaces a co-sized value network. The first removes reward-hacking as a structural failure mode; the second halves the active-parameter footprint of training and makes long-CoT trajectories practical to backpropagate through. The empirical proof point treated in detail in section 8 — DeepSeek-R1-Zero's AIME 2024 climb from 15.6% to 77.9% pass@1 under one continuous GRPO run (DeepSeek-AI, 2025a, §2.3)[53] — is the cleanest single number in the reasoning-LLM literature, and the central evidence that the triangle composes.

This section is deliberately complementary to Paper 2's RL-stage treatment (Paper 2 §11, which covers GRPO/DAPO as one stage of the six-stage post-training pipeline). The angle here is different: how RLVR *shapes* the test-time compute axis. The training reward is terminal and binary, but the policy that emerges is one whose inference-time trace length grows from $\sim$200 tokens to $\sim 10^4$ on its own (DeepSeek-AI, 2025a, §2.3)[54] — the verifier contains no length term, but the policy discovers that long traces win on math and code. RLVR is the training-time mechanism that ratchets the inference-time compute axis upward.

7.1 The verifiable reward

Define the reward for prompt x and sampled output y:

$$R_{\text{verifier}}(x, y) \;=\; \mathbb{1}\big[\,\text{verify}(x, y) = \textsf{true}\,\big] \;\in\; \{0, 1\}, \tag{20}$$

where verify is a deterministic program supplied alongside the prompt (DeepSeek-AI, 2025a, §2.2).[55] The instances that drive the post-2024 reasoning ecosystem are: *math* — exact-match against the canonical answer after sympy-style normalization, on competition benchmarks (AIME, MATH, AMC) whose answer space is small symbolic expressions or integers; *code* — compile + pass attached unit tests, with per-problem suites shipped by competitive-programming benchmarks (LiveCodeBench, Codeforces); *format* — a regex or parser that fires on the `<think>...</think><answer>...</answer>` envelope, used as a small constant add-on reward to keep the reasoning channel alive during early training (DeepSeek-AI, 2025a, §2.2).

The reward is *sparse* (one bit per trajectory) and *terminal* (the final answer is graded, intermediate steps are not, in the RLVR setting; the process-supervised contrast is section 5). Two structural failure modes of learned reward models drop out of eq. (20): reward-hacking the artifacts of

[53] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime`.
[54] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aha`.
[55] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-2-reward`; KB note: `kb/notes/post-training/rlvr-and-grpo.md#1-1`.

r_ϕ, and reward-overoptimization as $\mathrm{KL}(\pi_\theta \| \pi_{\text{ref}})$ grows (Ouyang et al., 2022, scaling laws for overoptimization).

deepen-cite: gao2023-overopt §4 (bibkey not yet in references.bib)

The verifier *is* the gold reward in its supported domains; the proxy–gold gap of RLHF closes by construction. DeepSeek-AI (2025a, §2.2) state the position explicitly: "we abstain from applying neural reward models — whether outcome-based or process-based — to reasoning tasks. This decision is predicated on our observation that neural reward models are susceptible to reward hacking during large-scale reinforcement learning."[56] The post-training-pipeline framing of why this matters is Paper 2 §11; the reasoning-triangle framing is that eq. (20)'s sparseness is what *forces* the policy to discover long-CoT solutions in the first place — the only signal RLVR provides is whether the final answer is correct, so the policy must learn to reach correct answers however it can, including by spending more inference-time tokens on the trace.

7.2 The GRPO objective

Shao et al. (2024) introduce *Group Relative Policy Optimization* (GRPO) in DeepSeekMath as a critic-free PPO variant: per-prompt sampling of a group of G trajectories, with the group's empirical reward distribution as the variance-reducing baseline.[57] The token-level objective (Shao et al., 2024, §4.1, Eq. 3), which DeepSeek-R1 later restates in a sequence-level shorthand at DeepSeek-AI (2025a, §2.1, Eq. 1)[58]:

$$\begin{aligned}\mathcal{J}_{\text{GRPO}}(\theta) \;=\; \mathbb{E}_{q\sim P(Q),\,\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)}\Bigg[\\ \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\Big(r_{i,t}(\theta)\,\hat{A}_{i,t},\;\text{clip}\big(r_{i,t}(\theta),1-\epsilon,1+\epsilon\big)\,\hat{A}_{i,t}\Big) \\ -\;\beta\;\mathrm{KL}\big[\pi_\theta\,\|\,\pi_{\text{ref}}\big]\Bigg],\end{aligned} \tag{21}$$

with importance ratio $r_{i,t}(\theta) = \pi_\theta(o_{i,t} \mid q, o_{i,<t}) \,/\, \pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})$ and *group-normalized advantage*

$$\hat{A}_{i,t} \;=\; \tilde{A}_i \;=\; \frac{R_i - \text{mean}(\{R_1,\dots,R_G\})}{\text{std}(\{R_1,\dots,R_G\})}, \tag{22}$$

constant in t across the trajectory (Shao et al., 2024, §4.1, Eq. verified against R1 §2.1 Eq. 3).[59]

Symbol table. q is the prompt; G the group size, typically 8–64 ($G = 16$ in R1-Zero (DeepSeek-AI, 2025a, §2.1)[60]); o_i the i-th sampled trajectory (chain-of-thought + answer); $R_i \in \{0, 1\}$ the verifier reward of eq. (20); π_{ref} a frozen reference (typically the SFT cold-start, see Paper 2 §SFT); β the KL coefficient (0.001 for R1-Zero (DeepSeek-AI, 2025a, §2.1)); $\epsilon = 0.2$ the PPO clip range. The KL is computed token-wise with Schulman's unbiased estimator, $\mathrm{KL} = \pi_{\text{ref}}/\pi_\theta - \log(\pi_{\text{ref}}/\pi_\theta) - 1$ (Shao et al., 2024, §4.1).

[56] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-2-reward`.
[57] KB excerpt: `kb/excerpts/shao2024.md#sec-4-1-motivation`.
[58] KB excerpts: `kb/excerpts/shao2024.md#sec-4-1` and `kb/excerpts/deepseek-r1.md#sec-2-1-grpo`.
[59] KB excerpt: `kb/excerpts/shao2024.md#sec-4-1`.
[60] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-1-hp`.

Operational consequence: no value network. The single load-bearing fact about GRPO at LLM scale is that eq. (22)'s $\tilde{A}_i$ is a function only of the group's empirical rewards $\{R_1, \ldots, R_G\}$ — there is no $V_\phi(s_t)$. PPO with GAE (Ouyang et al., 2022, §3.5)[61] requires a value network co-sized with the policy, doubling the active parameter count and the activation memory of training. GRPO removes the second network entirely. With value model: $\sim 2P$ active parameters and $\sim 2A$ activation memory per training step (where P is policy parameters and A its activation footprint), plus the value-network's own optimizer state. Without (GRPO): $P + A$ for the policy, P frozen for the reference, but $G\times$ rollouts per prompt at sequence length up to $L_{\max}$. The four co-resident model copies of RLHF's PPO infrastructure (policy, reference, reward, value) reduce to two (policy, reference); the verifier is a CPU subprocess, not a model. The trade is G rollouts of length $\leq L_{\max}$ per prompt against the value-network's parameter and activation footprint, and at sparse reward (where V_ϕ struggles to learn anyway) it is favorable (Shao et al., 2024, §4.1).[62] For reasoning training where $L_{\max} \in [10^4, 6.5 \times 10^4]$ (DeepSeek-AI, 2025a, §2.1), this is the difference between fitting the training run on a given cluster and not.

Intuition

GRPO is the median-of-rollouts as a policy gradient. Sample G trajectories from the current policy on prompt q, score each, and ask: which were *better than typical for this prompt*? Up-weight the better-than-mean log-probs, down-weight the worse-than-mean ones. The sign of $\tilde{A}_i$ in eq. (22) answers the question; its magnitude (in standard-deviation units of the group's reward distribution) is the step size. Returning to canonical form: eq. (22) is exactly the standardization of R_i within its group, and eq. (21) is the PPO-clipped surrogate with that standardized advantage broadcast over the trajectory's tokens. The *inference* interpretation, central to this paper's triangle, is that the group-mean is an empirical estimate of $p_0(q)$ — the fraction of trajectories the current policy gets right at this prompt — and GRPO's gradient is a trajectory-level importance-weighted update on the better-than-$p_0(q)$ rollouts. Under the various (2025) reading discussed in section 14, this implies RLVR cannot make the policy solve a problem it has *never* solved at any sample budget; it can only sharpen the distribution toward the rollouts that already succeed — a claim contested by the R1 narrative (DeepSeek-AI, 2025a, §2.3) and not yet decisively settled.

The intuition's last clause foreshadows the various (2025) contradiction (section 14, also discussed in Paper 2 §11): if the group-mean baseline is the binding mechanism, then RLVR's reach is bounded by the base policy's pass@K at any K, not by its pass@1. The 15.6 → 77.9 AIME climb is then read as "the base model could already solve 77.9% of these problems *at some sample budget*; GRPO transferred that to pass@1." Whether this exhausts the picture or merely captures the dominant term is the open question.

7.3 DAPO: four targeted refinements

Seed (2025) ("Decoupled clip and dynamic s**A**mpling **P**olicy **O**ptimization," ByteDance Seed, March 2025) introduce four modifications to GRPO that together cut training-step count to AIME-2024 score 50 by roughly half on Qwen2.5-32B-base relative to the GRPO baseline (Seed, 2025, §4).[63] Each modification addresses a measured failure mode of the GRPO recipe under the specific demands

[61] KB excerpt: `kb/excerpts/ouyang2022-instructgpt.md#sec-3`.
[62] KB excerpt: `kb/excerpts/shao2024.md#sec-4-1-motivation`.
[63] KB excerpt: `kb/excerpts/dapo2025.md#sec-4-headline`.

of long-CoT training; together they are the canonical 2025 statement of where the GRPO objective leaks gradient signal.

(i) Clip-Higher (Eq. 10). GRPO's symmetric clip $[1-\epsilon, 1+\epsilon]$ caps probability *increases* on under-probable exploration tokens at the same rate as it caps *decreases* on already-favored tokens. DAPO decouples the bound:

$$\mathcal{J}_{\text{DAPO}}(\theta) \;=\; \mathbb{E}\left[\frac{1}{G}\sum_{i}\min\Big(r_{i,t}(\theta)\,\hat{A}_i,\;\text{clip}\big(r_{i,t}(\theta),1-\epsilon_{\text{low}},1+\epsilon_{\text{high}}\big)\,\hat{A}_i\Big)\right], \tag{23}$$

with $\epsilon_{\text{low}} = 0.2$ kept at the PPO default and ϵ_{high} enlarged (e.g., 0.28) (Seed, 2025, Eq. 10).[64] The asymmetry permits larger upward updates on under-probable correct tokens and prevents collapse onto already-favored trajectories — collapse being the failure mode in which the entropy of the policy at long-CoT positions decays before the policy has explored alternative reasoning paths.

(ii) Dynamic Sampling (Eq. 11). Whenever all G trajectories in a group share the same reward (all-correct or all-incorrect), the group-normalized advantage eq. (22) is identically zero by construction and the sample contributes no gradient. DAPO enforces

$$0 \;<\; \big|\{\, o_i : \text{is\_equivalent}(a, o_i)\,\}\big| \;<\; G \tag{24}$$

(Seed, 2025, Eq. 11), discarding and re-sampling groups that fail the constraint.[65] Compute concentrates on informative gradients. The structural reason this matters under the reasoning-triangle framing is that as the policy's pass@1 climbs through training, the all-correct degenerate group becomes more frequent on easy prompts; without dynamic sampling, an increasing fraction of the rollout budget produces no learning signal.

(iii) Token-Level Loss (Eq. 12). GRPO normalizes by $1/|o_i|$ inside the group sum and then by $1/G$ outside, so longer trajectories contribute proportionally less per token. DAPO replaces this with token-aggregated normalization:

$$\mathcal{L}_{\text{DAPO}}(\theta) \;=\; -\frac{1}{\sum_{i=1}^{G}|o_i|}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}L_{i,t}(\theta). \tag{25}$$

Every token weighs equally regardless of trajectory length (Seed, 2025, Eq. 12).[66] This is the modification most directly aligned with the reasoning-triangle thesis of this paper: GRPO's per-trajectory normalization systematically under-weights the long correct CoTs that section 7.2's response-length growth produces, which is precisely the regime the inference-time-compute axis cares about. DAPO's token-level loss removes that bias.

(iv) Overlong Reward Shaping (Eq. 13). Trajectories that hit $L_{\max}$ are typically penalized as failures, but the failure signal is uninformative — the model just ran out of budget. DAPO replaces the hard zero with a length-based soft penalty:

$$R_{\text{length}}(o_i) \;=\; \begin{cases} 0 & |o_i| \le L_{\text{accept}}, \\ \text{linear interp.} & L_{\text{accept}} < |o_i| < L_{\max}, \\ -1 & |o_i| \ge L_{\max}, \end{cases} \tag{26}$$

[64] KB excerpt: `kb/excerpts/dapo2025.md#sec-3-clip-higher`.
[65] KB excerpt: `kb/excerpts/dapo2025.md#sec-3-dynamic-sampling`.
[66] KB excerpt: `kb/excerpts/dapo2025.md#sec-3-token-loss`.

(Seed, 2025, Eq. 13).[67] Variance from length-truncated samples falls; the genuine signal that overlong reasoning is bad is preserved. The interaction with eq. (25) is structural: without eq. (25)'s token-level normalization, the soft penalty would still be aggregated against the longest trajectories, restoring the bias eq. (25) just removed.

Analogy

The four DAPO modifications can be read as the field's $\mathcal{O}(\epsilon^2)$ correction to GRPO once the long-CoT regime made the leading-order behavior visible. Symmetric clipping, equal-weight trajectories, hard length penalties, and uniform group acceptance are all reasonable defaults at trajectory length $\sim 10^2$ tokens; at $\sim 10^4$ tokens each defaults to a small bias whose accumulation shows up as wasted gradient steps. Returning to canonical form: each DAPO equation localises one of those biases. eq. (23) is the asymmetric trust-region around the importance ratio, eq. (24) the support condition for eq. (22) to have nonzero variance, eq. (25) the un-biased token-aggregator that replaces GRPO's per-trajectory normalisation, and eq. (26) the length-conditioned reward that decouples "ran out of budget" from "failed."

Per-step procedure (GRPO and DAPO share this). For each batch of B prompts: (1) sample G trajectories per prompt from $\pi_{\theta_{\text{old}}}$ at temperature $T \in [0.7, 1.0]$ capped at $L_{\max}$ tokens (R1-Zero uses $L_{\max} = 32{,}768$ before step 8.2k and 65,536 afterward (DeepSeek-AI, 2025a, §2.1)); (2) score with the verifier; (3) compute $\tilde{A}_i$ via eq. (22) and broadcast token-wise (or apply DAPO's dynamic-sampling and token-level normalization); (4) take $K \in \{1, 4\}$ PPO inner-loop steps on the buffered $(B \cdot G)$ trajectories; (5) refresh $\theta_{\text{old}} \leftarrow \theta$. The verifier is a CPU subprocess; rollouts dominate wall-clock time.

7.4 Empirical headline: R1-Zero and DAPO

Two evidence points define the 2025 RLVR-for-reasoning literature. First, DeepSeek-AI (2025a, §2.3)[68] report that DeepSeek-R1-Zero's average pass@1 on AIME 2024 climbs from 15.6% to 77.9% over one continuous GRPO run on the DeepSeek-V3 base, with 86.7% achieved at majority-vote-over-64, "surpass[ing] the average performance across all human competitors." The full run is 10,400 steps, ~ 1.6 epochs over the training distribution (DeepSeek-AI, 2025a, §2.1); the only signal is the binary verifier of eq. (20) plus a small format reward. The detailed protocol — cold-start SFT, the rejection-sampling SFT, the final helpfulness-and-harmlessness RL stage — is the subject of section 8; here we cite the headline as the proof that the RL leg of the triangle, alone, can produce frontier-grade reasoning on verifier-supported domains.

Second, Seed (2025, §4)[69] reach AIME 2024 score 50 on Qwen2.5-32B-base in roughly half the training steps relative to the GRPO baseline using the four modifications of section 7.3. The training code, curated dataset, and verl framework integration are open-sourced. As of 2026-Q2 DAPO and its descendants are the practical default for open-weight RLVR on long-CoT tasks; GRPO remains the canonical reference and the simpler starting point.

The *cross-leg* reading of these numbers is the section's thesis: the response-length growth observed during the R1-Zero run (a few hundred tokens early in training to roughly 10,000 tokens at convergence (DeepSeek-AI, 2025a, §2.3)[70]) is, by definition, movement along the inference-time-

[67] KB excerpt: `kb/excerpts/dapo2025.md#sec-3-overlong`.
[68] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime`.
[69] KB excerpt: `kb/excerpts/dapo2025.md#sec-4-headline`.
[70] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aha`.

compute axis of section 4; and the "aha moment" — a sudden spike in the policy's use of `wait` during reflection — is self-reflection patterns that interact with verifier signals at inference (section 6). The same RLVR run thus moves the policy along all three legs simultaneously: train against eq. (20), get a longer trace, get more self-verification structure for free. section 10 returns to the operational tradeoff between spending marginal compute on RLVR training versus inference search; the headline above sets the training-side anchor of that tradeoff.

7.5 Open contradiction: VAPO — are value models really useless?

Contradiction

Did GRPO show that value models are useless, or that *naïve* value models are useless? VAPO (Value-Augmented PPO, ByteDance Seed, April 2025, arXiv:2504.05118) re-introduces a learned value function under the GRPO/RLVR objective, but trains it more carefully than GAE-PPO: shared backbone with the policy, value-loss-clipped, advantage normalized per-batch. The paper reports VAPO outperforming DAPO on long-CoT tasks at fewer steps, suggesting the GRPO-vs-PPO gap is driven not by "V_ϕ is the wrong tool" but by "V_ϕ trained the way GAE-PPO trains it is the wrong tool." The reasoning-triangle reading would split: if VAPO replicates, group-baselines are an implementation choice, not a structural feature of the leg; if it does not, GRPO's critic-free design is part of why RLVR scales to 10^4-token traces in the first place. Whether VAPO's gain over DAPO replicates outside the original recipe is open as of 2026-Q2; only the original paper has reported the comparison, and follow-up evidence is limited to a small set of reproductions. Cross-paper to Paper 2 §11 (the same contradiction in post-training-pipeline framing) and section 14 (the unified resolution attempt across the three legs).

deepen-cite: vapo2025 — bibkey not yet in references.bib; arXiv:2504.05118 §3 replication evidence

The VAPO contradiction has a near-neighbor on the credit-assignment side: VinePPO

deepen-cite: kazemnejad2025-vineppo — bibkey not yet in references.bib; arXiv:2410.01679 §3

keeps a value baseline but replaces V_ϕ's per-token bootstrap with Monte Carlo re-simulation, sampling M continuations from each intermediate state and averaging their rewards. REINFORCE-class methods (REINFORCE++, RLOO, GRPO-no-clip) drop the PPO clip entirely, arguing the KL anchor and small per-step learning rate already provide the trust region.

deepen-cite: ahmadian2024-rloo — bibkey not yet in references.bib

The empirical gaps between these variants are small in 2025–2026 reports on math and code; whether the gap survives transfer to agentic and long-horizon RLVR is single-source. The unifying observation is that eq. (22)'s group-mean baseline is the simplest member of a family of zero-bias variance-reducers, and the open question is which family member best amortises the cost of long-CoT rollouts.

7.6 Forward link

This section established the third leg of the reasoning triangle in isolation: the verifiable reward of eq. (20), the GRPO objective of eq. (21) with its critic-free group-mean advantage eq. (22), the four DAPO modifications eq. (23)–eq. (26), and the empirical proof points: R1-Zero's AIME climb on GRPO and DAPO's halved step-count to matched score on Qwen2.5-32B-base. The post-training-pipeline framing of the same machinery (Paper 2 §11) treats RLVR as the successor

to RLHF in the six-stage training pipeline; this section's framing instead treats the RL leg as the upstream cause of the inference-time compute axis the previous sections of this paper studied.

Section 8 is the empirical proof point that the triangle *composes*. R1's four-stage pipeline — cold-start SFT, GRPO with verifiable rewards, rejection-sampling SFT, general-domain RL — combines the RL leg of this section with the inference-compute axis of section 4 and the verifier signals of section 5, and the resulting model is the strongest single piece of evidence for the triangle as a co-design rather than three independent gains. Section 10 picks up the operational tradeoff this section deliberately did not close: at fixed total compute, when does the marginal FLOP go to RLVR training versus inference search versus pretraining? The open answer turns on the same empirical anecdotes (DAPO's step efficiency (Seed, 2025, §4), R1's training-token cost (DeepSeek-AI, 2025a, §2.3), Snell's FLOPs-matched curves (Snell et al., 2024, §6)[71]) that section 7.4 cited as headlines, now read against each other rather than in isolation.

8 DeepSeek-R1 and the reasoning-trained model family

If sections 4, 5 and 7 each treated one leg of the reasoning triangle in isolation, this section is the empirical proof point that the three legs *compose*. DeepSeek-AI (2025a) train DeepSeek-R1 and DeepSeek-R1-Zero on the DeepSeek-V3 base (DeepSeek-AI, 2025a, §2)[72] using a four-stage pipeline whose RL phase moves the policy along the inference-compute axis (response length grows from $\sim$200 to $\sim 10^4$ tokens (DeepSeek-AI, 2025a, §2.3)[73]) using only the verifiable terminal reward of eq. (20), with no PRM in the headline RL loop (DeepSeek-AI, 2025a, §2.2).[74] The headline number — AIME 2024 average pass@1 climbs from 15.6% to 77.9%, reaching 86.7% with self-consistency over 64 samples (DeepSeek-AI, 2025a, §2.3)[75] — is the cleanest single piece of evidence in the 2025–2026 reasoning literature that the triangle is a co-design rather than three independent gains that happen to ship together.

This section walks the four-stage R1 protocol, contrasts R1 with R1-Zero (no cold start) to isolate what the SFT initialisation does and does not contribute, transcribes the long-CoT compounding curve that links response length to verifier accuracy, and traces the distillation track and the open-weight reasoning ecosystem (Tülu 3, Qwen3-thinking, QwQ, OpenR1) that R1 seeded. It closes on the AIME-2024 contamination contradiction, the load-bearing open question about whether the headline number is the gain it appears to be.

8.1 The four-stage R1 protocol

DeepSeek-AI (2025a, §2.3) establish a four-stage training protocol for DeepSeek-R1 that subsequent open-replication work largely follows.[76] The pipeline is the operational form of the reasoning triangle: each stage moves the policy along a specific leg, and the legs compose in a fixed order.

Stage 1 — cold-start SFT. A small set ($\sim 10^4$ examples) of curated long-CoT exemplars is used for SFT on the DeepSeek-V3 base (DeepSeek-AI, 2025a, §3)[77] The cold-start data is described

[71] KB excerpt: `kb/excerpts/snell2024.md#sec-1-flops-matched`.
[72] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2`.
[73] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aha`.
[74] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-2-reward`.
[75] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime`.
[76] KB note: `kb/notes/reasoning/reasoning-training.md#2`.
[77] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-3-pipeline`.

by DeepSeek-AI (2025a, §3) as "thousands of cold-start data that exhibits a conversational, human-aligned thinking process." The SFT pass installs the `<think>...</think><answer>...</answer>` format (DeepSeek-AI, 2025a, §2)[78] that the format reward of section 7.1 subsequently maintains, and seeds the language-consistency property that R1-Zero's purely-RL-trained traces lack (section 8.2).

Stage 2 — reasoning-oriented RL (GRPO with verifiable rewards). The cold-started policy is RL-trained with the GRPO objective eq. (21) of section 7.2, using the binary verifier reward of eq. (20) on math, code, and STEM problems with mechanically-checkable answers (DeepSeek-AI, 2025a, §2.2).[79] The verifier is a sum of an *accuracy reward* (binary, from sympy-style exact-match for math, unit-test execution for code, regex for multiple-choice) and a small constant *format reward* for the `<think>...</think>` envelope (DeepSeek-AI, 2025a, §2.2).[80] R1's protocol explicitly excludes neural reward models: DeepSeek-AI (2025a, §2.2) state "we abstain from applying neural reward models—whether outcome-based or process-based—to reasoning tasks" on the grounds that learned RMs are reward-hackable under large-scale RL.

The empirical content of Stage 2 is the headline of the entire R1 paper. Hyperparameters (DeepSeek-AI, 2025a, §2.1)[81]: learning rate 3×10^{-6}, KL coefficient $\beta = 0.001$, sampling temperature $T = 1$ for rollout, group size $G = 16$, max trajectory length $L_{\max} = 32{,}768$ tokens before step 8.2k and 65,536 afterward, 10,400 training steps total (≈ 1.6 epochs over the training distribution). Over this run, two coupled quantities move together: average response length grows from a few hundred tokens to $\sim 10^4$, and AIME 2024 average pass@1 grows from 15.6% to 77.9% (DeepSeek-AI, 2025a, §2.3).[82] DeepSeek-AI (2025a, §2.3) additionally describe an "aha moment" — a sudden spike in the policy's use of the `wait` token during reflection passages.[83] The verifier is agnostic to length and to reflection structure; the policy discovers both as instrumentally useful for hitting the binary reward.

Stage 3 — rejection-sampling SFT. The Stage-2 RLVR policy generates a large pool of solutions; the verifier filters for correctness and a writing-quality filter further prunes the set, yielding $\sim$ 600K reasoning traces that are concatenated with $\sim$ 200K non-reasoning examples (general chat, safety formatting, non-reasoning task prompts) into an $\sim$ 800K mixed SFT set (DeepSeek-AI, 2025a, §3)[84] The Stage-2 policy is fine-tuned on this set with standard cross-entropy SFT. This stage is the bridge between "specialist reasoner from Stage 2" and "general assistant that also reasons," and it is where the reasoning capability becomes expressible as supervised data — the point that makes the distillation track of section 8.4 possible at all.

Stage 4 — general-domain RL. A second RL phase combines verifiable rewards (for the reasoning subset of prompts) with preference-based rewards (for helpfulness, harmlessness, instruction following on the broader prompt distribution) (DeepSeek-AI, 2025a, §3).[85] The released DeepSeek-R1 weights are the output of this stage. Only Stage 2 of the four is purely RLVR; Stage 4 is an RLVR/RLHF hybrid whose preference component is the same machinery as the "RL stage" that bookends post-training in Paper 2 §RLHF / §RLVR. The two SFT stages (Stage 1 and Stage 3)

[78] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2`.
[79] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-2-reward`.
[80] KB note: `kb/notes/post-training/rlvr-and-grpo.md#2-3`.
[81] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-1-hp`.
[82] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime`.
[83] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aha`.
[84] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-3-pipeline`.
[85] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-3-pipeline`.

bookend the reasoning RL phase: cold-start SFT seeds it, rejection-sampling SFT consolidates its outputs.

8.2 R1-Zero versus R1: what does cold-start add?

DeepSeek-R1-Zero is the same Stage-2 RLVR run applied to the DeepSeek-V3 base directly, without Stage 1's cold-start SFT (DeepSeek-AI, 2025a, §2).[86] The structural comparison cleanly isolates what the cold-start contributes. R1-Zero reaches *the same reasoning capability headline* — 15.6%→77.9% AIME 2024 pass@1, 86.7% with self-consistency (DeepSeek-AI, 2025a, §2.3)[87] — but its traces are unreadable: "DeepSeek-R1-Zero struggles with challenges like poor readability, and language mixing, as DeepSeek-V3-Base is trained on multiple languages, especially English and Chinese" (DeepSeek-AI, 2025a, §3).[88] The traces interleave Chinese and English at the token level; the format envelope drifts; the prose is brittle. Capability and legibility decouple.

The cold start, on this evidence, contributes three things (DeepSeek-AI, 2025a, §3)[89]: *format* (the `<think>...</think><answer>...</answer>` envelope is seeded so the format reward of section 7.1 acts on a non-zero baseline rather than carving the envelope from scratch); *language consistency* (a single language in the trace because the cold-start exemplars are monolingual); and *regularisation* (a starting point inside the manifold of human mathematical writing rather than at the multilingual base distribution's mode). What it does *not* contribute is reasoning capability: that emerges in Stage 2 either way.

Intuition

R1-Zero is the pure-RL reasoning experiment; R1 is the same experiment with a few thousand pages of human reasoning-style writing mixed in at initialisation. Capability tracks the verifier signal; legibility tracks what the policy was writing-like before that signal arrived. Returning to math: GRPO's group-baseline eq. (22) only sees the binary verifier reward, so the gradient is identical for a Chinese-English-mixed trace and a clean-English trace at the same correctness label. The cold-start biases the policy's initial π_θ toward one of those styles; the RL phase preserves the bias because the gradient does not punish it.

The R1-Zero / R1 split mirrors a long-standing pattern in RL: the all-RL agent reaches comparable capability to the SFT-then-RL agent, at the cost of some surface property the latter inherited from the supervised initialisation.[90] The reading available from R1's evidence: a small-but-curated supervised initialisation substantially shortens the RL horizon to legibility without changing the asymptote on capability.

8.3 The long-CoT compounding curve

The empirical signature of Stage 2 that links R1's evidence to the inference-compute axis of section 4 is the *long-CoT compounding effect*: response length and verifier accuracy grow together over the RL run, in a relationship that is qualitatively monotone-concave in $\log L$.[91] Formally, let $L = |o|$

[86] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2`.
[87] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime`.
[88] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-3-pipeline`.
[89] KB note: `kb/notes/reasoning/reasoning-training.md#2`.
[90] KB note: `kb/notes/reasoning/reasoning-training.md#5` draws the AlphaGo / AlphaGo-Zero analogy in this form.
[91] KB note: `kb/notes/reasoning/reasoning-training.md#2-stage-2`; KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime` and `#sec-2-3-aha`.

denote the trajectory's token length and $p(L) = \mathbb{E}_{q,o||o|\approx L}[\mathbb{1}\{\text{verify}(q,o) = \texttt{true}\}]$ the verifier pass rate at that length, both measured as the policy evolves across training. DeepSeek-AI (2025a, §2.3) report:

$$p(L) \;\approx\; a \log L + b, \qquad L \in [\sim 200,\ \sim 10^4], \tag{27}$$

qualitatively (no closed-form fit is reported in the paper; the curve is shown as Figure 3 of DeepSeek-AI (2025a)). The same qualitative shape is reported by et al. (Stanford+) on a small reasoning-trained model under explicit budget forcing (section 9), and by the Snell-anchor empirics (Snell et al., 2024, §5–6)[92] on test-time-strategy curves at fixed base models. As of 2026-Q2 no parametric scaling law for $p(L)$ with the structural status of Chinchilla's $L(N, D)$ has been attested in the literature; the contradiction is catalogued at the paper level in section 7.4 and revisited in §14.

Shape and tensor notes. The training-time monotone-concave curve eq. (27) is a *joint* property of the policy and the verifier: the policy emits longer traces because longer traces hit the binary reward more often on the prompts the dataset samples; the verifier supplies the signal that makes longer traces cost-justified. Trace-length growth is therefore not a separate training objective with its own loss term — the GRPO objective eq. (21) contains no length term — but a fixed-point of the RL dynamics under sparse verifiable reward (DeepSeek-AI, 2025a, §2.2.2).[93] The shape of the curve at inference time, holding the policy fixed and varying L via budget forcing, is the subject of section 9; the shape during training, with the policy itself moving along L, is the subject of this section.

Analogy

The trace-length growth of Stage 2 is a policy-improvement fixed point under sparse reward: longer trace ⇒ more chances to self-correct ⇒ higher verifier pass rate ⇒ GRPO gradient prefers the longer-trace mode of the rollout distribution ⇒ next iteration's expected length increases. Returning to canonical form: in eq. (22) the trajectory-level $\tilde{A}_i$ is positive on the i-th rollout exactly when R_i is above the group's mean; on prompts where the long-trace rollouts hit $R_i = 1$ more often than the short-trace ones, those long-trace rollouts carry positive standardised advantage and pull the policy's expected length upward through repeated iterations of eq. (21). The dynamics are the verifier-conditioned analogue of standard policy-gradient ascent on a reward whose support is a subset of "what the policy already does sometimes" — the same mechanism flagged in section 7.2's intuition box about RLVR's reach being bounded by the base policy's pass@K at any K (various, 2025).

The long-context architectural prerequisite is non-negotiable. Trace lengths in $[10^4, 6.5\times10^4]$ tokens (DeepSeek-AI, 2025a, §2.1) require an attention substrate that costs sub-quadratically per token — some combination of MLA, GQA, or sliding-window attention — to make the per-step decoder inference cost tractable at training-time rollout volume. Paper 1 §long-context-attention is the precondition for this section's 10^4-token traces; without the architectural innovations of DeepSeek-V2/V3 there is no policy that can be RL-trained on trajectories of this length within feasible compute.

[92] KB excerpt: `kb/excerpts/snell2024.md#sec-5-2-search`.
[93] KB note: `kb/notes/reasoning/reasoning-training.md#5`.

8.4 The distillation track and the open-weight ecosystem

DeepSeek-AI (2025a, §4)[94] include a separate track that distills R1 into smaller dense bases via *pure SFT on the 800K Stage-3 set* (the same 600K reasoning + 200K non-reasoning data of section 8.1). The distillation targets span the 1.5B–70B range using Qwen2.5 and LLaMA-3 bases. The notable empirical result: the distilled 32B model matches or exceeds OpenAI's o1-mini on multiple reasoning benchmarks, and the distilled 7B is competitive with strong baselines at 7B scale, all without any RL on the small models themselves (DeepSeek-AI, 2025a, §4). The implication DeepSeek-AI (2025a) highlight is structural: *the reasoning capability that Stage 2 elicits is expressible as a (large) SFT dataset.* The small-model trainer does not need to run RLVR; the trainer at the teacher scale does, but only once.

Intuition

At small scales the bottleneck on long-CoT capability is data — trace exemplars at the right depth and difficulty — not algorithm. Once a teacher emits a sufficient corpus of correct long-CoT traces verified against ground truth, vanilla SFT on those traces transfers the capability efficiently. RLVR's role in the lineage is to *produce* that corpus when no prior trace-generating distribution exists; the RL is needed once at teacher scale, not at every student scale. Returning to math: the SFT cross-entropy on a verifier-filtered Stage-3-style trace set is an upper bound on the KL divergence the student would need to traverse to reach the teacher's verifier-conditional distribution; once the bound is small enough, the student inherits the teacher's pass-rate without on-policy verifier signal.

The distillation track seeded a measurable open-weight reasoning ecosystem in 2025–2026, of which the most visible entries are: **Tülu 3** (et al. , AI2) — a reproducible RLVR-on-Llama recipe with verifier rewards on math/code prompts, released alongside training data; **OpenR1**

deepen-cite: openr1 — citation-pending

— a Hugging Face community project that reproduced R1-Zero-style runs on smaller bases with public code; **Qwen3-thinking**

deepen-cite: qwen3-thinking — citation-pending; cf. Team and Alibaba (2025) for the broader Qwen3 lineage

— a single model that toggles between thinking and non-thinking modes via system prompt, with mode-tagged post-training; **QwQ**

deepen-cite: qwq — citation-pending

— Qwen's standalone reasoning preview that predates Qwen3-thinking. The structural commitment shared across this ecosystem is the R1 protocol's order: cold-start SFT, then RLVR on verifiable rewards, then either rejection-sampling SFT (open-weight reproductions) or its functional equivalent (commercial closed-weight reasoning models). section 13 returns to which entries in the ecosystem ship which legs of the triangle and at what budgets.

8.5 The AIME-2024 contamination contradiction

The R1 headline number is the cleanest single piece of empirical evidence for the reasoning triangle's composability, which makes its contamination status the load-bearing open question of this section.

[94] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-4`.

Contradiction

Was a fraction of AIME-2024-style problems plausibly in DeepSeek-V3's pretraining corpus? The 15.6%→77.9% AIME 2024 pass@1 jump (DeepSeek-AI, 2025a, §2.3) attributes the gain to RLVR-elicited reasoning. The competing reading is that AIME-style problems and their solutions are widely indexed on the open web — in problem-set archives, contest-prep textbooks, math forums, and AoPS-style community posts — and that *some fraction* of the pretraining-corpus distribution overlaps with the test distribution. Under that reading, part of the headline gain is recall of memorised solutions surfaced through long-CoT self-prompting, not novel reasoning composed under RL. Reported contamination-detection methodologies (n-gram overlap on solutions, near-duplicate matching, decontamination-by-substring) all have known false-negative modes against post-hoc-paraphrased problems and arithmetic isomorphisms (date shifts, variable renames). DeepSeek-AI (2025a) do not report a contamination audit on the AIME 2024 train/test boundary, and as of 2026-Q2 no third-party study has reproduced an audit on R1's pretraining mix. The cross-paper treatment of contamination methodology is Paper 5 §contamination, which catalogues the methodologies that disagree on what counts as contamination, and section 14 returns to this specific AIME-2024-on-R1 question with the post-2026 evidence base.

The contamination contradiction interacts non-trivially with the various (2025) "no new capabilities" result of section 7.2 and the contradictions of §14. If the base model's pass@K at moderate K on AIME-style problems is already high because of pretraining contamination, then the various reading — RLVR sharpens pass@1 but does not expand the solvable set — becomes near-tautological on this benchmark. The R1 evidence does not distinguish "RL elicited reasoning capability that the base model could not realise on its own at any sample budget" from "RL surfaced memorised solution patterns the base model held latently from pretraining." Both readings are compatible with the headline curve; only a contamination-controlled rerun on a held-out reasoning benchmark would discriminate.

8.6 Forward link

This section established the empirical proof point that the reasoning triangle composes: the four-stage R1 protocol moves the DeepSeek-V3 base from non-reasoning capability to AIME-2024 pass@1 of 77.9% (and 86.7% with self-consistency) using GRPO eq. (21) on the verifier reward eq. (20), with no PRM in the headline RL loop and no neural reward model anywhere in the pipeline (DeepSeek-AI, 2025a, §2.2). The R1-Zero comparison isolates cold-start SFT's role to legibility and format rather than capability; the long-CoT compounding curve eq. (27) links the response-length growth observed during training to the inference-compute axis of section 4; the distillation track (DeepSeek-AI, 2025a, §4) demonstrates that the elicited reasoning capability is expressible as a supervised dataset once.

Three downstream sections pick up directly from R1's evidence. Section 9 generalises the s1 (et al. , Stanford+) minimal recipe — a small model, 1K reasoning SFT examples, and explicit budget-forcing of the `<think>` channel — as the inference-time complement to R1's training-time long-CoT discovery, and tests whether the same monotone-concave-in-log L shape that eq. (27) observed during training holds at inference on a fixed reasoning-trained policy. Section 11 opens the question whether R1's traces causally drive the answers — whether the verifier's binary signal selects for traces z such that $p_\theta(y \mid x, z) \neq p_\theta(y \mid x, z')$ on intervention-corrupted z' — which is the load-bearing question for any downstream use of CoT as a probe surface (Paper 4 in toto)

or a monitorable channel (Paper 5 §alignment-faking). Section 12 re-asks the contradiction of section 5.7: given that R1 achieved its headline with terminal verifier rewards alone, do PRMs survive the reasoning-trained-model era as a training signal, or only as inference-time rerankers? The R1 evidence is consistent with either reading; the distinguishing empirics live in the post-R1 PRM literature catalogued in section 12.

9 Test-time scaling beyond CoT: think-tokens, budget forcing, reasoning architectures

Sections 4 and 6 treat inference-time compute as a *sampling* or *search* budget — N parallel traces, B branches, N_{rollout} MCTS simulations. A third family holds $N = 1$ but lengthens the single trace, spending compute inside one autoregressive decode rather than across many. This is the o1 / R1 / QwQ / Gemini-2.5-thinking paradigm.[95] et al. (Stanford+) establish the minimum recipe for it: 1,000 curated long-CoT exemplars for SFT, plus a **budget forcing** rule that appends the literal token `Wait` to extend thinking and appends an end-of-thinking delimiter to truncate it (et al. , Stanford+, §3.1).[96] The exercise isolates *which* of the moving parts in a reasoning-trained model (section 8) is doing the test-time scaling work, and in doing so locates a **reasoning-architecture** axis that is separate from the base-architecture axis of Paper 1: the same Transformer, but a training-and-inference protocol that designates `<think>...</think>` as a structurally distinct phase from the answer phase. This section formalises the thinking-token budget L as the operative compute knob, transcribes the s1 mechanism, draws the training-time vs inference-time line that separates s1 from R1 cold start, and forwards to section 10's fixed-compute trade-off.

9.1 Thinking-token budget L

Let x be a problem and $z = (z_1, \ldots, z_L)$ a thinking trace emitted before the answer phase. *Sequential* test-time compute holds $N = 1$ but treats L as the explicit compute knob:

$$z \sim p_\theta(\cdot \mid x;\ \text{length budget } L), \qquad y \sim p_\theta(\cdot \mid x,\ z). \tag{28}$$

The compute cost grows in L along two terms: a per-token decode that is itself $O(L)$ in the prefix-attention pass, so total cost is $\Theta(L^2)$ in the trace-length-bound regime where the KV cache dominates.[97] At inference time the trace is bracketed by special tokens `<think>` ... `</think>`, after which the model emits the answer; the brackets are part of the chat template and the `</think>` delimiter is the model's signal that thinking is complete.

This is the third TTC family of section 4: not N samples (sampling, section 3), not branching (search, section 6), but length. The two control surfaces exposed in production are:

1. **API-level thinking budgets.** Gemini 2.5 takes a `thinkingBudget` integer; OpenAI's o-series exposes `reasoning_effort` levels; Anthropic exposes a `thinking` block with token caps.[98] The user sets a target L; the serving stack honours it.

2. **Budget forcing.** A purely test-time edit to the decoding loop, requiring no API surface and no retraining (et al. , Stanford+, §3.1). We turn to it next.

[95] KB note: `kb/notes/reasoning/test-time-compute.md#2.3`.
[96] KB excerpt: `kb/excerpts/s1-2025.md#sec-3-1-budget`.
[97] KB note: `kb/notes/reasoning/test-time-compute.md#2.3`.
[98] KB note: `kb/notes/reasoning/test-time-compute.md#2.3` §1.

9.2 The s1 budget-forcing rule

et al. (Stanford+, §3.1) define budget forcing as two single-line edits to autoregressive decoding, parameterised by a minimum budget $L_{\min}$ and a maximum budget $L_{\max}$. Let ℓ denote the current trace length and let $\langle$/think$\rangle$ denote the end-of-thinking token. Then:

$$\text{(min)} \quad \text{if } \ell < L_{\min} \text{ and } \arg\max_{v} p_\theta(v \mid x, z_{1:\ell}) = \langle\text{/think}\rangle : \\ p_\theta(\langle\text{/think}\rangle \mid \cdot) \leftarrow -\infty, \quad z_{\ell+1} \leftarrow \texttt{Wait}, \tag{29}$$

$$\text{(max)} \quad \text{if } \ell \geq L_{\max} : \quad z_{\ell+1} \leftarrow \langle\text{/think}\rangle. \tag{30}$$

The min-rule (eq. (29)) suppresses the end-of-thinking token's probability and appends the literal string `Wait` to the trace when the model tries to stop too early; the max-rule (eq. (30)) hard-injects the end-of-thinking delimiter when the trace runs too long. The `Wait` insertion is the load-bearing piece: rather than a benign continuation cue, it is a *prompt* to re-examine the work just produced (et al. , Stanford+, §3.1).[99] The paper's verbatim description: "This can lead the model to double-check its answer, often fixing incorrect reasoning steps" (et al. , Stanford+, abstract).[100]

The recipe is then three lines:

1. Curate 1,000 (problem, long-CoT trace, answer) examples filtered by quality, difficulty, and diversity (et al. , Stanford+, §2.2).[101]
2. SFT a Qwen2.5-32B-Instruct base on these 1,000 traces ($\sim$ 7 H100-GPU-hours).
3. Apply budget forcing at test time per eqs. (29) and (30).

No reward model. No RL. No search tree. The budget-forcing rule itself is $\sim$ 10 lines of decoder code.

Intuition

Budget forcing lives on Snell's test-time-compute axis (section 4) but exercises it from the *sequential* side: where Snell varies N at fixed L, s1 varies L at $N = 1$. Operationally, `Wait` tokens are the sequential analogue of resampling — each `Wait` forces the model to revisit its own partial trace and emit a refinement, much as each new sample in self-consistency (section 3) draws a fresh attempt at the problem. Returning to math: the compute expended per problem is $C \propto L$ for sequential and $C \propto N$ for sampling, and the two axes can be swept independently. s1's ablation (et al. , Stanford+, §5.1)[a] shows the sequential axis is non-trivial in isolation: AIME 2024 lifts from 50.0% (SFT only, no budget forcing) to 56.7% (SFT plus budget forcing) on the same checkpoint. The lift is what budget forcing *adds* on top of SFT — the sequential TTC dial, exposed as a post-hoc decoder edit.

[a] KB excerpt: `kb/excerpts/s1-2025.md#sec-5-1-ablation`.

9.3 Logarithmic-ish returns to L

The s1 paper sweeps L through budget forcing and reports extrapolation *beyond* the SFT trace lengths (et al. , Stanford+, abstract, §4.2): pushing the minimum-thinking budget upward continues

[99] KB excerpt: `kb/excerpts/s1-2025.md#sec-3-1-budget`.
[100] KB excerpt: `kb/excerpts/s1-2025.md#abstract`.
[101] KB excerpt: `kb/excerpts/s1-2025.md#sec-2-2-curation`.

to lift accuracy past the lengths the model saw in training. The concrete numbers from Table 1 (et al. , Stanford+, §4.2):[102]

Model	AIME24	MATH500	GPQA Diamond
s1-32B (with budget forcing)	56.7%	93.0%	59.6%
o1-preview	44.6%	85.5%	73.3%
Qwen2.5-32B-Instruct (base)	26.7%	84.0%	49.0%

Table 2: s1-32B vs. o1-preview and the underlying Qwen2.5-32B-Instruct base (et al. , Stanford+, Table 1, §4.2).[103] On competition-math benchmarks (AIME24, MATH500), 1,000 SFT examples plus budget forcing exceeds o1-preview by +12.1 and +7.5 percentage points respectively; on GPQA Diamond, o1-preview retains a substantial lead, indicating that the recipe transfers to math-style reasoning more readily than to broader scientific QA.

The shape of accuracy as a function of L on the s1 curves is *monotone-concave* in $\log L$ over the range tested (et al. , Stanford+, §4.2); no closed-form scaling law is reported. This is consistent with the broader Snell-style framing of section 4 that posits inference-time compute as a scaling axis comparable to training compute, but with the functional form left empirical (Snell et al., 2024, §1). The headline ablation (et al. , Stanford+, §5.1) additionally controls for the *interaction* between SFT data scale and the budget-forcing rule: training on the full 59,000-example pool reaches 53.3% AIME 2024 in $\sim$ 394 H100-GPU-hours, while the curated 1,000 plus budget forcing reaches 56.7% in $\sim$ 7 H100-GPU-hours.[104] Curation does most of the SFT-side work; budget forcing extracts the rest at inference.

Contradiction

The s1 thesis — 1,000 examples plus budget forcing *suffices* for o1-preview-level competition math — is in explicit tension with the DeepSeek-R1 framing (section 8) that *RL on top of SFT* is required for the strongest performance (DeepSeek-AI, 2025a, §2.2.4).[a] The clearest reconciliation in the 2025–26 literature: SFT on curated traces gets the model into the right regime cheaply; RL refines within that regime but cannot create the regime from scratch on a base that lacks the latent capability. s1 operates within the regime; R1 grows the regime via outcome-reward RL. Both can be right because they target different points on the training-compute axis.

[a] KB note: `kb/notes/reasoning/test-time-compute.md#3` flags this contradiction explicitly.

9.4 Inference-time discipline vs. training-time discipline

Budget forcing and R1's cold-start SFT (section 8) are *both* mechanisms that shape the same `<think>` channel, but at different stages of the pipeline. The distinction is load-bearing for cross-paper bookkeeping:

- **Budget forcing is inference-time.** eqs. (29) and (30) change the decoding loop, not the weights. The model is whatever the SFT step left behind; the rule is a post-hoc dial on L at serve time. This is the same category as Gemini's `thinkingBudget` or OpenAI's `reasoning_effort` — a knob exposed to the user or the serving stack, not baked into θ.

[102] KB excerpt: `kb/excerpts/s1-2025.md#sec-4-2-table`.
[103] KB excerpt: `kb/excerpts/s1-2025.md#sec-4-2-table`.
[104] KB excerpt: `kb/excerpts/s1-2025.md#sec-5-1-ablation`.

- **Cold-start SFT is training-time.** R1's Stage 1 (DeepSeek-AI, 2025a, §2.3) fine-tunes the base model on long-CoT exemplars before any RL, ensuring readable traces and a starting point that matches the chat template's `<think>` ... `</think>` format.[105] It changes θ. Whatever distribution-shape the base model has after Stage 1 is what every later mechanism — Stage 2 GRPO, Stage 3 rejection-sampling SFT, and any inference-time budget rule — operates on top of.

The two compose. s1 exhibits the inference-time leg in isolation by running on a Qwen2.5-32B-Instruct base whose pretraining and instruction tuning supply the latent reasoning capability (et al. , Stanford+, §5.1). R1 exhibits the training-time leg in isolation through R1-Zero, where GRPO from the base model produces the trace-lengthening behaviour without any cold-start SFT (DeepSeek-AI, 2025a, §2.2.4). The frontier-2026 picture (section 13) is models that ship *both* — training-time cold-start to seed the regime, then inference-time budget exposure as a productionised dial.

Intuition

The architecture taxonomy of Paper 1 (Paper 1 §reasoning-architectures) treats reasoning as an architectural element by asking what changes in the Transformer when it must emit and consume long traces: long-context attention, KV-cache reuse, position-encoding behaviour at extrapolated lengths. The training taxonomy of Paper 2 (Paper 2 §SFT and §post-training) treats it as a training-pipeline element: which datasets, which losses, which RL stages produce the trace- emitting policy. *Reasoning-architecture*, in the sense introduced by s1 and R1 jointly, is a third axis: the training-and-inference *protocol* that designates `<think>` ... `</think>` as a structurally distinct phase, separates thinking-token budgets from answer-token budgets, and exposes L as a serve-time control surface. Returning to math: the underlying model is the same $\theta \in \mathbb{R}^{|\theta|}$ as in Paper 1; the protocol is a constraint on the input-output trajectory (x, z, y) that turns $L = |z|$ into an additive compute axis on top of the architectural and training axes.

9.5 Reasoning-architecture as a third axis

The cross-paper claim worth stating sharply: budget forcing is the *simplest possible* demonstration that reasoning-architecture is a distinct degree of freedom from base architecture and from training recipe. The same Qwen2.5-32B-Instruct base that table 2 reports at 26.7% AIME 2024 zero-shot lifts to 50.0% under SFT-on-1K-traces and to 56.7% under SFT plus budget forcing (et al. , Stanford+, §4.2, §5.1). The base architecture did not change. The training-data scale changed by a factor of ~ 30 relative to the 59K-pool ablation but only marginally relative to instruction tuning. The reasoning-architecture — the chat-template separation of thinking from answering, plus the rule that lets a serve-time dial control how much thinking happens — absorbs the rest. The rest is ~ 30 percentage points of AIME 2024 pass@1.

This locates two cross-paper bookkeeping items:

1. In the architecture taxonomy of Paper 1 (Paper 1 §reasoning-architectures), reasoning shows up as long-context attention quality and KV-cache costs — the *enablers* of long-trace inference. The `<think>` bracket itself is a chat-template artifact, not an architectural primitive; it lives at the protocol layer.

[105] KB note: `kb/notes/reasoning/reasoning-training.md#2` (Stage 1).

2. In the training taxonomy of Paper 2 (Paper 2 §SFT, §post-training), reasoning shows up as the curated long-CoT SFT stage and the RLVR stage that follows (sections 7 and 8). The *format* of the traces (delimiters, line-by-line structure, inline verification) is part of what SFT installs; what RL does is push toward longer and more verifier-aligned versions of that format (DeepSeek-AI, 2025a, §2.2.2).

The third axis — the reasoning-architecture / serve-time-protocol layer — is where budget forcing, `thinkingBudget` APIs, `reasoning_effort` levels, and Anthropic's `thinking` blocks all live. None of them changes the model weights; all of them modify what L the model is allowed (or required) to spend.

9.6 Forward link

Budget forcing exposes L as a clean inference-time knob, but it leaves the central operational question of the reasoning triangle unanswered: at fixed total compute, when does spending marginal FLOPs on *more* thinking-token budget pay better than spending them on RLVR training, on parallel sampling (section 3), or on a verifier-guided search (section 6)? s1's headline suggests the trade-off is real — 7 GPU-hours of SFT plus a longer L at inference beats 394 GPU-hours of SFT at the same inference length (et al. , Stanford+, §5.1) — but the comparison is one slice through a many-dimensional surface. Section 10 picks up the search-vs-RL fixed-compute trade-off directly, transcribing Snell's FLOPs-matched curves (Snell et al., 2024, §6) for the inference-vs-pretraining axis and lining them up against DAPO's step-efficiency and R1's training-token costs for the inference-vs-RL axis. The thinking-token budget L that this section formalised becomes one of three FLOPs-bearing variables on that trade-off surface, alongside the parallel-sample count N of section 3 and the RLVR step count of section 7.

10 Search-vs-RL under fixed compute

Section 4 (the Snell anchor) settled one slice of the triangle's central operational tradeoff: at fixed total compute $C_{\text{train}} + C_{\text{test}}$, when does spending the marginal FLOP on inference-time search beat spending it on *pretraining*? The empirical answer is conditional on difficulty regime: on easy and intermediate prompts test-time compute substitutes for $\sim 14\times$ more pretraining FLOPs at matched accuracy (Snell et al., 2024, §6);[106] on the hardest tail it does not. Section 7 introduced a third allocation channel that did not exist when Snell's curves were drawn: RL on verifiable rewards (RLVR) at GRPO/DAPO scale, which spends FLOPs on *post-training* the proposer rather than on either pretraining the proposer or running it longer at inference. The triangle's central operational question therefore generalises: **at fixed total compute, when does the marginal FLOP go to (a) more inference search/sampling, (b) more RLVR training, or (c) more pretraining?** This section transcribes what is settled, isolates what is anecdotal, and demarcates the open frontier.

10.1 The three-channel allocation

Total compute spent to produce a deployed answer to a single prompt decomposes into three additive components:

$$C_{\text{total}} = \underbrace{C_{\text{pre}}}_{\text{pretraining}} + \underbrace{C_{\text{RL}}}_{\text{RLVR / GRPO post-training}} + \underbrace{C_{\text{test}}}_{\text{inference search/sampling}} . \tag{31}$$

[106] KB excerpt: `kb/excerpts/snell2024.md#sec-1-flops-matched`.

The three terms are amortised differently. C_{pre} is paid once per base model and amortises across every prompt the model ever sees. C_{RL} is paid once per post-training run and amortises across every prompt the post-trained checkpoint serves. C_{test} is paid *per prompt at inference* and does not amortise. The appropriate accounting for a budget question therefore depends on the deployment volume: a one-shot research evaluation amortises C_{pre} and C_{RL} over very few queries and is inference-cheap; a productionised consumer endpoint amortises both over 10^9+ queries and is inference-bound.

Snell's FLOPs-matched experiment of eq. (6) is the two-channel restriction $C_{\text{RL}} = 0$:

$$C_{\text{small}}^{\text{train}} + C^{\text{test}} \;\approx\; C_{\text{large}}^{\text{train}} + C_{\text{large}}^{\text{1-pass}}. \tag{32}$$

Snell et al. (2024, §6) compare a smaller base model running compute-optimal test-time strategies against a 14× larger base model running a single inference pass at matched total FLOPs. The match is between C_{pre} on the right and $C_{\text{pre}} + C_{\text{test}}$ on the left; the RL channel does not appear because PaLM 2-S (Codey) is not RLVR-trained (Snell et al., 2024, §4).[107] What section 7 added is a second axis of post-training compute that was not in the original FLOPs-matched menu.

Intuition

The folk gloss of eq. (31) is "where do the next million GPU-hours go?" The folk gloss is exactly right; the only subtlety is that the answer depends on the deployment volume and the prompt-difficulty distribution simultaneously. Returning to math: each of $C_{\text{pre}}, C_{\text{RL}}, C_{\text{test}}$ is a knob in eq. (31), and the joint argmax over the three at fixed C_{total} subject to a target accuracy on a target prompt distribution is the natural generalisation of Snell's eq. (3) from a θ-only argmax at fixed C_{test} to a $(\theta_{\text{pre}}, \theta_{\text{RL}}, \theta_{\text{test}})$ argmax at fixed C_{total}.

10.2 Snell's pretraining-vs-inference slice

The Snell two-channel result establishes the substitution direction between C_{pre} and C_{test} (Snell et al., 2024, §6).[108] At matched total FLOPs, the smaller-base-plus-test-time configuration (the left-hand side of eq. (32)) outperforms the 14×-larger-base configuration on easy and intermediate difficulty bins, breaks even or under-performs on the hard bin, and materially under-performs on the very-hard tail where $\widehat{\text{pass@1}}(q)$ on the small base is near zero. The substitution ratio is therefore not a single number; it is a *difficulty-conditional curve*, and the 14× headline is the easy-bin reading.

Two consequences for eq. (31) follow directly. First, on the easy/intermediate tail, marginal FLOPs are better spent on C_{test} than on C_{pre}. Second, on the very-hard tail, the order reverses: marginal FLOPs are better spent on C_{pre} than on C_{test}. The cross-over is benchmark-specific and base-model-specific; Snell et al. (2024, §6) report it for the MATH benchmark on PaLM 2-S, and the cross-over has not been re-measured on a frontier-2026 reasoning-trained base model (section 4.7).

10.3 The RL-vs-inference slice

The third channel of eq. (31) requires partial empirical answers from the RLVR literature, since no Snell-style FLOPs-matched study has been published for C_{RL} vs. C_{test} as of 2026-Q2. Two anchor data points exist.

[107] KB excerpt: `kb/excerpts/snell2024.md#sec-4-setup`.
[108] KB excerpt: `kb/excerpts/snell2024.md#sec-1-flops-matched`.

R1's training-token cost. DeepSeek-AI (2025a, §2.1)[109] report DeepSeek-R1-Zero training as 10,400 GRPO steps on $G = 16$ rollouts per prompt with maximum trajectory length 32,768 tokens (raised to 65,536 tokens after step 8,200). The training-time token budget is therefore order $10^4 \cdot 16 \cdot 3.3 \cdot 10^4 \approx 5 \cdot 10^9$ tokens of generation per epoch (rough), times 1.6 epochs (DeepSeek-AI, 2025a, §2.1). AIME 2024 pass@1 climbs from 15.6% on DeepSeek-V3-Base to 77.9% on R1-Zero (DeepSeek-AI, 2025a, §2.3);[110] self-consistency at inference adds a further 8.8% to 86.7% at $N = 64$ samples (DeepSeek-AI, 2025a, §2.3). The $N = 64$ inference branch costs ~ 64 forward-pass-equivalents per prompt; the GRPO training branch costs $\sim 5 \cdot 10^9$ tokens, amortised across every deployed query. For a deployment that serves $\gtrsim 10^8$ queries the per-query share of C_{RL} is below the per-query C_{test} at $N = 64$; for a one-shot evaluation it is not.

DAPO's step-efficiency. Seed (2025, §4)[111] report that DAPO reaches 50 points on AIME 2024 with the Qwen2.5-32B-base in roughly half the GRPO steps the DeepSeek-R1-Zero recipe would require to reach a comparable score. The four targeted modifications — clip-higher, dynamic sampling, token-level loss, overlong reward shaping — compose multiplicatively on the step-efficiency dimension (Seed, 2025, §3). From the perspective of eq. (31), DAPO halves C_{RL} at matched post-training accuracy. The substitution direction is therefore: a better RL recipe makes the RL channel cheaper without changing C_{pre} or C_{test}, and the saved C_{RL} is re-allocatable to either of the other two channels.

The composite R1 + self-consistency anecdote. The R1 empirics combine the second and third channels. Starting from the DeepSeek-V3-Base model (the C_{pre} channel, fixed), GRPO with verifiable rewards lifts AIME 2024 from 15.6% to 77.9% pass@1 (the C_{RL} channel, $\sim 10^4$ GRPO steps); self-consistency at $N = 64$ then adds 8.8% to 86.7% (the C_{test} channel, N-fold inference). Reading the curve as a sequence of marginal returns: the first units of C_{RL} deliver the largest accuracy gain (+62.3 percentage points from 15.6% to 77.9%); the first units of C_{test} on top of the trained model deliver an incremental gain of +8.8 points at the cost of 64× inference flops per query. The marginal-FLOP ranking is anecdotal and benchmark-specific, but the qualitative pattern — RL channel buys the bulk of the gain on math, inference channel buys the long tail of sample-efficient sharpening — recurs across the 2025-2026 reasoning-trained-model literature (DeepSeek-AI, 2025a, §2.3).

10.4 The identity that ties the channels

The two-channel eq. (32) generalises in the obvious way to three channels. Holding total compute fixed,

$$C^{\text{train}}_{\text{small}} + C^{\text{small}}_{\text{RL}} + C^{\text{test}} \;\approx\; C^{\text{train}}_{\text{large}} + C^{\text{1-pass}}_{\text{large}}, \tag{33}$$

where the left side is a smaller base model post-trained with RLVR plus inference-time search, and the right side is the 14×-larger un-RL'd base model running a single inference pass. Equation (33) is a budget bookkeeping identity, not a scaling law: it commits to no functional form for how accuracy depends on the three components on either side. The empirical question is whether (and on which prompt-difficulty bins) the left-hand configuration matches the right-hand accuracy. As of 2026-Q2 the three-channel version of the FLOPs-matched experiment has not been published; the two-channel restriction is Snell's, and the third channel's empirical content is the anecdotes of section 10.3.

[109] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-1-hp`.
[110] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime`.
[111] KB excerpt: `kb/excerpts/dapo2025.md#sec-4-headline`.

What a closed-form would look like. A Chinchilla-style joint frontier in the three channels would be a parametric loss

$$L(N, D, C_{\text{RL}}, C_{\text{test}}) \;=\; E \;+\; \frac{A}{N^{\alpha}} \;+\; \frac{B}{D^{\beta}} \;+\; \Phi(C_{\text{RL}}) \;+\; \Psi(C_{\text{test}}, q), \tag{34}$$

generalising Hoffmann and et al. (DeepMind, §3, Eq. 2)'s $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ with two additional terms Φ and Ψ capturing RL-channel and inference-channel returns. The functional forms of Φ and Ψ are unknown: et al. (Stanford+)'s budget-forcing experiment is suggestive of roughly logarithmic returns to thinking-token count for Ψ on a small reasoning-trained model (section 4.7), but no parametric fit with the structural status of Chinchilla's law has been attested for either Φ or Ψ. The cross-derivative $\partial^2 L / \partial C_{\text{RL}}\, \partial C_{\text{test}}$ — whether the RL and inference channels substitute or complement at the margin — is the empirical content section 10.3's anecdotes hint at, but no closed-form has been fitted.

10.5 Where the curves cross is benchmark-specific

The three-channel allocation question is not solvable in the abstract. The cross-over points among $\{C_{\text{pre}}, C_{\text{RL}}, C_{\text{test}}\}$ depend on at least four contingencies, each of which shifts the curves measurably:

1. **Prompt-difficulty distribution.** Easy prompts favour C_{test} (Snell's revision regime); intermediate prompts favour C_{test} in search form (best-of-N-with-PRM, beam, MCTS); the hardest tail favours C_{pre} (Snell et al., 2024, §6). RLVR training shifts the difficulty distribution itself: an RL-trained model's difficulty bins are not the base model's bins (DeepSeek-AI, 2025a, §2.3), which means the prompt-difficulty contingency interacts with C_{RL} rather than being orthogonal to it.

2. **Verifier availability.** The whole RLVR channel exists because a programmatic verifier can be applied at training time (Shao et al., 2024, §1.1). Domains without a verifier (open-ended generation, agentic tasks without a programmatic check) cannot spend the C_{RL} channel directly, so the three-channel allocation collapses to Snell's two-channel eq. (32). The frontier extension — LLM-as-judge, generative PRMs as soft verifiers (section 12) — is exactly the attempt to make C_{RL} available outside the verifiable domain at the cost of soft-verifier reward-hacking risk.

3. **Base-model checkpoint.** The substitution ratios are measured against a specific base model. PaLM 2-S on MATH gives 14× at the easy bin (Snell et al., 2024, §6); the same experiment on Qwen2.5-32B-base, on Llama 4, or on the still-classified o1-base would produce different ratios. Snell et al. (2024, §6) explicitly note their result depends on "the specific conditions on the pretraining and inference workload."

4. **Deployment-volume amortisation.** The per-query share of C_{pre} and C_{RL} scales as $1/(\text{queries served})$ while C_{test} does not amortise. For a research evaluation, C_{test} dominates the per-query budget; for a frontier production endpoint, C_{pre} and C_{RL} are nearly-free-at-the-margin and C_{test} is the binding constraint.

The four contingencies multiply, not add. A statement of the form "the optimal split between RL training and inference search is X:Y" is therefore meaningful only when scoped to all four: a specific prompt-difficulty distribution, a specific verifier availability, a specific base model, and a specific deployment volume. The 2025-2026 literature reports such scoped statements — the R1 anecdote of section 10.3 is one — but does not extract a universal exchange rate.

Contradiction

No closed-form three-channel tradeoff curve is attested. Snell et al. (2024, §6) establish the pretraining-vs-inference substitution at 14× on the easy/intermediate MATH bin under PaLM 2-S. DeepSeek-AI (2025a, §2.3) establish that GRPO on DeepSeek-V3-Base converts $\sim 10^4$ training steps into +62.3 points of AIME 2024 pass@1 with self-consistency adding a further +8.8 at $N = 64$. Seed (2025, §4) establish that DAPO halves C_{RL} at matched post-training accuracy. The three results are anchor anecdotes, each on a different base model and benchmark. No published study has run a three-channel FLOPs-matched experiment holding $C_{\text{total}} = C_{\text{pre}} + C_{\text{RL}} + C_{\text{test}}$ fixed and varying the allocation, and no parametric fit to eq. (34) with the structural status of Chinchilla's $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ exists. Whether a joint $(C_{\text{train}}, C_{\text{RL}}, C_{\text{test}})$ frontier with a clean closed form exists at all is open; the contingencies of section 10.5 suggest the answer may be "only within a fixed prompt-difficulty distribution, verifier setting, base model, and deployment regime," which is a much weaker statement than the universal Chinchilla-style law it would generalise. Re-visited in section 14.

10.6 Operating-point heuristics from the anchor anecdotes

Although no closed-form tradeoff is attested, the anchor results of sections 10.2 and 10.3 support three operating-point heuristics that are stable across the 2025-2026 reasoning literature:

- **If the verifier is available, spend on C_{RL} first.** The R1 anecdote shows the first units of GRPO buy the largest accuracy gain on math benchmarks (DeepSeek-AI, 2025a, §2.3).[112] The marginal return on C_{RL} exceeds the marginal return on either C_{test} (small inference samples on an un-RL'd base) or C_{pre} (Snell's 14× pretraining substitution on an un-RL'd base) until the policy saturates the verifier-detectable gain.
- **Once the policy is RL-trained, C_{test} delivers incremental sharpening.** Self-consistency on R1-Zero adds 8.8 points at $N = 64$ (DeepSeek-AI, 2025a, §2.3); the same algorithm on the un-RL'd base would not approach this number. The inference-channel return depends on the base distribution it operates on (sections 3 and 4), and the RL-trained distribution is sharper.
- **The very-hard tail wants C_{pre}.** For prompts the base model essentially never solves $(\widehat{\text{pass@1}}(q) \to 0)$, Snell et al. (2024, §6) report no inference strategy recovers the gap. RLVR cannot bootstrap on a G-group of trajectories that all score $R_i = 0$ either, because the group-normalised advantage of eq. (22) is zero by construction (Shao et al., 2024, §4.1); this is the failure mode DAPO's dynamic sampling addresses by discarding all-zero groups (Seed, 2025, §3)[113] but cannot solve at the level of the underlying capability gap. The very-hard tail therefore requires expanding the base distribution's support, which is the C_{pre} channel.

The three heuristics are scoped: they hold for verifiable-reward domains, for the prompt-difficulty bins where a verifier discriminates, and on the timescales where amortisation favours one-time training costs over per-query inference costs. None of them is a substitute for a closed-form tradeoff curve; they are operating-point recipes that the 2026 frontier deploys.

[112] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime`.
[113] KB excerpt: `kb/excerpts/dapo2025.md#sec-3-dynamic-sampling`.

Forum signal

The shorthand "RL is more compute-efficient than search" is sometimes cited as a derivable consequence of the R1 anecdote. It is not. The R1 anecdote shows that on *a specific base model and benchmark, with a specific verifier and a specific deployment-volume amortisation*, the first units of C_{RL} deliver more accuracy per FLOP than the first units of C_{test} on top of the un-RL'd base. The shorthand drops every contingency. Cited or repeated without scope, it should be treated as a discovery signal that RL buys a lot on math under verifiers, not as a numerical exchange rate transferable to non-verifiable domains, frontier base models, or research-scale deployments. Snell et al. (2024, §6) make the same point about the 14× substitution ratio.

10.7 Cross-paper and forward links

The three-channel allocation question is the operational form of the triangle's central tradeoff. Section 11 (CoT faithfulness) is structurally adjacent: it asks what causal work the C_{test} channel's reasoning trace actually does in producing the answer, which is upstream of any quantitative split between C_{RL} and C_{test} — if the trace is post-hoc rationalisation (Arcuschin et al., 2025, KB note),

deepen-cite: arcuschin2025-cot-wild §3, citation-pending

the inference-channel return is upper-bounded by what answer-only sampling could achieve. Section 8 (the R1 family, forthcoming) makes section 10.3's composite anecdote concrete by walking the four-stage R1 protocol; the empirics of C_{RL} in this section are taken from that protocol's first RL stage. Section 13 (frontier snapshot, forthcoming) catalogues the production combinations of the three channels deployed by R1, o1/o3, Gemini 2.5-thinking, Claude 3.7-extended-thinking, Qwen3-thinking, and QwQ; the snapshot is the empirical census the closed-form eq. (34) would have to fit. Section 14 (contradictions live and open frontiers) collects the open-frontier reading of this section's central contradiction — whether a joint $(C_{\text{train}}, C_{\text{RL}}, C_{\text{test}})$ scaling law exists — alongside the other five `[CONTRADICTION]` markers carried forward from sections 2, 6 to 8, 11 and 12.

The structural picture: eq. (31) is the budget identity; eq. (33) is its FLOPs-matched form; eq. (34) is the parametric law that would close the loop and currently does not exist. The triangle's three legs are individually well-characterised (sections 4, 7 and 8), but their joint frontier is the open research direction this section demarcates. The closing thesis of section 14 returns to this frontier as the most consequential of the paper's six live contradictions for the next model generation's compute-allocation decisions.

11 CoT faithfulness: does the trace cause the answer?

The decoding factorisation $z \sim p_\theta(\cdot \mid x)$, $y \sim p_\theta(\cdot \mid x, z)$ that eq. (1) introduces is a generative model: the trace is sampled, then the answer is sampled conditional on the trace. As a *statistical* dependence claim this is uncontroversial. The *causal* claim — that perturbing z propagates to a different y — is not the same statement, and the 2025–2026 wave of empirical work finds that the gap between the two can be substantial. **CoT faithfulness** is the operational name for the question: does the trace z in fact carry the information that determines y, or has y already been chosen by some other mechanism with z generated as a post-hoc rationalisation that happens to be human-readable? This section makes the question precise, summarises the three load-bearing 2025–2026 measurements (Arcuschin et al. "CoT in the wild", Shen et al. FaithCoT-Bench, Mehta et al. on whether scaling improves faithfulness), states the open contradiction with R1-style reasoning-training framings

(section 8), and draws the cross-paper threads: Paper 4's interpretability programme uses CoT as a probe surface and Paper 5's alignment programme treats CoT as a monitorable channel — both are compromised to the extent that traces are unfaithful.

11.1 Operational definition: from conditional to causal

Following the protocol shared across Arcuschin et al. (2025); et al. (2025b, 2026) and the KB note's framing,[114] the trace z is *faithful* to the answer y on input x when an intervention on z that should change the answer in fact does, and an intervention that should not does not. Formally, fix x and let $\mathcal{I} : z \mapsto z'$ be an intervention map. The trace is faithful with respect to $\mathcal{I}$ when

$$p_\theta(y \mid x, z) \;\neq\; p_\theta(y \mid x, z' = \mathcal{I}(z)) \tag{35}$$

under interventions $\mathcal{I}$ that change the load-bearing content of the trace (e.g., flipping an arithmetic step, contradicting an intermediate conclusion, deleting the chain), and is faithful in the opposite direction when eq. (35) fails to hold under interventions that should be answer-irrelevant (e.g., paraphrasing the trace into different words but the same content). The discrepancy between $p_\theta(y \mid x, z)$ and $p_\theta(y \mid x, z')$ — measured by an answer-distribution distance such as the answer-marginal total variation or the binary indicator $\mathbb{1}[\hat{y}(z) \neq \hat{y}(z')]$ — is the empirical surface of the faithfulness question.

Three intervention families dominate the 2025–2026 literature (et al., 2025b, §3):[115]

1. **Paraphrase z'.** Same content, different surface form. A faithful trace channel should leave the answer distribution unchanged; the failure mode is the model committing to a different y when an incidental phrase in z is swapped.

2. **Contradict z'.** Replace one or more steps with an explicit contradiction (e.g., flip an inequality, negate a numeric claim). A faithful trace channel should propagate the contradiction into a changed y; the failure mode is the answer surviving an internally-inconsistent trace.

3. **Truncate z'.** Drop the tail of z before the trace reaches its conclusion. A faithful trace channel should yield a substantially worse y; the failure mode is the answer being recoverable from the prompt x alone, with z's contribution to $p_\theta(y \mid x, z)$ approximating $p_\theta(y \mid x)$.

The third intervention is the cleanest operational test. If $\mathcal{D}(p_\theta(y \mid x, z), p_\theta(y \mid x))$ — some divergence between the trace-conditioned and trace-free answer distributions — is small, the trace is *not contributing* to the answer, regardless of how reasonable it reads.

Intuition

Faithfulness is a Granger-causality-like test on the trace channel: does conditioning on z *change* our prediction of y relative to conditioning on x alone, and do interventions on z propagate to y in the directions content-causality would predict? Returning to math: eq. (35) compares two answer distributions indexed by counterfactual traces z and $z' = \mathcal{I}(z)$, and the operational metric is a divergence between them. The Granger analogy is loose — Granger causality is a statement about temporal predictive precedence in linear time series, not about

[114] KB note: `kb/notes/reasoning/chain-of-thought.md#4`.

[115] Excerpt status: `kb/excerpts/shen2025-faithcot-bench.md` is abstract-only as of 2026-05;

deepen-cite: shen2025-faithcot-bench §3, citation-pending

generative counterfactuals — but the canonical machinery is the same: hold the prompt fixed, vary the candidate cause, and measure the response in the answer distribution.

11.2 Arcuschin et al.: CoT in the wild is not always faithful

Arcuschin et al. (2025) run the cleanest naturalistic-prompt measurement of unfaithful CoT to date.[116] The methodological contribution is to drop the artificial-bias scaffolding that earlier faithfulness work (e.g., adding a known-misleading hint to the prompt) used to elicit unfaithful traces, and instead measure rates on naturalistic, unbiased prompts — the regime production users actually inhabit. The headline per-model rationalisation rates: **GPT-4o-mini 13%**, **Haiku 3.5 7%** on production non-thinking models; **Sonnet 3.7 thinking 0.04%**, **Gemini 2.5 Pro 0.14%**, **R1 0.37%** on thinking-on configurations (Arcuschin et al., 2025, abstract).[117] The non-thinking-vs-thinking gap is roughly two orders of magnitude in measured unfaithfulness rate and is the empirical anchor of the faithfulness scaling question section 11.4 returns to.

A second contribution is the diagnosis of *unfaithful illogical shortcuts* on hard math problems: the model produces a long, plausibly structured trace whose intermediate steps are individually defensible but whose chain does not in fact derive the emitted answer; the answer appears to be reached by a separate route the trace does not document. (Arcuschin et al., 2025, §4).[118] This is the failure mode that breaks the simplest reading of eq. (1): the conditional $y \sim p_\theta(\cdot \mid x, z)$ is well-defined, but the de-facto computation of y does not flow through the trace tokens.

11.3 Shen et al. FaithCoT-Bench: instance-level measurement

et al. (2025b) promote faithfulness measurement from a per-distribution rate to a **per-instance** discriminative problem.[119] Their FINE-CoT dataset contributes $>$ 1,000 expert-annotated trajectories from four LLMs across four domains, with $>$ 300 labelled unfaithful, against which they benchmark eleven detectors covering three paradigms:

- **Counterfactual detectors** apply intervention maps $\mathcal{I}$ from section 11.1 and measure the answer-distribution shift; an unfaithful trace is one where content-changing interventions fail to change y (et al., 2025b, §3).
- **Logit-based detectors** read internal model signals (token-level entropies, attention to trace tokens vs. prompt tokens) to predict per-instance unfaithfulness without explicit intervention (et al., 2025b, §3).
- **LLM-as-judge detectors** present the trajectory to a separate language model and ask whether the trace's stated reasoning in fact derives the emitted answer (et al., 2025b, §3).

[116] Excerpt status: `kb/excerpts/arcuschin2025-cot-wild.md` is abstract-only as of 2026-05;

deepen-cite: arcuschin2025-cot-wild abstract, citation-pending

[117] KB note: `kb/notes/reasoning/chain-of-thought.md#4.1`.

[118]

deepen-cite: arcuschin2025-cot-wild §4, citation-pending

[119] Excerpt status: `kb/excerpts/shen2025-faithcot-bench.md` is abstract-only as of 2026-05;

deepen-cite: shen2025-faithcot-bench abstract, citation-pending

The headline empirical finding: detection is **harder in knowledge-intensive domains and on more advanced models** (et al., 2025b, abstract). The intuition is uncomfortable for the field's defaults: the very models whose trace-dependent answer quality is best are the ones whose unfaithfulness is hardest to detect mechanistically, because their traces are surface-coherent enough that LLM-as-judge fails and their internal logit signal does not visibly collapse on the unfaithful instances.

A second methodological point of FaithCoT-Bench is that *instance*-level measurement, not aggregate rate, is the load-bearing object. A model with average rationalisation rate 0.14% (et al., 2026, §4) could still be unfaithful on the specific subpopulation of prompts where the user actually depends on trace transparency (high-stakes decisions, monitorable-channel deployments — Paper 5's alignment surface). Aggregate measurement underweights tail risk.

11.4 Mehta et al.: does inference scaling improve faithfulness?

et al. (2026) ask the most consequential 2026 question for the paper's thesis: does *scaling* the inference-time compute axis (section 4) — specifically, self-consistency at $N > 1$ — *also* improve faithfulness, or are accuracy gains and faithfulness gains decoupled?[120] The study runs four frontier models (**GPT-5.2, Claude Opus 4.5, Gemini-3-flash-preview, DeepSeek-v3.2**) on 100 GSM8K problems at varying self-consistency budgets and measures both task accuracy and a per-trajectory faithfulness score in parallel.

The headline result is non-monotonic: **inference scaling does not uniformly improve faithfulness** (et al., 2026, abstract). The Claude Opus 4.5 configuration provides the cleanest illustration: accuracy *decreases* from 78% → 74.3% while measured faithfulness *rises* from 0.270 → 0.891 at $N = 5$.[121] For eq. (1)'s reading of self-consistency this is a puzzle: more samples should average out per-sample noise in the *trace* channel, leaving the conditional $p_\theta(y \mid x, z)$ better-determined; but at the same time, the votes are over the *answer* channel, and the relationship between trace-channel quality and answer-channel quality is exactly what faithfulness measurement is testing. Mehta et al.'s finding is that the two channels move differently per model.

The operational implication: **self-consistency must be tested per-model before deployment** (et al., 2026, abstract) for any application that depends on traces being a faithful read of the model's computation. The framework's thesis — that inference compute, search, and verification are co-designed levers (sections 4 and 5) — picks up a fourth operational variable here: the *interpretability cost* of the inference-compute leg, which the 2026 evidence reports is not a free externality of accuracy.

11.5 The contradiction with reasoning-training framings

The clean read of DeepSeek-AI (2025a)'s GRPO outcome — long traces emerge from RLVR, accuracy rises in lockstep with trace length, truncating the trace destroys the gain — is that the trace is a *causal substrate*: the model is doing genuine compute on the trace channel and the answer is being read off the result. The Arcuschin and Mehta evidence complicates this read at two scales.

[120] Excerpt status: `kb/excerpts/mehta2026-faithfulness-scaling.md` is abstract-only as of 2026-05;
deepen-cite: mehta2026-faithfulness-scaling abstract, citation-pending

[121] KB index: `kb/index/papers.json` — abstract.
deepen-cite: mehta2026-faithfulness-scaling §4 (or wherever the per-model table lives in the PDF; section anchor pending population), citation-pending

Contradiction

Reasoning-training as causal-trace evidence vs. unfaithful-trace evidence. DeepSeek-AI (2025a, §2.2) report that RLVR training grows average response length from ~ 200 to $\sim 10{,}000$ tokens

deepen-cite: deepseek-r1 §2.2 trace-length growth curve, KB excerpt-anchor pending; current anchor covers the rule-based-reward passage

with AIME 2024 pass1 climbing 15.6% → 77.9%[a] a correlation that the simplest reading attributes to the trace's content driving the answer's correctness. Yet Arcuschin et al. (2025) find R1 itself exhibits a measurable rationalisation rate of 0.37% (Arcuschin et al., 2025, abstract) on naturalistic prompts, and the unfaithful-illogical-shortcut failure mode of section 11.2 is a regime where trace tokens are emitted in the right format but the answer is not in fact a function of them. et al. (2026)'s scaling-of-faithfulness result sharpens the tension: one frontier model's accuracy drops while its faithfulness rises under self-consistency, a pattern incompatible with the simple reading that traces are a causal substrate at all scales. The 2026 picture is regime-dependent and the question is unresolved; section 14 catalogues this alongside the other live faithfulness contradictions of sections 11.2 and 11.4.

[a] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime`.

A second sub-tension within the empirical wave itself: the per-rate gap of two orders of magnitude between non-thinking and thinking configurations (Arcuschin et al., 2025, abstract) cannot be cleanly attributed to long-CoT SFT, RLVR post-training, or general scale by published 2025–2026 ablations (et al., 2026, §4); the controlled ablation that varies one of these factors at fixed others has not been run at frontier scale.[122] The reason matters: if the gap is RLVR-driven, it is benchmarkable and tunable through the training pipeline of section 7; if it is scale-driven, faithfulness is on a separate scaling curve from accuracy, and Mehta et al.'s non-monotonic per-model results are the first signal of where the two curves cross.

11.6 Cross-paper consequences: probe surface and monitor channel

The faithfulness question lands twice in later papers, and §11's resolution shapes both.

Paper 4 — CoT as a probe surface. The interpretability programme of Paper 4 reads internal computation through several methods: logit lenses, linear probes, sparse autoencoders, activation patching, and circuit tracing. CoT traces are the *output-side* probe surface for that programme — they are human-readable, token-aligned with the residual stream, and stable under repeated decoding. The faithfulness question gates this use: **if traces are unfaithful, the probe surface is decoupled from the underlying computation**. Paper 4's interpretability programme returns to this section as a precondition for treating CoT-as-probe as a primary methodology, and uses Mehta et al.'s per-model variation to argue that probe-surface validity must be re-established model-by-model rather than assumed from the family.

Paper 5 — CoT as an alignment monitor. The alignment programme of Paper 5 treats the CoT channel as a *monitorable surface*: detectors that read traces look for warning signs of misalignment, scheming, sycophancy, and alignment-faking before the answer is committed. The canonical 2024 result on alignment-faking (et al. , Anthropic / Redwood Research) demonstrates

[122] KB note: `kb/notes/reasoning/chain-of-thought.md#4.1`.

that a model can behave correctly on observable outputs while internally pursuing a divergent objective; if the trace channel is unfaithful in eq. (35)'s sense, this monitoring approach has a blind spot exactly where it most needs to see. Paper 5's §alignment-faking cites forward to this section as the empirical bound on CoT-as-monitor: a 0.04–0.37% unfaithfulness rate on thinking models (Arcuschin et al., 2025, abstract) is small in aggregate but consequential at scale, and the per-instance harder-on-advanced-models finding of et al. (2025b) is the most direct warning that the channel degrades exactly in the regime where alignment monitoring matters most.

The shared structural point across both papers: *aggregate unfaithfulness rates do not bound deployment risk*. The instance-level distribution matters, and the 2025–2026 evidence is that the unfaithful instances cluster in knowledge-intensive domains and on advanced models (et al., 2025b, abstract) — the same regime where downstream uses of the trace channel are most consequential.

11.7 Forward link: does the verifier signal still matter?

The structural adjacency to section 12 is direct. Both sections ask the same shape of question: *has the post-2024 training pipeline absorbed a function the field used to assign to an external module?* For verifiers (PRMs), the question is whether process-reward signal still earns its inference and training cost once RLVR-trained policies internalise step-level verification. For traces, the question is whether the CoT channel still earns its interpretability and monitoring premium once the policy has been RL-trained to optimise the answer-channel directly. The 2026 evidence, in both cases, is that the absorption is partial and regime-dependent: PRMs continue to add value at inference time (sections 5.4 and 12) while their training-time value shrinks, and traces continue to drive answers on the on-distribution majority of instances while their reliability degrades exactly on the harder, higher-stakes tail. Section 14 catalogues both contradictions as twin live frontiers, and the closing thesis of the paper — section 4's triangle is co-designed in 2026 — must inherit the qualifier that two of its three legs are operationally sound but interpretively contested. Section 12 carries the verifier half of the question forward; this section's evidence is the trace half.

12 Process supervision after R1: do PRMs still help?

Section 5 introduced the verifier leg of the reasoning triangle on a 2023 generator: PRM-best-of-N dominates ORM-best-of-N at matched N on PRM800K-grade verifiers, with min-aggregation as the strongest aggregator and PRM-weighted self-consistency as a budget-friendly compromise (Lightman et al., 2023, §4). Two years later, the strongest open-weight reasoning model — DeepSeek-R1 — reached its headline AIME 2024 jump using *terminal verifier rewards only* and explicitly abstained from any neural reward model, whether outcome- or process-based, in the RL loop (DeepSeek-AI, 2025a, §2.2).[123] The question this section confronts is the one section 5 deferred: in the post-R1 era, do PRMs still earn their training and inference cost?

The 2025–2026 picture is mixed. PRMs continue to add value at *inference* time in a number of measured settings (best-of-N reranking, search guidance) on reasoning-trained policies; their value as a *training* signal beyond verifier-only RLVR is contested, with both diminishing-returns evidence and continued-gain claims surveyed by et al. (2025d).

deepen-cite: zheng2025-prm-survey §2–3 (excerpt skeleton-only as of 2026-05; PDF acquisition pending)

This section walks the evidence on both sides, formalises the generative-PRM dominance claim and its 2026 caveats, and frames the contradiction that section 14 catalogues.

[123] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-2-reward`.

12.1 R1 is the data point

DeepSeek-AI (2025a, §2.1, §2.2) train R1-Zero with the GRPO objective of eq. (21) under a rule-based reward:

$$r_i \;=\; \mathbb{1}[\,\text{verify}(x, o_i) = \mathsf{true}\,] \;+\; \alpha\,\mathbb{1}[\,\text{format}(o_i) = \mathsf{ok}\,], \tag{36}$$

where verify$(\cdot)$ is an answer-match check on math problems and a unit-test / compiler check on code, and format$(\cdot)$ scores adherence to the `<think>...</think><answer>...</answer>` schema. No PRM appears in eq. (36); no neural reward model of any kind appears in the loop. The DeepSeek authors are explicit about the choice[124] (DeepSeek-AI, 2025a, §2.2):

> *[W]e abstain from applying neural reward models — whether outcome-based or process-based — to reasoning tasks. This decision is predicated on our observation that neural reward models are susceptible to reward hacking during large-scale reinforcement learning.*

The headline empirics are well-known: AIME 2024 pass@1 from 15.6% to 77.9% during RL, and to 86.7% with self-consistency at inference time (DeepSeek-AI, 2025a, §2.3).[125] A reasoning model that does not place at the AIME human-competitor level cannot be explained by "the missing PRM"; the gap is already closed without one. This is the load-bearing data point that reframes the post-2023 PRM literature from "does process supervision beat outcome supervision?" (settled by Lightman et al. (2023) on the 2023 generator) to "is process supervision still net-positive on top of verifier-only RLVR?"

Intuition

RLVR can be read as ORM-with-a-perfect-verifier. The R1 reward eq. (36) is precisely the outcome reward $R^O_\phi(x, z, y) \in \{0, 1\}$ of eq. (9) with the learned classifier replaced by an exact symbolic check. Returning to math: where Lightman et al. (2023) compared learned-ORM-best-of-N against learned-PRM-best-of-N, R1 substitutes the learned ORM with a zero-error verifier and pushes the resulting binary signal through GRPO rather than through reranking. The ORM half of the 2023 comparison has been replaced by something strictly stronger; whether the PRM half still wins requires re-running the experiment on this new generator class.

12.2 Inference-time PRMs on reasoning-trained policies

The cleaner part of the 2025–2026 picture: PRMs as inference-time re-rankers and search guides on reasoning-trained policies. Two consistent findings appear across the surveyed literature.

Generative PRMs dominate discriminative ones on verifiable-step domains. Et al. (2025a) report that ThinkPRM, a generative verifier (eq. (16)) fine-tuned on 1% of PRM800K labels and self-generated synthetic verification traces, beats discriminative PRMs trained on the full PRM800K on ProcessBench, MATH-500, and AIME 2024 under best-of-N selection and reward-guided search; out-of-domain it beats the full-PRM800K discriminative baseline by 8% on a GPQA-Diamond subset and 4.5% on LiveCodeBench, and beats LLM-as-a-Judge by 7.2% on ProcessBench at matched verification-token budget (et al., 2025a, abstract).[126] The mechanism is

[124] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-2-reward`.
[125] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime`.
[126] KB excerpt: `kb/excerpts/thinkprm2025.md#headline`.

the one foreshadowed in section 5.4: the verifier's CoT can apply algebraic-verification, type-checking, and dimensional-analysis steps that a feed-forward classifier head cannot internalise from labelled examples alone (et al., 2025a, §5).[127] The 2025 PRM survey (et al., 2025d) echoes the framing while adding a domain caveat: generative PRMs win where each step has a verifiable certificate (algebraic equality, equation-of-motion balance, type compatibility) the verifier's CoT can actually apply.

deepen-cite: zheng2025-prm-survey §3 (excerpt skeleton-only as of 2026-05)

Reasoning-aware discriminative PRMs close part of the gap at lower inference cost. She et al. (2025) train a discriminative PRM that emits a short reasoning preamble before scoring; the architecture remains a verdict head but the input is enriched by a brief verifier-reasoning prefix. The reported gain is distribution-shift-robustness rather than peak-benchmark performance.

deepen-cite: she2025-r-prm §3 (excerpt skeleton-only as of 2026-05)

Operationally, this collapses the eq. (16)-vs-eq. (7) dichotomy of section 5.4: as the preamble length L_{preamble} grows, R-PRM converges to ThinkPRM; as it shrinks to zero, R-PRM recovers the discriminative classifier. The cost profile for best-of-N scales as $O(N \times L_{\text{preamble}})$, intermediate between the $O(N)$ classifier and the $O(N \times L_{\text{verify}})$ generative verifier. Et al. (Huawei) add an orthogonal axis (entropy-driven step boundaries) which we treat in section 5.5; both threads belong to the same finding — the post-2023 PRM landscape is more multi-axial than the PRM800K-grade-classifier baseline implied.

The *combined* reading: at inference time, the strongest 2026 PRMs are generative, are trained with two orders of magnitude fewer human labels than PRM800K (offset by self-generated synthetic verification traces), and continue to outperform vanilla discriminative PRMs on competition math benchmarks. Inference-time process supervision did not vanish post-R1; it shifted form.

12.3 Training-time PRMs after R1: where the contradiction lives

The training-time picture is the contested one. Three positions are attested in 2025–2026.

Position A: PRM-shaped GRPO is theoretically attractive but empirically risky. The middle ground of section 5.5 (and eq. (36)'s sparse-binary critique) is the *denser* reward of substituting or augmenting r_i with a per-step or per-trace PRM score. Several R1-replication efforts try this approach (et al., 2025a, §4).[128] The known failure mode is *process hacking*: a misaligned PRM rewards locally-plausible-looking steps without the trace converging on the correct answer — the step-level analogue of the outcome-hacking failure mode that DeepSeek-AI (2025a, §2.2) cite as their reason for abstaining from neural reward models entirely. When the PRM aligns well, dense signal beats sparse signal for credit assignment; when it misaligns, the policy exploits the gap.

Position B: PRM-best-of-N over R1-class outputs still adds gains, but smaller ones. Et al. (2025a) report continued PRM-best-of-N gains over reasoning-trained policies on AIME 2024 and MATH-500; et al. (2025d) find the same pattern across surveyed measurements "until the base model becomes a strong reasoning model itself," at which point the marginal external-PRM gain shrinks.

deepen-cite: zheng2025-prm-survey §4 discussion of post-R1 measurements (excerpt skeleton-only as of 2026-05)

[127] KB excerpt: `kb/excerpts/thinkprm2025.md#sec-5`.
[128] KB note: `kb/notes/reasoning/process-supervision.md#4.2`.

The mechanism: as the policy's own internal verification — the self-reflection and "aha moment" behaviours of DeepSeek-AI (2025a, §2.3)[129] — approaches the PRM's signal, the verifier-fidelity gap that the PRM exploits closes. This does not zero out, but it does erode: the same external PRM that delivered 20-point pass@N gains over a 2023 GPT-4 generator on PRM800K-MATH delivers smaller gains over an R1-distill at matched N.

Position C: process-vs-outcome RL is the wrong question; the question is which signal is cheap to verify. The third position, implicit in eq. (36), is that the binary outcome reward of RLVR is cheap because the verifier is exact (an answer-match check, a compiler), not because outcome supervision is intrinsically superior to process supervision. Where exact outcome verification is available — competition math, code, formal logic — the choice between sparse-but-zero-error and dense-but-noisy is unbalanced; the binary verifier wins on robustness alone, even if dense signal would theoretically reduce credit-assignment noise. Where exact outcome verification is *not* available — open-domain QA, scientific reasoning, planning — neither RLVR nor PRM-shaped RLVR has a clean empirical answer in 2026. This reframing is consistent with the survey (et al., 2025d) but is sharper than its conclusion.

Contradiction

PRMs as a training-time signal beyond verifier-only RLVR — unresolved. The 2023 PRM800K-era result that PRM > ORM at matched N (Lightman et al., 2023, Fig. 4) was established on a non-reasoning-trained generator. As the policy class moves to RLVR-trained reasoning models, the ORM in that comparison is replaced by a zero-error symbolic verifier (eq. (36)); the PRM in that comparison must now beat the generator's own internal verification. Et al. (2025a) report inference-time gains; et al. (2025d) catalogue mixed empirics, with most PRMs improving test-time-compute performance "until the base model becomes a strong reasoning model itself." R1 itself ships with *no* PRM in the headline RL loop (DeepSeek-AI, 2025a, §2.2). The PRM-after-R1 question splits into two: (i) at *inference time*, generative PRMs (ThinkPRM, R-PRM) continue to add value; (ii) at *training time*, whether PRM-shaped RLVR beats verifier-only RLVR at matched compute is benchmark-specific and unsettled. Section 14 returns to this as the most consequential open contradiction of the verifier leg.

12.4 Process-vs-outcome RL: when does the per-step signal pay?

A useful book-keeping question: at fixed total RL compute, when does the per-step PRM signal pay for the labelling and verifier-inference cost? Three terms are in tension.

Labelling cost. Human PRM800K-style labelling is $O(\text{problems} \times T \times c_{\text{annot}})$, where c_{annot} is annotation cost per step. Math-Shepherd-style automatic labelling (section 5.3) collapses this to $O(\text{problems} \times T \times K \times L_{\text{rollout}})$ inference tokens; ThinkPRM-style 1%-PRM800K + synthetic-verification collapses it further to $O(\text{problems}_{1\%} \times T \times c_{\text{annot}} + \text{synthetic verification traces})$.

Verifier-inference cost in the RL loop. Per RL step, a PRM-shaped reward requires $G \times T$ verifier passes (one per step of each of G rollouts) at cost

$$C_{\text{PRM-RL}} = G \times \mathbb{E}[T] \times C_{\text{verify-step}}, \tag{37}$$

[129] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aha`.

versus the verifier-only RLVR cost of $G \times C_{\text{verify-final}}$, which for an answer-match check is essentially $C_{\text{regex}} \approx 0$. With a generative verifier, $C_{\text{verify-step}} \propto L_{\text{verify}}$ tokens; on a long-CoT trace with $T \in [10, 100]$ and $L_{\text{verify}} \in [10^2, 10^3]$ tokens, the PRM-RL verifier-inference cost can dwarf the policy-rollout cost itself.

Signal density. The benefit side. PRM-shaped GRPO replaces a single binary r_i with a T-vector of step scores; under a well-aligned PRM, the gradient identifies the worst step locally rather than back-propagating one outcome label across an entire 10^3–10^4-token trace, which is the credit-assignment story of section 5.1.

The pay-off condition is roughly that the credit-assignment gain exceeds the $G \times T \times L_{\text{verify}}$ verifier-inference overhead and the process-hacking risk. As of 2026-Q2 there is no published empirical curve that establishes the crossover point on a reasoning-trained generator at frontier scale; section 14 records this as an open frontier question.

Intuition

Process supervision is dense supervision, and dense supervision pays off when the labelling signal is reliable. The 2023 PRM800K demonstration was on human labels (very reliable, very expensive); 2024 Math-Shepherd was on rollout-completion-rate labels (cheap, moderately reliable); 2025 ThinkPRM is on a generative verifier's self-graded signal (cheap, reliability tied to the verifier's own quality). Returning to math: the PRM-shaped GRPO objective replaces $\hat{A}_{i,t} = (R_i - \mu)/\sigma$ from eq. (22) with a per-step advantage $\hat{A}_{i,t} = R^P_\phi(x, s^{(i)}_{1:t}) - \bar{R}^P$ that pays off in proportion to R^P_ϕ's alignment with ground truth. When alignment is high, dense beats sparse; when alignment slips, the policy hacks the gap.

12.5 What the post-R1 evidence actually says

A summary, scoped to what section 5's anchors plus the 2025–2026 PRM survey support:

1. **R1 reached AIME 2024 pass@1 of 77.9% with no PRM in the RL loop** (DeepSeek-AI, 2025a, §2.3). This single data point establishes that strong reasoning is achievable from verifier-only RLVR; the PRM is not a precondition.

2. **Inference-time generative PRMs continue to outperform discriminative PRMs** on verifiable-certificate benchmarks (et al., 2025a, abstract). The dominance shifts from a training-data-volume axis (PRM800K) to a verifier-compute axis (L_{verify}).

3. **The marginal value of an external PRM at inference time shrinks as the base model becomes a strong reasoning model itself** (et al., 2025d). Mechanism: the policy's internal verification approaches the PRM's signal, closing the verifier-fidelity gap.

   deepen-cite: zheng2025-prm-survey §4 discussion

4. **PRM-shaped RLVR has no clean empirical advantage over verifier-only RLVR at frontier scale as of 2026-Q2.** The R1 authors abstain from neural reward models on reward-hacking grounds; the 2025 survey reports mixed empirics.

5. **R-PRM and EDU-PRM** (She et al., 2025; et al. , Huawei) demonstrate that the discriminative-vs-generative split is not the only PRM design axis: reasoning preambles and entropy-driven step boundaries each yield gains at lower verifier cost than full generative

ThinkPRM, suggesting the design space is more multi-axial than the 2023–2024 literature implied.

deepen-cite: she2025-r-prm §3 and cao2025-edu-prm §3 (excerpt skeleton-only as of 2026-05)

What the evidence does *not* settle: whether the verifier leg of the reasoning triangle is best instantiated as a search-time PRM, a training-time PRM-shaped reward, or a verifier-only RLVR signal combined with self-consistency. The three options coexist in the 2026 frontier model line-up (section 13) without a consensus winner.

12.6 Forward link

The PRM-after-R1 question splits cleanly into an inference-time half (answered: generative PRMs continue to add value, with diminishing returns as the policy's own verification improves) and a training-time half (open: PRM-shaped RLVR vs verifier-only RLVR has no settled empirical winner). Section 13 provides the 2026 snapshot table of which frontier reasoning models actually ship PRMs — and the headline answer is "almost none in the headline RL loop, several at inference time." Section 14 then collects this section's contradiction alongside the CoT-faithfulness question (section 11), the RLVR-generalisation-vs-memorisation question (sections 7 and 8), and the search-vs-RL fixed-compute tradeoff (section 10) as the open frontier of the verifier leg.

13 Frontier-2026 snapshot: reasoning protocols

The previous twelve sections decompose reasoning into three legs — inference-time compute (sections 3, 4 and 9), verifier signals (sections 5, 6 and 12), and RL on verifiable rewards (sections 7 and 8) — and ask, leg by leg, what the literature settled and what remains contested. This section closes that decomposition by collapsing it into one cross-cutting view: which *currently-deployed* reasoning model ships which legs of the triangle, and at what budget. The intended use is the same as Paper 1's architecture-side frontier snapshot — a reference figure that lets a reader place any new model release on the design space the rest of the paper formalises. Because the underlying training and serving stacks move on the order of weeks, this snapshot is dated 2026-Q2 and explicit about which entries are tier-A primary sources, which are vendor system cards, and which are still discovery-tier signals pending paper-grade documentation.

The argument structure is: a single 2-page comparison table (table 3) that codes each model along the three legs; three short subsections grouping the lineup into R1-style, o-series-style, and hybrid-think families; an agreement-and-divergence summary; and a forward link into the contradictions catalogued by section 14. The single load-bearing claim of the section is structural rather than empirical: *every* frontier-2026 reasoning entry ships RLVR plus long-CoT, *none* of them ship a process reward model in the headline RL loop, and the productionised exposure of inference-time compute has converged on a small number of API-shaped "thinking budget" surfaces.

13.1 The 2-page comparison table

Table 3 summarises six reasoning entries that collectively span the open-weight, open-API, and closed-frontier ends of the 2026 lineup. The columns trace the three legs of the triangle plus two productionisation columns. The legend below the table specifies exactly what counts as "ships" for each leg.

[130] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2`.

Three reading conventions follow from the table. First, the "Verifier in RL loop" column is uniformly "none" or "not disclosed": no frontier-2026 entry that has published a system card identifies a PRM as a reward source in the headline RL loop. The R1 authors' explicit no-PRM commitment (DeepSeek-AI, 2025a, §2.2)[131] is the strongest attested form; the closed-weight entries decline to specify, which on this question is operationally indistinguishable from absence at the level of evidence available.

Second, the "Inference search" column is also uniformly "none in headline" or "not disclosed." The MCTS-and-PRM-guided lineage of section 6 is a research lineage; deployed reasoning models in 2026 ship long-CoT plus optional self-consistency on top. Whether that is because search does not pay at frontier scale, because the extra orchestration is a serving-cost burden, or because the PRMs that would guide it are not strong enough on reasoning-trained policies (section 12) is not separable from the published 2026 evidence base.

Third, the "Budget exposure" column is the column where the field visibly converges on an API shape: a discrete-level reasoning-effort selector

deepen-cite: openai-o-series — API reference

, a token-cap `thinkingBudget`

deepen-cite: gemini2-5 — API reference

, an extended-thinking toggle

deepen-cite: claude-3-7

, or a system-prompt think / non-think mode

deepen-cite: qwen3-thinking

. All four are the productionised form of θ in eq. (3) — the strategy choice the section-4 anchor formalises as the prompt-conditional argmax over compute spend.

13.2 R1-style: long-CoT + RLVR + verifier-only

The R1 family is the cleanest published form of the triangle: open weights, an open paper with explicit hyperparameters, a verifier-only RL loop (DeepSeek-AI, 2025a, §2.2), and an RL phase that drives average response length from $\sim$200 to $\sim 10^4$ tokens with no length term in the reward (DeepSeek-AI, 2025a, §2.3).[132] Section 8 walked the four-stage protocol; the family it seeded by 2026-Q2 includes Qwen's QwQ reasoning-preview

deepen-cite: qwq

and Qwen3-thinking

deepen-cite: qwen3-thinking; cf. Team and Alibaba (2025)

, plus the open-replication track Tülu 3 (et al. , AI2) and the Hugging Face OpenR1 community effort

deepen-cite: openr1 — community reproduction; pending bibkey

.

The structural commitment shared across this row of table 3 is doctrinal more than algorithmic: a binary verifier reward in eq. (20) is treated as the gold reward in its supported domains, and a learned reward model is treated as a reward-hackable proxy that the field has now opted out of for the reasoning subset of post-training. DeepSeek-AI (2025a, §2.2) state the position; the

[131] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-2-reward`.
[132] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aha`.

open-replication track inherits it; the closed-frontier rows of the table neither contradict it on the published evidence nor positively confirm it.

13.3 o-series: RL-trained thinking with proprietary mix

OpenAI's o1 (Sept 2024) and o3 successor lineage

deepen-cite: openai-o1 — system card; openai-o3 — system card and capability writeups

are treated by the published material as the productionised form of the same "RL on reasoning trajectories" family that R1 reproduces in the open. The system-card narrative is uniformly that "the more reinforcement learning (...) and the more time it spends thinking, the better it performs on reasoning tasks"

deepen-cite: openai-o1 — system card

; no PRM is named in the headline reward source; no inference-time search algorithm is named in the deployed serving path. The o-series exposes a discrete `reasoning_effort` parameter at the API level

deepen-cite: openai-o-series — API reference

that maps to a thinking-token-spend budget, the same operative knob as Gemini 2.5's `thinkingBudget` but discretised rather than token-quantised.

The reading available from the published material: the o-series and the R1 family are operationally close enough on the published axes (long-CoT yes, RLVR yes, terminal verifier in the publicly-disclosed reward, no headline PRM, no headline inference search) that the remaining differences are training-data composition, base-model scale, and proprietary post-training tooling. None of those three are visible in the 2026 system-card disclosure level. The cross-paper hook to Paper 1's architecture-side snapshot is symmetric: closed-frontier architecture rows are also dominated by "not disclosed" on the load-bearing axes, and the convergence-vs-divergence reading in this paper depends on the same disclosure ceiling.

13.4 Hybrid think / non-think with productionised budget knobs

The third 2026 productionisation pattern — and the operationally distinct one — is the hybrid model that ships *both* a non-thinking decode path and a thinking-decode path, switched at request time by a system prompt or API parameter. Three exemplars:

- **Gemini 2.5** (DeepMind, 2025) exposes a `thinkingBudget` parameter that maps to a hard cap on thinking-token spend per request, with cheap mode and expensive mode at the two ends of the budget axis

  deepen-cite: gemini2-5 — API reference

  . This is the most-explicit token-cap formulation of eq. (3) in deployment.

- **Anthropic Claude 3.7 (and the Claude 4 family)** exposes "extended thinking" as a request-time toggle alongside a budget knob

  deepen-cite: claude-3-7 — Anthropic system card; analogous treatment in the Claude 4 family

  . The non-extended-thinking decode path remains production-standard for latency-sensitive consumer endpoints; the extended-thinking path is invoked when the caller has a hard problem.

- **Qwen3-thinking**

  deepen-cite: qwen3-thinking — system card

  is the open-weight version of the same pattern: a single model that toggles between thinking and non-thinking modes via a system-prompt flag, with mode-tagged post-training that shapes both branches.

The shared structural commitment across the hybrid row of table 3 is that the productionisation surface acknowledges eq. (3) explicitly: the caller *is* the strategy chooser θ^*, and the API exposes the budget axis N as a first-class request-time parameter. This is the operational form of the difficulty-binning operationalisation of Snell et al. (2024, §3.2)[133] — hard-problem callers select expensive mode; easy-problem callers select cheap mode — pushed to the API edge.

Intuition

The hybrid budget knob is Snell's compute-optimal selection problem shipped to the caller. Equation (3) defines $\theta^*_{q,y^*(q)}(N)$ as a function of the prompt q; the deployed API treats θ as a parameter the *caller* sets per request, on the assumption that the caller has more information about the problem's difficulty than a difficulty-classifier that the model would otherwise have to learn. Returning to math: the expectation $\mathbb{E}_{y\sim\text{Target}(\theta,N,q)}[\mathbb{1}_{y=y^*(q)}]$ that section 4.1 maximises is now maximised by a two-player decomposition — the model fixes the policy $\pi_\theta(\cdot \mid q$; thinking budget $L)$, and the caller fixes L at request time. The compute-optimal $\theta^*(N)$ of Snell's formal statement is then the joint optimum the two players together approximate, with the API surface as the coordination mechanism.

13.5 Where the lineup agrees

Four cross-row agreements are stable across the 2026-Q2 published material:

1. **RLVR is universal at frontier.** Every reasoning row of table 3 reports an RL phase whose reward signal includes a verifiable correctness component on math, code, or both. The R1 row is the most explicit (DeepSeek-AI, 2025a, §2.2); the o-series and Gemini 2.5 rows attribute headline gains to "more reinforcement learning"

   deepen-cite: openai-o1; gemini2-5

   on reasoning trajectories. The DAPO-class RL improvements of section 7 (Seed, 2025)[134] are part of the same family deployed in some open-weight reproductions (et al. , AI2).

2. **Long-CoT is universal.** Every row deploys a `<think>...</think>`-style structurally-separate reasoning phase ahead of the answer. The pattern is no longer paper-grade research; it is the deployment standard. Section 8 and section 9 cover the training-time and inference-time mechanisms, respectively.

3. **PRMs are absent from headline RL loops.** The "Verifier in RL loop" column is empty in every row that has published a system card, and explicitly empty in the R1 row (DeepSeek-AI, 2025a, §2.2). The post-R1 question section 12 catalogues — whether PRMs survive as a *training* signal beyond verifier-only RLVR — is not answered by current frontier deployments; it is answered by their non-adoption.

[133] KB excerpt: `kb/excerpts/snell2024.md#sec-3-2-difficulty`.
[134] KB excerpt: `kb/excerpts/dapo2025.md#sec-2`.

4. **Thinking-budget exposure is the productionised form of the inference-compute axis.** Every row that exposes any inference-compute knob does so as a budget cap or discrete-effort parameter — the operational form of θ in eq. (3). The convergence on this surface is strong enough that a 2026 caller can transfer expectations about what "high reasoning effort" means across vendors, even though the underlying policies and base models are distinct.

13.6 Where the lineup diverges

Three load-bearing axes of divergence remain, all of them either not disclosed by closed-frontier vendors or actively contested in the open-replication literature.

Training-data composition. R1's pretraining mix is DeepSeek-V3's (∼14.8T tokens, mix not fully disclosed (DeepSeek-AI, 2025a, §2)); the o-series and Claude families do not publish their pretraining mixes; Gemini 2.5 publishes only summary characterisations

deepen-cite: gemini2-5

. The AIME-2024 contamination contradiction of section 8.5 interacts non-trivially with this axis: rows whose pretraining mix is undisclosed cannot be contamination-audited from outside, and the headline reasoning numbers on AIME-class benchmarks are conditional on the audit not having been done.

Reasoning-architecture details. Whether the model is a single dense Transformer trained with mode-tagged data

deepen-cite: qwen3-thinking

, a model with a separate thinking-decode path

deepen-cite: claude-3-7 extended thinking

, or a model with no public architectural documentation at all (the o-series) is not visible to a caller, and is operationally distinct from the training-protocol differences of section 8.1. Paper 1's reasoning-architecture axis is the architecture-side counterpart; this paper's row taxonomy in table 3 is the training-protocol-side counterpart.

Search vs. no-search at inference. The "Inference search" column is uniformly "not disclosed" or "none." Whether some rows in fact run undisclosed PRM-guided beam or MCTS rollouts in serving is not separable from the available evidence. The research literature section 6 continues to push search-and-PRM methods on benchmarks; the deployed lineup either does not adopt them or does not disclose adoption. A correct 2026-Q2 reading is that search at inference is currently a research-and-eval-stack lineage, not a deployment-stack lineage, with the deployment-stack reasons either unrelated to algorithmic value or undisclosed.

Contradiction

The disclosure ceiling and what it means for the snapshot. Table 3 reports the 2026-Q2 published material at face value: *published* system cards say "no PRM in headline reward," *published* reports describe long-CoT and RLVR, and *published* APIs expose budget knobs. None of those statements rule out the contrary on closed-weight rows. The R1 row is the only one where the "Verifier in RL loop = none" cell is positively attested rather than not-disclosed (DeepSeek-AI, 2025a, §2.2).[a] Two readings of the column-uniformity that follows are both compatible with the evidence: *(i)* the field has converged on verifier-only RL

plus long-CoT plus thinking-budget knobs as the dominant pattern, or *(ii)* the closed-frontier vendors are running undisclosed PRM or search machinery and the disclosure ceiling produces the apparent convergence as a measurement artefact. The 2026-Q2 published evidence does not discriminate; section 14 catalogues this as a load-bearing open contradiction whose resolution requires either disclosure changes or third-party reverse-engineering at frontier scale.

[a] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-2-reward`.

13.7 Cross-paper hook and forward link

Paper 1's frontier-2026 architecture snapshot covers the architecture-side analog of table 3: convergence on KV-sharing variants (GQA / MLA), RMSNorm with Pre-LN, RoPE with position-extension methods, SwiGLU FFNs, MoE at scale. The reasoning side of the same lineup — this section's table — exhibits the same structural pattern: a small number of dominant-deployment choices (verifier-only RLVR, long-CoT trace, thinking-budget API), a small number of contested axes (training-data composition, search-at-inference, reasoning-architecture details), and a long tail of vendor-specific proprietary tooling that the 2026 disclosure regime does not surface. The two snapshots together are the synthesis the rest of the series' section-by-section treatments build toward.

The forward link is to section 14, which catalogues the six load-bearing open contradictions of the paper: PRM utility post-R1 (section 12), CoT faithfulness scaling (section 11), RLVR generalisation versus memorisation (sections 7 and 8), search-vs-RL under fixed compute (section 10), AIME-2024 contamination (section 8.5), and MCTS-vs-beam at matched budget (section 6). The frontier-2026 snapshot is not a resolution of those contradictions — it is a coordinate system in which they are localisable. A reader who knows where a 2026 model sits in table 3 can read off which of the six contradictions bears most directly on its expected behaviour, and which of the disclosed and undisclosed axes most affect what a third party can verify about it. The headline of the snapshot is structural and sober: the triangle is real, the three legs co-design, the productionisation surface has converged, and the open questions about which combination is optimal at which (compute, latency, domain) operating point will not be closed-form for at least another model generation.

14 Contradictions live and open frontiers

The previous thirteen sections ran a leg-by-leg decomposition of the reasoning triangle and flagged tensions as they arose. This section collapses those tensions into one place. The role is not to relitigate each result but to catalogue *which* questions the 2026-Q2 literature has not closed and *what shape* a closure would have to take. Six contradictions are load-bearing for the rest of the paper's claims; each is sourced from the body `contradiction` blocks of sections 5 to 8 and 10 to 13 and from `kb/index/contradictions.md` § reasoning.

The structural reading: the three legs of the triangle are real and co-designed in frontier 2026 reasoning models. Where the literature disagrees is not on whether the legs exist but on *which combination* is optimal at which $(C_{\text{train}}, C_{\text{RL}}, C_{\text{test}})$ point and at which prompt-difficulty regime. None of the six contradictions has a closed-form answer in 2026-Q2, and none is on track to acquire one before the next model generation. The implication for the paper's thesis is sharp: the triangle is a coordinate system, not a tradeoff curve — and section 13's snapshot is the localisation, not the resolution.

14.1 The six contradictions

(C1) PRM utility post-R1. The PRM800K-era headline "PRM > ORM at matched N" (Lightman et al., 2023, §4)[135] was established on a non-reasoning-trained generator. Once the generator is itself an RLVR-trained reasoner, the comparison shifts: the ORM in the original setup is replaced by a zero-error symbolic verifier (section 8), and the PRM must now beat the policy's own internal verification. Et al. (2025a) report inference-time gains from generative PRMs over R1-distill outputs; et al. (2025d) catalogue the 2025 picture as "most PRMs improve test-time-compute performance until the base model becomes a strong reasoning model itself." R1 ships with no PRM in the headline RL loop (DeepSeek-AI, 2025a, §2.2). The unresolved question splits cleanly: PRMs survive as inference-time rerankers of best-of-N and as guidance signals for tree search (sections 6 and 12.2); their survival as an RL-loop *training* signal beyond verifier-only RLVR is contested (section 12.3). Closure requires a controlled PRM-shaped GRPO vs verifier-only GRPO comparison at matched post-training compute on a reasoning-trained base; no published 2025–26 study has run it.

(C2) CoT faithfulness scaling. Two empirical waves are in tension. Arcuschin et al. (2025) report a measurable in-the-wild rationalisation rate of 0.37% on naturalistic prompts for R1 (Arcuschin et al., 2025, abstract), plus an "unfaithful illogical shortcut" failure mode where trace tokens are emitted in the right format but the answer is not in fact a function of them (section 11.2). Et al. (2026) measure non-thinking versus thinking configurations across frontier models and find a per-rate gap of two orders of magnitude favouring thinking models, but they also report that on at least one frontier model accuracy drops while faithfulness rises under self-consistency (et al., 2026, §4)[136] — a pattern incompatible with the simple reading that traces are a causal substrate at all scales. Et al. (2025b) propose FaithCoT-Bench as a dedicated faithfulness benchmark (section 11.3). The unresolved question: does faithfulness improve, plateau, or degrade with model scale and reasoning-training intensity? The published 2026 ablations cannot separate long-CoT SFT, RLVR post-training, and general scale at fixed others (section 11.4); the controlled ablation has not been run at frontier scale, and Mehta's non-monotonic per-model results suggest faithfulness sits on a separate curve from accuracy. Closure requires a one-axis-at-a-time factorial at frontier scale, which the disclosure regime of section 13 currently precludes for the closed rows.

(C3) RLVR generalisation vs memorisation. The R1 narrative (DeepSeek-AI, 2025a, §2.3)[137] reads the 15.6%→77.9% AIME 2024 pass@1 climb as RLVR-elicited reasoning capability. Various (2025)

deepen-cite: rlvr-limits-2025 §3-§5, KB-excerpt-pending

read it differently: the group-mean baseline of GRPO (section 7.2) bounds RLVR's reach by the base policy's pass@K at *any* K, not by its pass@1, and so the climb is "the base model could already solve 77.9% of these problems at some sample budget; GRPO transferred that to pass@1." Under this reading RLVR *sharpens* the policy within base-model competency but does not *expand* the solvable set. The 2026 picture is contested rather than settled: the various reading is one camp's first-order synthesis (the trace-growth result of DeepSeek-AI (2025a, §2.2), response length $\sim 200 \rightarrow \sim 10{,}000$ tokens, is not reconciled with the capability-ceiling claim, and one cleanly-stated form of the camp's synthesis is that "RLVR teaches *trace-length use* (when to spend more tokens)

[135] KB excerpt: `kb/excerpts/lightman2023-prm800k.md#sec-4-2-aggregation`.
[136] KB note: `kb/notes/reasoning/chain-of-thought.md#4.1`.
[137] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime`.

without teaching new primitives" (various, 2025, §5)), but at least two camps have published 2025-Q3+ counter-evidence. Wen et al. (2025) introduce CoT-Pass@K — a metric that scores both the final answer and the chain-of-thought derivation — and report a consistent and significant gap between RLVR-trained and base models on AIME 2024/2025 across all K up to 1024, in contrast to standard Pass@K where the gap closes; this directly challenges the boundary-collapse reading on the metric most exposed to whether the chain itself is novel. Dong et al. (2025) explicitly frame Yue's findings as "capability boundary collapse" and propose hybrid-policy optimization that — on their reported runs — maintains an advantage over both the base model and GRPO as K grows; the field broadly agrees the boundary issue Yue diagnoses is real, but disagrees about whether it is intrinsic to RLVR or tractable under better optimisation. Whether either camp's synthesis exhausts the picture or merely captures the dominant term on its preferred benchmark is open. Closure requires either a published pass@K-vs-base-model audit at large K on R1's training distribution, or a controlled experiment where RL is run on a base whose pass@K on the target benchmark is provably zero at $K = 10^6$.

(C4) Search-vs-RL under fixed compute. Snell et al. (2024, §6) establish a 14× pretraining-vs-inference substitution on the easy/intermediate MATH bin under PaLM 2-S. DeepSeek-AI (2025a, §2.3) establish that GRPO on DeepSeek-V3-Base converts $\sim 10^4$ training steps into +62.3 AIME 2024 pass@1 points with self-consistency adding a further +8.8 at $N = 64$. Seed (2025, §4) establish that DAPO halves C_{RL} at matched post-training accuracy. Each of the three results is an anchor anecdote on a different base model, benchmark, and verifier; no published study runs a three-channel FLOPs-matched experiment holding $C_{\text{total}} = C_{\text{pre}} + C_{\text{RL}} + C_{\text{test}}$ fixed and varying the allocation, and no parametric fit with the structural status of the Chinchilla law $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ exists for the joint $(C_{\text{train}}, C_{\text{RL}}, C_{\text{test}})$ frontier (section 10.4). Whether such a clean closed form exists *at all* is the open frontier question. The contingencies of section 10.5 suggest the answer may be "only within a fixed prompt-difficulty distribution, verifier setting, base model, and deployment regime," which is a much weaker statement than the universal scaling law that would generalise Chinchilla to the inference side. The operating-point heuristics of section 10.6 are what is locally available; a global exchange rate is not.

(C5) AIME 2024 contamination. The headline R1 jump (section 8.5) attributes the 15.6%→ 77.9% AIME 2024 climb to RLVR-elicited reasoning. The competing reading: AIME-style problems and their solutions are widely indexed on the open web — problem-set archives, contest-prep textbooks, math forums, AoPS-style community posts — and *some fraction* of the pretraining-corpus distribution overlaps with the test distribution. Under that reading, part of the headline gain is recall of memorised solutions surfaced through long-CoT self-prompting, not novel reasoning composed under RL. The arxiv preprint of DeepSeek-AI (2025a) did not report a contamination audit on the AIME 2024 train/test boundary; standard contamination-detection methodologies (n-gram overlap on solutions, near-duplicate matching, decontamination-by-substring) all have known false-negative modes against post-hoc-paraphrased problems and arithmetic isomorphisms, and as of 2026-Q2 no *third-party* study has reproduced an audit on R1's pretraining mix (section 8.5). The contradiction interacts non-trivially with (C3): if the base model's pass@K at moderate K is already high because of contamination, then the various reading becomes near-tautological on this benchmark. Cross-paper: Paper 5 §contamination is the load-bearing treatment of the methodology debate itself — which methodologies disagree on what counts as contamination, and what contamination-robust evaluation looks like.

Update (post-2025-09). The *Nature* publication of R1 (DeepSeek-AI, 2025b) narrows but does

not close this contradiction. The peer-reviewed version adds (i) an n-gram-overlap decontamination audit on the pre- and post-training data of R1, and (ii) a control experiment that re-runs the GRPO recipe on Qwen2-7B — a base model released June 2024, before any advanced reasoner shipped — and reproduces similar capability emergence (DeepSeek-AI, 2025b). The Nature reviewer-response thread, published alongside the paper, also addresses the sister "distilled-from-OpenAI-outputs" concern. Two open seams remain after this update: a *third-party* (non-DeepSeek) audit on R1's pretraining mix, and a contamination-robust AIME-class benchmark adopted field-wide. The closure recommended in section 14.2 for (C5) therefore narrows to those two items rather than "any audit at all," but the deflationary reading of the AIME headline is no longer the only honest reading; the Qwen2-7B control is the strongest single piece of evidence against it in the published 2026-Q2 literature.

(C6) MCTS vs beam at matched budget. Snell et al. (2024, §5–6) report BFS-V (the level-order beam-with-PRM of section 6.2) competitive with or outright better than alternative search strategies at matched compute on MATH with PaLM 2-S as the base model (Snell et al., 2024, §5.2)[138]: the extra MCTS machinery (UCT bookkeeping, rollout-derived $Q(s)$, the PUCT exploration constant c_{puct}) does not necessarily pay for itself. Et al. (Microsoft) foreground MCTS as the central mechanism by which 7B SLMs match o1-preview, and the AlphaZero structural parallel of section 6.5 runs entirely through the UCT recurrence (section 6.3). Reconciling the two: Snell's evaluation uses a non-reasoning-trained base and a single round of search; rStar-Math co-trains MCTS with the policy and PRM across four rounds of self-evolution (et al. , Microsoft, abstract). The "MCTS" that wins in rStar-Math is a different object from the "MCTS" that ties in Snell. Whether the MCTS-vs-BFS gap reverses on reasoning-trained base models, on harder benchmarks, or under multi-round co-evolution is not closed in 2026; the et al. (2025c) taxonomy catalogues the disagreement without resolving it.

deepen-cite: wei2025-tree-search-survey §5 (resolution discussion; excerpt file pending)

Closure requires a search-strategy comparison at matched budget on a reasoning-trained base under multi-round co-evolution; published 2025–26 work runs at most two of those three controls simultaneously.

14.2 What closure would require

Contradiction

The six contradictions share a methodological obstacle. Every closure listed above is gated by an experiment whose 2026-Q2 analogue does not exist in the public literature: a controlled, matched-compute factorial on a frontier-scale reasoning-trained base, with the closed-frontier vendors disclosing enough of their training and inference pipeline to make a third-party replication possible. The disclosure ceiling of section 13 is the binding constraint: the published evidence base on R1 is the only frontier-row in table 3 where the absence of a PRM in the headline reward is positively attested rather than not-disclosed (DeepSeek-AI, 2025a, §2.2). Two readings of the apparent column-uniformity of frontier reasoning models are both compatible with the 2026-Q2 evidence: *(i)* the field has converged on verifier-only RL plus long-CoT plus thinking-budget knobs as the dominant pattern, or *(ii)* the closed-frontier vendors are running undisclosed PRM or search machinery and the disclosure ceiling produces

[138] KB excerpt: `kb/excerpts/snell2024.md#sec-5-2-search`.

the apparent convergence as a measurement artefact. The six contradictions are not just empirically open — they are open *conditional on* a disclosure regime that the field cannot unilaterally change.

The closures tabulated by contradiction: *(C1)* a PRM-shaped GRPO vs verifier-only GRPO comparison at matched post-training compute on a reasoning-trained base; *(C2)* a one-axis-at-a-time factorial separating long-CoT SFT, RLVR post-training, and general scale at fixed others, run at frontier scale and on a faithfulness-stable benchmark; *(C3)* a published pass@K-vs-base-model audit at large K on R1's training distribution, or RL on a base whose pass@K at $K = 10^6$ is provably zero on the target benchmark; *(C4)* a three-channel FLOPs-matched experiment varying the allocation of C_{pre}, C_{RL}, C_{test} at fixed C_{total}, with at least three base models and three benchmark families to separate base-model from prompt-distribution effects; *(C5)* a third-party contamination audit on R1's pretraining mix plus a contamination-robust AIME-class evaluation methodology established cross-paper in Paper 5; *(C6)* a search-strategy comparison at matched budget on a reasoning-trained base under multi-round co-evolution. None of the six closures is a closed-form derivation; all are empirical experiments whose run cost rivals frontier post-training. The 2026-Q2 reasonable expectation is that the closures will arrive piecemeal, on different benchmarks, with caveats that limit generalisation — not as a unified resolution.

14.3 What is settled

Intuition

The contradictions are about *margins*, not foundations. What the 2025–26 literature does establish is structural: the three legs of the triangle exist as separable mechanisms; each composes with the others; the productionised exposure of inference-time compute (thinking-budget knobs, reasoning-effort levels) has converged (section 13); and the empirical anchors on each leg (Snell et al., 2024; Lightman et al., 2023; Shao et al., 2024; DeepSeek-AI, 2025a; Seed, 2025; et al. , Stanford+) hold up across published replications. The triangle is real. The contradictions concern which combination is optimal at which point, not whether the legs themselves are real.

The settled-vs-contested split, leg by leg, parallels the agreement-and-divergence summary of section 13. Settled: *every* frontier-2026 reasoning entry ships RLVR plus long-CoT, *none* ship a process reward model in the headline RL loop, and the productionised exposure of inference-time compute has converged on a small number of API-shaped "thinking budget" surfaces. Contested: the contamination-vs-elicitation attribution of the AIME-class headlines (C3, C5), the form of the inference-time scaling law (the $L(\theta, N)$ functional form open question of section 4.7, partially overlapping with C4), the faithfulness behaviour of long-CoT under scale (C2), and the PRM-vs-verifier and beam-vs-MCTS implementation choices (C1, C6).

14.4 Closing thesis

The reasoning gains observed between 2022 and 2026 are the joint product of inference-time compute scaling, verifier signals, and RL on verifiable rewards. The empirical evidence that these three legs are co-designed — not three independent gains shipped in convoy — is strongest on the open-weight rows of table 3, weakest on the closed-frontier rows where the disclosure ceiling (DeepSeek-AI, 2025a,

§2.2)[139] restricts what can be positively attested. The six contradictions of section 14.1 are not anomalies to be explained away; they are the live frontier of the 2026 reasoning research programme.

The framing this paper has pushed throughout is that the productive 2026 question is not "is reasoning real" — the triangle's empirical effects on AIME, MATH, GPQA, and the agentic-benchmarks of section 13 are not in dispute — but "which combination of legs is optimal at which $(C_{\text{train}}, C_{\text{RL}}, C_{\text{test}})$ operating point, for which prompt-difficulty distribution, on which base model, under which deployment latency budget?" The 2025–26 literature has not closed any of those questions, and the closures listed in section 14.2 are empirical experiments of frontier-scale cost gated by a disclosure regime the field does not yet operate under. The reasonable forecast is that the answer will not be closed-form for at least another model generation. What the rest of the series can build on is the coordinate system this paper has named — inference compute, verifier signal, RL on verifiable reward — and the localisation of each open question on it. Paper 4 §faithfulness and Paper 5 §AIME-contamination both backreference this section's (C2) and (C5) directly; Paper 2's RLHF-to-RLVR transition treats (C3) as the hinge between learned and rule-based reward sources.

The triangle is real and co-designed in frontier 2026 reasoning models. The open questions are about margins and operating points, not foundations. They will outlast the next model generation.

Glossary

Activation patching the canonical causal-intervention method

Aha moment (R1-Zero) A spike in the model's use of reflection words like "wait" during training, taken as evidence of self-reorganization of reasoning structure '[deepseek-r1 §2.3]'.

Autoregressive Generating tokens one at a time, left to right, where each prediction depends only on previous tokens.

Budget forcing The s1 2025 mechanism. When the model emits its end-of-thinking token before the budget is reached, suppress it and append the continuation token "Wait" to force more reasoning; hard-truncate to enforce shorter budgets. (et al. , Stanford+, §3.2)

Capability ceiling (RLVR) The empirical finding that RLVR improves pass@1 on problems the base model could already solve at pass@k for some k, but does not expand the set of solvable problems. Set at pre-training. (various, 2025, §3)

Chain-of-thought (CoT) A prompting and decoding pattern in which the model produces an intermediate sequence of reasoning steps z before emitting a final answer y. In few-shot CoT (Wei 2022), the prompt contains exemplars with hand-written reasoning traces; in zero-shot CoT (Kojima 2022), the trigger phrase "Let's think step by step" elicits the same behaviour without exemplars. (et al. , Google Brain, §1; kojima2022 §3)

Circuit tracing Anthropic's 2025 methodology: replace MLPs

Claude Anthropic's proprietary decoder-only LLM family. No public architecture paper; structural family shares the decoder-only Transformer lineage.

Cold-start SFT A small-scale ($\sim 10^4$ examples) SFT pass on long-CoT exemplars, run before RL to fix readability problems of pure-RL outputs (R1-Zero). Does not change asymptotic capability but improves trace coherence. (DeepSeek-AI, 2025a, §2.2.4)

[139] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-2-reward`.

Constrained decoding Generation procedure that, at each step,

CoT faithfulness The extent to which the trace z causally drives the answer y versus being post-hoc rationalisation. Measured by counterfactual perturbation. Thinking models report rates 0.04–0.14

DAPO ByteDance's GRPO variant. Four modifications: clip-higher (asymmetric clip), dynamic sampling (drop zero-variance groups), token-level loss (vs sequence-level), overlong reward shaping. AIME 2024 50pts on Qwen2.5-32B. (Seed, 2025, §2)

Debate (AI safety) Two AI debaters in a zero-sum game judged

Decontamination (Phi-4 sense) Held-out test sets, especially November 2024 AMC math contests and unleaked HumanEval+, used to verify the synthetic data did not leak benchmark answers via the teacher. (et al. , Microsoft, §1.1)

discriminative PRM A classifier-style PRM: outputs a probability of step correctness in one feed-forward pass. Cheap to evaluate; classical Math-Shepherd / OpenAI PRM lineage. (Lightman et al., 2023, §2.2)

Feature a direction $\mathbf{d} \in \mathbb{R}^n$ in some

FlashAttention Exact attention computed with re-organized memory access: tile K, V into SRAM-resident blocks and use **online softmax** to combine block results, never materializing the $N \times N$ attention matrix in HBM. Forward + backward IO complexity $\Theta(N^2 d^2 M^{-1})$ vs. $\Theta(Nd + N^2)$ for the standard implementation '[dao2022 §3]'.

Format reward In RLVR, a small constant reward for trajectories that conform to a structural template (e.g., emit a '<think>...</think>' block). Keeps the chain-of-thought pathway alive during early training '[deepseek-r1 §2.2]'.

Generative PRM (ThinkPRM) A small reasoning-LLM verifier that emits its own short CoT evaluating each step before producing a verdict. Trained on 1

GPQA Diamond 198-question subset of GPQA (Rein et al. 2023) where two PhD experts agree on the answer and at least one non-expert validator with web access got it wrong. The "Google-proof" property is operationalized via the 34

GQA (Grouped-Query Attention) Interpolation between MHA and MQA: query heads are partitioned into g groups, each group shares one W^K, W^V. GQA-1 = MQA; GQA-h = MHA. KV cache per token is $2 n_g d_h$ elements '[ainslie2023 §2.2]'.

greedy decoding Selecting the highest-probability token at each step.

Group-normalized advantage In GRPO, $\tilde{A}_i = (R_i - \text{mean}(\{R_j\}))/\text{std}(\{R_j\})$, broadcast to every token of trajectory o_i. Replaces GAE-derived per-token advantages '[shao2024 §4.1; deepseek-r1 §2.1 Eq.3]'.

GRPO (Group Relative Policy Optimization) Critic-free PPO variant: replaces the value model with a group-mean reward baseline. Sample G trajectories per prompt, normalize rewards within the group, use the standardized advantage in the PPO loss. Used by DeepSeek-R1, Qwen 3, Tülu 3 RLVR '[shao2024 §4.1; deepseek-r1 §2.1 Eq.1]'.

GRPO (Group Relative Policy Optimisation) Critic-free PPO variant (Shao 2024). Per-prompt group of G samples; advantage is each sample's reward minus group mean, divided by group std. Eliminates the value-network requirement. (Shao et al., 2024, §4.1; deepseek-r1 §2.1)

Inference-time search TTC family that constructs and explores a tree (or DAG) of partial reasoning prefixes, using a verifier or value function to direct expansion. Beam / BFS / MCTS variants. (et al. , Microsoft, §3)

KV cache The per-layer cache of key/value tensors $K_{1:t}, V_{1:t}$ produced during autoregressive decoding so each new token's attention is $O(t)$ rather than $O(t^2)$. Size per token per layer is $2 \cdot n_{\text{kv}} \cdot d_h \cdot$ bytes. (Kwon et al., 2023, §2; kb/notes/inference/kv-cache-management#§1)

LLM Large Language Model. A neural network with billions of parameters trained on large text corpora to predict and generate language.

LLM-as-judge Evaluation pattern where a strong LLM scores another model's responses (e.g., AlpacaEval, MT-Bench, Arena-Hard). Introduces a contamination axis where the same model class scores its own outputs generously. Sidestepped by string-match (GAIA, FrontierMath) or executable verifiers (SWE-bench, OSWorld).

MCTS-RAG MCTS extended over retrieval decisions in addition to reasoning steps. Bridges search-based TTC and retrieval-augmented generation. [mcts-rag-2025, arXiv 2503.20757]

Multi-head Latent Attention (MLA) DeepSeek's architectural compression: project to a low-rank latent $C \in \mathbb{R}^{d_c}$ shared across heads, expand back to per-head (K, V) at attention time. Achieves 93

N-gram overlap Standard contamination-detection method (Brown et al. 2020 GPT-3 §C): flag training documents containing benchmark n-grams of length $\geq$13. Cheap; misses paraphrase contamination.

Outcome Reward Model (ORM) A scoring function $R_\phi^O : (x, z, y) \to [0, 1]$ trained on (prompt, complete solution, outcome correctness) triples. Sparse signal — one label per trace. (Lightman et al., 2023, §2.1)

PaLM Google's 540B-parameter decoder-only LLM (Chowdhery et al., 2022). Uses SwiGLU, RoPE, parallel attention/FFN layers. Reference point for scaling experiments.

Parameters The learned weights of the model (embedding matrices, projection matrices, normalization scales, etc.).

Pre-LN Modern placement: normalize before the sub-layer, then add residually. Gradients scale as $O(d\sqrt{\ln d/L})$, enabling stable training without warmup.

Precision pinning (DeepSeek-V3) Components held at BF16/FP32 even with FP8 elsewhere: embedding module, output head, MoE gating, normalization, attention. (DeepSeek-AI, 2024, §3.3.1)

PRM800K OpenAI's released dataset of $\approx$ 800,000 human step-correctness labels on GPT-4-generated MATH solutions. The foundational reference dataset for PRM training. (Lightman et al., 2023, §3)

Probe A (typically linear) classifier $g : \mathbb{R}^d \to \mathcal{Y}$ trained on frozen LM activations $\mathbf{h}^{(\ell,t)}$ to predict an external property y. The model's parameters are held fixed; only the probe is trained. Foundational to interpretability methodology '[alain-bengio-2017 §1]'.

Process Preference Model (PPM) rStar-Math's variant: trained on preferences over MCTS-discovered step pairs, rather than absolute step labels. Avoids naïve step-score annotation. (et al. , Microsoft, §3)

Process Reward Model (PRM) A scoring function $R_\phi^P : (x, s_{1:t}) \to [0, 1]$ trained on per-step correctness labels. Dense signal — one label per step. Outperforms ORM at matched best-of-N on MATH. (Lightman et al., 2023, §2.2, §4)

ProcessBench Standard benchmark for PRM step-error detection (identify the first incorrect step in a partial trace). [arXiv 2412.06559]

Query (Q) Projected representation of the position that is "asking" for information in attention.

R1-Zero DeepSeek-R1-Zero, the version of R1 trained with **no SFT cold-start** — pure GRPO RL applied directly to the DeepSeek-V3 base. Achieves AIME 2024 pass@1 of 77.9

reasoning-MCTS Monte Carlo Tree Search where actions are next-step reasoning generations and the value function is a PRM. Selection-expansion-simulation-backup loop, UCT-style exploration. (et al. , Microsoft, §3)

Reasoning model An LLM trained (typically with RL on verifiable rewards) to produce long-form chain-of-thought before its final answer. o1, DeepSeek-R1, Qwen3-thinking-mode, Gemini 2.5-thinking-mode are exemplars '[deepseek-r1; kb/index/papers.json#deepseek-r1]'.

rejection-sampling SFT Generate candidate solutions with the current policy, filter by verifier (and optional quality filter), SFT on the survivors. Used as Stage 3 of the R1 pipeline. (DeepSeek-AI, 2025a, §2.3.3)

Residual stream the linear data bus running through a

Reward hacking Policy exploits an artifact of the reward signal that does not correspond to true quality (e.g., verbosity, sycophancy, format-shortcuts). Distinct from overoptimization in that hacking can occur even at low KL '[deepseek-r1 §2.2]'.

reward model (RM, r_ϕ) A neural network trained on Bradley–Terry preferences to score response quality. Initialized from π^{SFT} with a scalar head replacing the LM head. Used as the reward signal for PPO '[ouyang2022-instructgpt §3.5]'.

RLHF (Reinforcement Learning from Human Feedback) The 3-stage post-training pipeline: SFT → reward-model training on pairwise preferences → PPO with KL anchor against π^{SFT}. Canonical reference is InstructGPT '[ouyang2022-instructgpt §3]'.

RLVR (RL with Verifiable Rewards) RL post-training where the reward is computed mechanically from the emitted answer (regex match for math, unit-test execution for code, proof-checker for formal proofs), not from a learned reward model. The defining feature of post-2024 reasoning training. (DeepSeek-AI, 2025a, §2.2.1)

RMSNorm (Root Mean Square Normalization) Simplified LayerNorm that drops mean-centering and bias, normalizing only by the RMS of the input. 7–64

RoPE (Rotary Position Embedding) Positional encoding that rotates query and key vectors by position-dependent angles, making attention dot products depend on relative position. Applied at every layer to Q and K only. Used by LLaMA and most modern LLMs.

Saturation A benchmark is saturated when frontier models score within the verifier-error-rate noise floor of one another, losing discriminative power. MMLU at >90

Scaling law An empirical or theoretical functional relationship between a measure of model performance (typically cross-entropy loss) and one or more "scale" inputs: parameters N, training tokens D, training compute C, or test-time compute N_{test}. Canonical reference: ‘kaplan2020‘.

Scheming A model schemes when, instructed to pursue a goal G,

Self-consistency Sample N chain-of-thoughts independently at $T > 0$; majority-vote the final answers. Wang et al. 2022. Cheap baseline that requires no verifier.

Sliding-window attention Each position attends only to the previous w tokens; compute $O(Nwd)$ vs $O(N^2 d)$. Local-only by construction; relies on stacked layers' growing receptive field for long-range dependencies. (Longformer 2020, Mistral 2023)

SwiGLU $\text{FFN}_{\text{SwiGLU}}(x) = (\text{Swish}_1(xW) \otimes xV)W_2$. Three-matrix FFN with Swish gating. Universal in modern decoder-only LLMs. (Shazeer, 2020, §2 Eq.6)

Sycophancy A response is sycophantic if it is more aligned with

Test-time compute (TTC) The inference-side compute spent on a single problem instance to improve answer quality, beyond a single greedy forward pass. Recognised in 2024 as a co-equal scaling axis with training compute. (Snell et al., 2024, §1)

Test-time compute-optimal scaling strategy The argmax over test-time hyperparameters θ of expected output correctness, at a fixed test-time compute budget N, conditioned on a prompt q: $\theta^*_{q,y^*(q)}(N) = \arg\max_\theta \mathbb{E}_{y\sim\text{Target}(\theta,N,q)}[\mathbb{1}_{y=y^*(q)}]$ ‘[snell2024 §3.1 Eq.(1); kb/excerpts/snell2024#sec-3-1]‘.

Thinking budget Productised TTC control: API-level integer (Gemini 2.5 ‘thinkingBudget‘), effort level (OpenAI o-series), or token cap (Anthropic thinking block). (DeepMind, 2025, §3)

Transformer Neural network architecture introduced by Vaswani et al. (2017) that replaces recurrence with self-attention, allowing parallel processing of sequences.

Value (V) Projected representation of the content to be aggregated. Weighted by attention scores to produce the output.

VAPO (Value-Augmented PPO) RL variant that re-introduces a learned value function on top of GRPO's design, with shared backbone and value-loss-clipping. Reportedly outperforms DAPO on long-CoT tasks ‘[vapo-2025]‘.

Verifiable reward A 0/1 reward computed by a programmatic check: $R(x, y) = \mathbb{1}[\text{verify}(x, y)]$. Sparse and terminal ‘[deepseek-r1 §2.2]‘.

VinePPO PPO variant using per-prefix Monte Carlo re-simulation as value estimate. Drops the value network. 3× wall-clock speedup over PPO on MATH/GSM8K ‘[vineppo-2024]‘.

Vocabulary (V) The fixed set of all tokens a model can recognize. Size ranges from 32K (LLaMA) to 50K (GPT-3).

References

Arcuschin, Janiak, Krzyzanowski, Rajamanoharan, Nanda, and Conmy. Chain-of-thought reasoning in the wild is not always faithful. arXiv 2503.08679 (v4 Jun 2025), 2025. URL https://arxiv.org/abs/2503.08679.

Google DeepMind. Gemini 2.5 technical report. arXiv (DeepMind Tech Report), 2025. URL https://arxiv.org/abs/2507.06261.

DeepSeek-AI. Deepseek-v3 technical report. arXiv (DeepSeek Tech Report), 2024. URL https://arxiv.org/abs/2412.19437. KB excerpt: kb/excerpts/deepseek-v3-training.md.

DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv (Nature 2025), 2025a. URL https://arxiv.org/abs/2501.12948. KB excerpt: kb/excerpts/deepseek-r1.md.

DeepSeek-AI. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. Nature 645(8081), 18 September 2025, 2025b. URL https://www.nature.com/articles/s41586-025-09422-z.

Dong, Jiang, Tao, Liu, et al. Rl-plus: Countering capability boundary collapse of llms in reinforcement learning with hybrid-policy optimization. arXiv:2508.00222, 2025. URL https://arxiv.org/abs/2508.00222.

Khalifa et al. Process reward models that think. arXiv, 2025a. URL https://arxiv.org/abs/2504.16828. KB excerpt: kb/excerpts/thinkprm2025.md.

Mehta et al. Does inference scaling improve reasoning faithfulness? a multi-model analysis of self-consistency tradeoffs. arXiv 2601.06423, 2026. URL https://arxiv.org/abs/2601.06423.

Shen et al. Faithcot-bench: Benchmarking instance-level faithfulness of chain-of-thought reasoning. arXiv 2510.04040, 2025b. URL https://arxiv.org/abs/2510.04040.

Wei et al. Unifying tree search algorithm and reward design for llm reasoning: A survey. arXiv 2510.09988, 2025c. URL https://arxiv.org/abs/2510.09988.

Zheng et al. A survey of process reward models: From outcome signals to process supervisions for large language models. arXiv 2510.08049, 2025d. URL https://arxiv.org/abs/2510.08049.

Wang et al. (AI2). Tülu 3: Pushing frontiers in open language model post-training. AI2 Tech Report, 2024. URL https://arxiv.org/abs/2411.15124.

Greenblatt et al. (Anthropic / Redwood Research). Alignment faking in large language models. arXiv, 2024. URL https://arxiv.org/abs/2412.14093. KB excerpt: kb/excerpts/greenblatt2024-alignment-faking.md.

Wei et al. (Google Brain). Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022. URL https://arxiv.org/abs/2201.11903. KB excerpt: kb/excerpts/wei2022-cot.md.

Cao et al. (Huawei). More bang for the buck: Process reward modeling with entropy-driven uncertainty. arXiv 2503.22233, 2025. URL https://arxiv.org/abs/2503.22233.

Abdin et al. (Microsoft). Phi-4 technical report. Microsoft Tech Report, 2024. URL https://arxiv.org/abs/2412.08905. KB excerpt: kb/excerpts/phi4.md.

Guan et al. (Microsoft). rstar-math: Small llms can master math reasoning with self-evolved deep thinking. arXiv, 2025. URL https://arxiv.org/abs/2501.04519. KB excerpt: kb/excerpts/rstar-math2025.md.

Muennighoff et al. (Stanford+). s1: Simple test-time scaling. arXiv, 2025. URL https://arxiv.org/abs/2501.19393. KB excerpt: kb/excerpts/s1-2025.md.

Hoffmann and Borgeaud et al. (DeepMind). Training compute-optimal large language models. arXiv, 2022. URL https://arxiv.org/abs/2203.15556. KB excerpt: kb/excerpts/hoffmann2022-chinchilla.md.

Hu, Zhao, Zhao, and Cohan. Mcts-rag: Enhancing retrieval-augmented generation with monte carlo tree search. In *EMNLP Findings 2025*, 2025. URL https://arxiv.org/abs/2503.20757.

Kaplan and McCandlish et al. (OpenAI). Scaling laws for neural language models. arXiv, 2020. URL https://arxiv.org/abs/2001.08361. KB excerpt: kb/excerpts/kaplan2020.md.

Kojima, Gu, Reid, Matsuo, and Iwasawa. Large language models are zero-shot reasoners. In *NeurIPS*, 2022. URL https://arxiv.org/abs/2205.11916. KB excerpt: kb/excerpts/kojima2022.md.

Kwon, Li, Zhuang, Sheng, Zheng, Yu, Gonzalez, Zhang, and Stoica. Efficient memory management for large language model serving with pagedattention. SOSP 2023, 2023. URL https://arxiv.org/abs/2309.06180. KB excerpt: kb/excerpts/kwon2023.md.

Lightman, Kosaraju, Burda, Edwards, Baker, Lee, Leike, Schulman, Sutskever, and Cobbe. Let's verify step by step. In *ICLR 2024*, 2023. URL https://arxiv.org/abs/2305.20050. KB excerpt: kb/excerpts/lightman2023-prm800k.md.

Ouyang, Wu, Jiang, Almeida, Wainwright, Mishkin, Zhang, Agarwal, Slama, Ray, Schulman, Hilton, Kelton, Miller, Simens, Askell, Welinder, Christiano, Leike, and Lowe. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022. URL https://arxiv.org/abs/2203.02155. KB excerpt: kb/excerpts/ouyang2022-instructgpt.md.

ByteDance Seed. Dapo: An open-source llm reinforcement learning system at scale. arXiv, 2025. URL https://arxiv.org/abs/2503.14476. KB excerpt: kb/excerpts/dapo2025.md.

Shao, Wang, Zhu, and et al. (DeepSeek-AI). Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv (DeepSeek Tech Report), 2024. URL https://arxiv.org/abs/2402.03300. KB excerpt: kb/excerpts/shao2024.md.

Shazeer. Glu variants improve transformer. arXiv, 2020. URL https://arxiv.org/abs/2002.05202. KB excerpt: kb/excerpts/shazeer2020.md.

She, Liu, Liu, Chen, Huang, and Huang. R-prm: Reasoning-driven process reward modeling. arXiv 2503.21295, 2025. URL https://arxiv.org/abs/2503.21295.

Snell, Lee, Xu, and Kumar (UC Berkeley / Google DeepMind). Scaling llm test-time compute optimally can be more effective than scaling model parameters. arXiv, 2024. URL https://arxiv.org/abs/2408.03314. KB excerpt: kb/excerpts/snell2024.md.

Qwen Team and Alibaba. Qwen3 technical report. arXiv (Qwen Tech Report), 2025. URL https://arxiv.org/abs/2505.09388.

various. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? In *NeurIPS 2025 (oral)*, 2025. URL https://arxiv.org/abs/2504.13837.

Wang, Wei, Schuurmans, Le, Chi, Narang, Chowdhery, and Zhou. Self-consistency improves chain of thought reasoning in language models. In *ICLR 2023*, 2022. URL https://arxiv.org/abs/2203.11171. KB excerpt: kb/excerpts/wang2022-self-consistency.md.

Wang, Li, Shao, Xu, Dai, Li, Chen, Wu, and Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In *ACL 2024*, 2024. URL https://arxiv.org/abs/2312.08935.

Wen, Liu, Zheng, Ye, Wu, Wang, Xu, Liang, Li, Miao, Bian, and Yang. Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base llms. In *ICLR 2026 (arXiv:2506.14245)*, 2025. URL https://arxiv.org/abs/2506.14245.

Zhang, Zhoubian, Hu, Yue, Dong, and Tang. Rest-mcts*: Llm self-training via process reward guided tree search. In *NeurIPS 2024*, 2024. URL https://arxiv.org/abs/2406.03816.

Model	Long-CoT	RLVR	Verifier in RL loop	Inference search	Budget exposure
DeepSeek-R1 (DeepSeek-AI, 2025a)	yes; $\sim 10^4$ tok traces (DeepSeek-AI, 2025a, §2.3)	yes; GRPO (DeepSeek-AI, 2025a, §2.1), terminal binary reward	none; explicit no-PRM commitment (DeepSeek-AI, 2025a, §2.2)	none in headline; self-consistency optional (DeepSeek-AI, 2025a, §2.3)	none (open-weights, fixed schema)
Qwen3-thinking	yes; toggleable via system prompt	yes deepen-cite: qwen3-thinking — system card pending in references.bib; cf. Team and Alibaba (2025) for the lineage and base	verifier-only in headline reports deepen-cite: qwen3-thinking	none in headline	think / non-think mode toggle deepen-cite: qwen3-thinking
QwQ	yes; reasoning-preview model	yes deepen-cite: qwq — Qwen's standalone reasoning preview, predates Qwen3-thinking; system card pending	verifier-only deepen-cite: qwq	none in headline	none (always-on thinking)
OpenAI o1 / o3	yes; thinking-first generation	yes (RL-class on reasoning trajectories per the system card; whether the reward is verifier-derived in the RLVR sense is not disclosed): "the more reinforcement learning (...) the more time it spends thinking" deepen-cite:	not disclosed; headline reports do not name a PRM deepen-cite: openai-o1	not disclosed	`reasoning_effort` ∈ {low, medium, high} deepen-cite: openai-o-series — API reference